client/src/components/DocumentCard.jsx

```
import { Link } from 'react-router-dom';

export default function DocumentCard({ document }) {
  // Типы документов на русском
  const typeNames = {
    drawing: 'Чертёж',
    standard: 'Стандарт',
    specification: 'Спецификация',
    instruction: 'Инструкция',
    manual: 'Руководство',
    other: 'Другое',
  };

  // Статусы на русском
  const statusNames = {
    draft: 'Черновик',
    pending_approval: 'На согласовании',
    approved: 'Утверждён',
    in_library: 'В библиотеке',
    archived: 'В архиве',
    withdrawn: 'Отозван',
  };

  // Цвета статусов
  const statusColors = {
    draft: 'bg-gray-100 text-gray-800',
    pending_approval: 'bg-yellow-100 text-yellow-800',
    approved: 'bg-green-100 text-green-800',
    in_library: 'bg-blue-100 text-blue-800',
    archived: 'bg-gray-100 text-gray-600',
    withdrawn: 'bg-red-100 text-red-800',
  };

  // Форматирование даты
  const formatDate = (dateString) => {
    return new Date(dateString).toLocaleDateString('ru-RU', {
      day: '2-digit',
      month: '2-digit',
      year: 'numeric',
    });
  };

  return (
    <Link to={`/documents/${document.id}`} className="block">
      <div className="card hover:shadow-lg transition-shadow cursor-
pointer">
        <div className="flex justify-between items-start mb-3">
          {/* Код документа */}
          <span className="text-sm font-mono text-gray-500">
            {document.code}
          </span>
          {/* Статус */}
```

```
          <span className={`px-2 py-1 rounded-full text-xs font-medium
${statusColors[document.status]}`}>
            {statusNames[document.status]}
          </span>
        </div>

        {/* Название */}
        <h3 className="text-lg font-semibold text-gray-800 mb-2 line-
clamp-2">
          {document.title}
        </h3>

        {/* Информация */}
        <div className="text-sm text-gray-600 space-y-1">
          <div className="flex justify-between">
            <span>Тип:</span>
            <span className="font-
medium">{typeNames[document.type]}</span>
          </div>
          {document.category_name && (
            <div className="flex justify-between">
              <span>Категория:</span>
              <span className="font-
medium">{document.category_name}</span>
            </div>
          )}
          <div className="flex justify-between">
            <span>Версия:</span>
            <span className="font-
medium">{document.current_version}</span>
          </div>
          <div className="flex justify-between">
            <span>Обновлён:</span>
            <span className="font-
medium">{formatDate(document.updated_at)}</span>
          </div>
        </div>

        {/* Теги */}
        {document.tags && document.tags.length > 0 && document.tags[0] &&
(
          <div className="mt-3 flex flex-wrap gap-1">
            {document.tags.map((tag, index) => (
              <span
                key={index}
                className="px-2 py-0.5 bg-gray-100 text-gray-600 rounded
text-xs"
              >
                {tag}
              </span>
            ))}
          </div>
        )}
      </div>
```

```
    </Link>
  );
}
```

client/src/components/Header.jsx

```
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../contexts/AuthContext';

export default function Header() {
  const { user, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate('/login');
  };

  const roleNames = {
    admin: 'Администратор',
    librarian: 'Библиотекарь',
    department_head: 'Руководитель отдела',
    technical_specialist: 'Технический специалист',
  };

  const hasRole = (roles) => {
    if (typeof roles === 'string') {
      return user?.role === roles;
    }
    return roles.includes(user?.role);
  };

  return (
    <header className="bg-white shadow-sm border-b">
      <div className="max-w-7xl mx-auto px-4 py-3">
        <div className="flex items-center justify-between">
          {/* Логотип */}
          <Link to="/" className="text-xl font-bold text-blue-600">
            Электронная библиотека НТД
          </Link>

          {/* Навигация */}
          <nav className="flex items-center space-x-6">
            {/* Поиск - НЕ для админа */}
            {user?.role !== 'admin' && (
              <Link
                to="/search"
                className="text-gray-600 hover:text-blue-600 transition-
colors"
              >
                Поиск
              </Link>
            )}
```

```
            {/* Избранное - НЕ для админа */}
            {user?.role !== 'admin' && (
              <Link
                to="/favorites"
                className="text-gray-600 hover:text-blue-600 transition-
colors"
              >
                Избранное
              </Link>
            )}

            {/* Ссылки для библиотекаря */}
            {hasRole('librarian') && (
              <>
                <Link
                  to="/upload"
                  className="text-gray-600 hover:text-blue-600
transition-colors"
                >
                  Загрузить
                </Link>
                <Link
                  to="/librarian"
                  className="text-gray-600 hover:text-blue-600
transition-colors"
                >
                  Управление
                </Link>
                <Link
                  to="/tickets"
                  className="text-gray-600 hover:text-blue-600
transition-colors"
                >
                  Согласования
                </Link>
              </>
            )}

            {/* Ссылки для руководителя отдела */}
            {hasRole('department_head') && (
              <Link
                to="/tickets"
                className="text-gray-600 hover:text-blue-600 transition-
colors"
              >
                Согласования
              </Link>
            )}

            {/* Ссылка для администратора */}
            {hasRole('admin') && (
              <Link
                to="/admin"
```

```
                className="text-gray-600 hover:text-blue-600 transition-
colors"
              >
                Администрирование
              </Link>
            )}
          </nav>

          {/* Профиль пользователя */}
          <div className="flex items-center space-x-4">
            <Link to="/profile" className="text-right hover:opacity-80">
              <div className="text-sm font-medium text-gray-800">
                {user?.full_name}
              </div>
              <div className="text-xs text-gray-500">
                {roleNames[user?.role]}
              </div>
            </Link>
            <button
              onClick={handleLogout}
              className="btn btn-secondary text-sm"
            >
              Выйти
            </button>
          </div>
        </div>
      </div>
    </header>
  );
}
```

client/src/components/Layout.jsx

```
import { Link, useNavigate, useLocation } from 'react-router-dom';
import { useAuth } from '../contexts/AuthContext';
import { useSettings } from '../contexts/SettingsContext';
import { getApiUrl } from '../services/api';

export default function Layout({ children }) {
  const { user, logout } = useAuth();
  const { settings } = useSettings();
  const navigate = useNavigate();
  const location = useLocation();

  const handleLogout = () => {
    logout();
    navigate('/login');
  };

  const roleNames = {
    admin: 'Администратор',
    librarian: 'Библиотекарь',
    department_head: 'Руководитель отдела',
    technical_specialist: 'Технический специалист',
```

```
};

const getInitials = (name) => {
  if (!name) return 'U';
  const parts = name.split(' ');
  if (parts.length >= 2) {
    return (parts[0][0] + parts[1][0]).toUpperCase();
  }
  return name.substring(0, 2).toUpperCase();
};

const getAvatarColor = (name) => {
  if (!name) return 'bg-gray-400';
  const colors = [
    'bg-blue-500',
    'bg-green-500',
    'bg-yellow-500',
    'bg-red-500',
    'bg-purple-500',
    'bg-pink-500',
    'bg-indigo-500',
    'bg-teal-500',
  ];
  const index = name.charCodeAt(0) % colors.length;
  return colors[index];
};

const getAvatarUrl = (avatarPath) => {
  if (!avatarPath) return null;
  const apiUrl = getApiUrl();
  const baseUrl = apiUrl.replace('/api', '');
  return `${baseUrl}${avatarPath}`;
};

// Меню в зависимости от роли
const getMenuItems = () => {
  const items = [];

  if (user?.role !== 'admin') {
    items.push({ path: '/', label: 'Главная' });
    items.push({ path: '/search', label: 'Поиск' });
    items.push({ path: '/favorites', label: 'Избранное' });
  }

  if (user?.role === 'librarian') {
    items.push({ path: '/upload', label: 'Загрузить' });
    items.push({ path: '/tickets', label: 'Согласование' });
    items.push({ path: '/categories', label: 'Категории' });
  }

  if (user?.role === 'department_head') {
    items.push({ path: '/tickets', label: 'Согласование' });
  }
```

```
    if (user?.role === 'admin') {
      // Для админа показываем название текущей вкладки или
"Администрирование"
      const adminTabNames = {
        '/admin': 'Администрирование',
        '/admin/users': 'Пользователи',
        '/admin/settings': 'Настройки',
        '/admin/backups': 'Резервные копии',
        '/admin/analytics': 'Аналитика',
        '/admin/audit': 'Журнал действий',
      };

      // Если на странице профиля - показываем ссылку на админку
      if (location.pathname === '/profile') {
        items.push({ path: '/admin', label: 'Администрирование' });
      } else {
        items.push({ path: '/admin', label: 'Администрирование' });
      }
    }

    return items;
  };

  const menuItems = getMenuItems();

  return (
    <div className="min-h-screen bg-gray-100">
      {/* Header */}
      <header className="bg-white shadow">
        <div className="max-w-7xl mx-auto px-4 py-4">
          <div className="flex items-center justify-between">
            {/* Логотип и название */}
            <div className="flex items-center space-x-4">
              <Link to={user?.role === 'admin' ? '/admin' : '/'}
className="text-xl font-bold text-blue-600">
                {settings.app_name || 'Электронная библиотека НТД'}
              </Link>
            </div>

            {/* Навигация */}
            <nav className="hidden md:flex items-center space-x-6">
              {menuItems.map((item) => (
                <Link
                  key={item.path}
                  to={item.path}
                  className={`text-sm font-medium ${
                    location.pathname === item.path ||
                    (item.path === '/admin' &&
location.pathname.startsWith('/admin'))
                      ? 'text-blue-600'
                      : 'text-gray-600 hover:text-gray-900'
                  }`}
                >
                  {item.label}
```

```
                </Link>
              ))}
            </nav>

            {/* Пользователь с аватаром */}
            <div className="flex items-center space-x-4">
              <Link to="/profile" className="flex items-center space-x-3
hover:opacity-80">
                {user?.avatar_url ? (
                  <img
                    src={getAvatarUrl(user.avatar_url)}
                    alt="Аватар"
                    className="w-9 h-9 rounded-full object-cover"
                  />
                ) : (
                  <div className={`w-9 h-9 rounded-full flex items-center
justify-center text-white text-sm font-bold
${getAvatarColor(user?.full_name)}`}>
                    {getInitials(user?.full_name)}
                  </div>
                )}
                <div className="text-right hidden sm:block">
                  <div className="text-sm font-medium text-gray-
900">{user?.full_name}</div>
                  <div className="text-xs text-gray-
500">{roleNames[user?.role]}</div>
                </div>
              </Link>

              <button
                onClick={handleLogout}
                className="px-3 py-1.5 text-sm text-red-600 hover:text-
red-800 hover:bg-red-50 rounded"
              >
                Выйти
              </button>
            </div>
          </div>

          {/* Мобильное меню */}
          <nav className="md:hidden mt-4 flex flex-wrap gap-2">
            {menuItems.map((item) => (
              <Link
                key={item.path}
                to={item.path}
                className={`px-3 py-1.5 rounded text-sm ${
                  location.pathname === item.path ||
                  (item.path === '/admin' &&
location.pathname.startsWith('/admin'))
                    ? 'bg-blue-600 text-white'
                    : 'bg-gray-200 text-gray-700'
                }`}
              >
                {item.label}
```

```
              </Link>
            ))}
          </nav>
        </div>
      </header>

      {/* Main content */}
      <main className="max-w-7xl mx-auto px-4 py-6">{children}</main>
    </div>
  );
}
```

client/src/components/ProtectedRoute.jsx

```
import { Navigate } from 'react-router-dom';
import { useAuth } from '../contexts/AuthContext';

export default function ProtectedRoute({ children, roles }) {
  const { isAuthenticated, loading, hasRole } = useAuth();

  // Пока загружается - показываем индикатор
  if (loading) {
    return (
      <div className="min-h-screen flex items-center justify-center">
        <div className="text-xl text-gray-600">Загрузка...</div>
      </div>
    );
  }

  // Если не авторизован - перенаправляем на логин
  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  // Если указаны роли и у пользователя нет нужной роли
  if (roles && !hasRole(roles)) {
    return (
      <div className="min-h-screen flex items-center justify-center">
        <div className="text-center">
          <h1 className="text-2xl font-bold text-red-600 mb-2">
            Доступ запрещён
          </h1>
          <p className="text-gray-600">
            У вас недостаточно прав для просмотра этой страницы
          </p>
        </div>
      </div>
    );
  }

  return children;
}
```

client/src/components/SearchForm.jsx

```
import { useState } from 'react';

export default function SearchForm({ onSearch, categories }) {
  const [query, setQuery] = useState('');
  const [category, setCategory] = useState('');
  const [type, setType] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    onSearch({ query, category, type });
  };

  const handleReset = () => {
    setQuery('');
    setCategory('');
    setType('');
    onSearch({});
  };

  // Типы документов
  const documentTypes = [
    { value: 'drawing', label: 'Чертёж' },
    { value: 'standard', label: 'Стандарт' },
    { value: 'specification', label: 'Спецификация' },
    { value: 'instruction', label: 'Инструкция' },
    { value: 'manual', label: 'Руководство' },
    { value: 'other', label: 'Другое' },
  ];

  return (
    <form onSubmit={handleSubmit} className="card mb-6">
      <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
        {/* Поисковый запрос */}
        <div className="md:col-span-2">
          <label className="block text-sm font-medium text-gray-700 mb-
1">
            Поиск по названию или коду
          </label>
          <input
            type="text"
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            placeholder="Введите запрос..."
            className="input"
          />
        </div>

        {/* Категория */}
        <div>
          <label className="block text-sm font-medium text-gray-700 mb-
1">
            Категория
          </label>
```

```
          <select
            value={category}
            onChange={(e) => setCategory(e.target.value)}
            className="input"
          >
            <option value="">Все категории</option>
            {categories?.map((cat) => (
              <option key={cat.id} value={cat.id}>
                {cat.name}
              </option>
            ))}
          </select>
        </div>

        {/* Тип документа */}
        <div>
          <label className="block text-sm font-medium text-gray-700 mb-
1">
            Тип документа
          </label>
          <select
            value={type}
            onChange={(e) => setType(e.target.value)}
            className="input"
          >
            <option value="">Все типы</option>
            {documentTypes.map((t) => (
              <option key={t.value} value={t.value}>
                {t.label}
              </option>
            ))}
          </select>
        </div>
      </div>

      {/* Кнопки */}
      <div className="mt-4 flex space-x-2">
        <button type="submit" className="btn btn-primary">
          Найти
        </button>
        <button type="button" onClick={handleReset} className="btn btn-
secondary">
          Сбросить
        </button>
      </div>
    </form>
  );
}
```

client/src/contexts/AuthContext.jsx

```
import { createContext, useContext, useState, useEffect } from 'react';
import { authApi } from '../services/api';
```

```
const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const token = localStorage.getItem('token');
    const savedUser = localStorage.getItem('user');

    if (token && savedUser) {
      setUser(JSON.parse(savedUser));
    }
    setLoading(false);
  }, []);

  const login = async (email, password) => {
    console.log('[v0] AuthContext login called with:', { email });

    const response = await authApi.login({ email, password });

    console.log('[v0] Login response:', response.data);

    const { token, user } = response.data;
    localStorage.setItem('token', token);
    localStorage.setItem('user', JSON.stringify(user));
    setUser(user);
  };

  const register = async (data) => {
    const response = await authApi.register(data);
    const { token, user } = response.data;
    localStorage.setItem('token', token);
    localStorage.setItem('user', JSON.stringify(user));
    setUser(user);
  };

  const logout = () => {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, loading, login, register, logout
}}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
```

```
    throw new Error('useAuth must be used within AuthProvider');
  }
  return context;
}
```

client/src/contexts/SettingsContent.jsx

```
import { createContext, useContext, useState, useEffect } from 'react';
import { settingsApi } from '../services/api';
import { useAuth } from './AuthContext';

const SettingsContext = createContext();

export function SettingsProvider({ children }) {
  const { user } = useAuth();
  const [settings, setSettings] = useState({
    app_name: 'Электронная библиотека НТД',
  });
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    if (user) {
      loadSettings();
    }
  }, [user]);

  const loadSettings = async () => {
    try {
      const response = await settingsApi.getPublic();
      setSettings(response.data);
    } catch (error) {
      console.error('Ошибка загрузки настроек:', error);
    }
    setLoading(false);
  };

  const refreshSettings = () => {
    loadSettings();
  };

  return (
    <SettingsContext.Provider value={{ settings, loading, refreshSettings
}}>
      {children}
    </SettingsContext.Provider>
  );
}

export function useSettings() {
  const context = useContext(SettingsContext);
  if (!context) {
    throw new Error('useSettings must be used within SettingsProvider');
  }
  return context;
```

```
}
```

client/src/pages/AdminPage.jsx

```
import { useState, useEffect } from 'react';
import { usersApi, settingsApi, backupApi } from '../services/api';
import { useSettings } from '../contexts/SettingsContext';
import Layout from '../components/Layout';

export default function AdminPage() {
  const { refreshSettings } = useSettings();
  const [activeTab, setActiveTab] = useState('users');

  // Users
  const [users, setUsers] = useState([]);
  const [userSearch, setUserSearch] = useState('');
  const [showUserModal, setShowUserModal] = useState(false);
  const [editingUser, setEditingUser] = useState(null);
  const [userForm, setUserForm] = useState({
    email: '',
    password: '',
    full_name: '',
    position: '',
    role: 'technical_specialist',
  });

  // Settings
  const [settings, setSettings] = useState([]);
  const [newSetting, setNewSetting] = useState({ key: '', value: '',
description: '' });

  // Backups
  const [backups, setBackups] = useState([]);

  // Analytics
  const [analytics, setAnalytics] = useState(null);

  // Audit
  const [auditLogs, setAuditLogs] = useState([]);

  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState('');

  const typeNames = {
    drawing: 'Чертеж',
    standard: 'Стандарт',
    specification: 'Спецификация',
    instruction: 'Инструкция',
    manual: 'Руководство',
    other: 'Другое',
  };

  const tabNames = {
```

```
    users: 'Пользователи',
    settings: 'Настройки',
    backups: 'Резервные копии',
    analytics: 'Аналитика',
    audit: 'Журнал действий',
  };

  useEffect(() => {
    loadData();
  }, [activeTab]);

  const loadData = async () => {
    setLoading(true);
    setError('');

    try {
      if (activeTab === 'users') {
        const response = await usersApi.getAll();
        setUsers(response.data);
      } else if (activeTab === 'settings') {
        const response = await settingsApi.getAll();
        setSettings(response.data);
      } else if (activeTab === 'backups') {
        const response = await backupApi.getAll();
        setBackups(response.data);
      } else if (activeTab === 'analytics') {
        const response = await usersApi.getAnalytics();
        setAnalytics(response.data);
      } else if (activeTab === 'audit') {
        const response = await usersApi.getAuditLogs();
        setAuditLogs(response.data);
      }
    } catch (err) {
      console.error('Ошибка загрузки:', err);
      setError('Ошибка загрузки данных: ' + (err.response?.data?.error ||
err.message));
    }

    setLoading(false);
  };

  // Фильтрация пользователей
  const filteredUsers = users.filter(user => {
    if (!userSearch.trim()) return true;
    const search = userSearch.toLowerCase();
    return (
      user.full_name?.toLowerCase().includes(search) ||
      user.email?.toLowerCase().includes(search) ||
      user.position?.toLowerCase().includes(search)
    );
  });

  // User handlers
  const handleUserSubmit = async (e) => {
```

```
  e.preventDefault();
  setError('');

  try {
    if (editingUser) {
      await usersApi.update(editingUser.id, userForm);
      setSuccess('Пользователь обновлен');
    } else {
      await usersApi.create(userForm);
      setSuccess('Пользователь создан');
    }
    setShowUserModal(false);
    resetUserForm();
    loadData();
  } catch (err) {
    setError(err.response?.data?.error || 'Ошибка сохранения');
  }
};

const handleEditUser = (user) => {
  setEditingUser(user);
  setUserForm({
    email: user.email,
    password: '',
    full_name: user.full_name,
    position: user.position || '',
    role: user.role,
  });
  setShowUserModal(true);
};

const handleDeleteUser = async (userId) => {
  if (!confirm('Удалить этого пользователя?')) return;

  try {
    await usersApi.delete(userId);
    setSuccess('Пользователь удален');
    loadData();
  } catch (err) {
    setError(err.response?.data?.error || 'Ошибка удаления');
  }
};

const resetUserForm = () => {
  setEditingUser(null);
  setUserForm({
    email: '',
    password: '',
    full_name: '',
    position: '',
    role: 'technical_specialist',
  });
};
```

```
  // Settings handlers
  const handleSettingChange = async (key, value) => {
    try {
      await settingsApi.update(key, value);
      setSuccess('Настройка сохранена');
      refreshSettings();
      loadData();
    } catch (err) {
      setError('Ошибка сохранения настройки');
    }
  };

  const handleAddSetting = async (e) => {
    e.preventDefault();
    if (!newSetting.key || !newSetting.value) {
      setError('Заполните ключ и значение');
      return;
    }
    try {
      await settingsApi.update(newSetting.key, newSetting.value,
newSetting.description);
      setSuccess('Настройка добавлена');
      setNewSetting({ key: '', value: '', description: '' });
      refreshSettings();
      loadData();
    } catch (err) {
      setError('Ошибка добавления настройки');
    }
  };

  // Backup handlers
  const handleCreateBackup = async () => {
    setLoading(true);
    try {
      await backupApi.create();
      setSuccess('Резервная копия создана');
      loadData();
    } catch (err) {
      setError('Ошибка создания резервной копии');
    }
    setLoading(false);
  };

  const handleRestoreBackup = async (filename) => {
    if (!confirm('Восстановить данные из этой резервной копии? Текущие
данные будут перезаписаны!')) return;

    setLoading(true);
    try {
      await backupApi.restore(filename);
      setSuccess('Данные восстановлены из резервной копии');
      loadData();
    } catch (err) {
      setError(err.response?.data?.error || 'Ошибка восстановления');
```

```
    }
    setLoading(false);
  };

  const handleDownloadBackup = async (filename) => {
    try {
      const response = await backupApi.download(filename);
      const url = window.URL.createObjectURL(new Blob([response.data]));
      const link = document.createElement('a');
      link.href = url;
      link.download = filename;
      link.click();
      window.URL.revokeObjectURL(url);
    } catch (err) {
      setError('Ошибка скачивания');
    }
  };

  const handleDeleteBackup = async (id) => {
    if (!confirm('Удалить эту резервную копию?')) return;

    try {
      await backupApi.delete(id);
      setSuccess('Резервная копия удалена');
      loadData();
    } catch (err) {
      setError('Ошибка удаления');
    }
  };

  const formatDate = (dateString) => {
    return new Date(dateString).toLocaleString('ru-RU');
  };

  const formatFileSize = (bytes) => {
    if (!bytes) return '0 Б';
    const k = 1024;
    const sizes = ['Б', 'КБ', 'МБ', 'ГБ'];
    const i = Math.floor(Math.log(bytes) / Math.log(k));
    return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + ' ' +
sizes[i];
  };

  const roleNames = {
    admin: 'Администратор',
    librarian: 'Библиотекарь',
    department_head: 'Руководитель отдела',
    technical_specialist: 'Технический специалист',
  };

  const actionNames = {
    user_login: 'Вход в систему',
    user_register: 'Регистрация',
    document_create: 'Создание документа',
```

```
    document_update: 'Обновление документа',
    document_delete: 'Удаление документа',
    document_view: 'Просмотр документа',
    document_download: 'Скачивание документа',
  };

  // Группируем настройки по категориям
  const settingsGroups = {
    general: {
      title: 'Основные настройки',
      keys: ['app_name', 'company_name', 'support_email'],
    },
    files: {
      title: 'Настройки файлов',
      keys: ['max_file_size', 'allowed_extensions', 'preview_enabled'],
    },
    security: {
      title: 'Безопасность',
      keys: ['session_timeout', 'max_login_attempts',
'password_min_length'],
    },
    notifications: {
      title: 'Уведомления',
      keys: ['email_notifications', 'notify_on_upload',
'notify_on_approval'],
    },
  };

  const getSettingInput = (setting) => {
    const key = setting.key;

    // Числовые настройки
    if (['max_file_size'].includes(key)) {
      return (
        <div className="flex items-center space-x-2">
          <input
            type="number"
            defaultValue={Math.round(parseInt(setting.value) / 1024 /
1024)}
            className="input w-32"
            onBlur={(e) => {
              const bytes = parseInt(e.target.value) * 1024 * 1024;
              if (bytes !== parseInt(setting.value)) {
                handleSettingChange(key, bytes.toString());
              }
            }}
          />
          <span className="text-gray-500">МБ</span>
        </div>
      );
    }

    if (['session_timeout'].includes(key)) {
      return (
```

```
        <div className="flex items-center space-x-2">
          <input
            type="number"
            defaultValue={parseInt(setting.value)}
            className="input w-32"
            onBlur={(e) => {
              if (e.target.value !== setting.value) {
                handleSettingChange(key, e.target.value);
              }
            }}
          />
          <span className="text-gray-500">минут</span>
        </div>
      );
    }

    if (['max_login_attempts', 'password_min_length'].includes(key)) {
      return (
        <input
          type="number"
          defaultValue={parseInt(setting.value)}
          className="input w-32"
          onBlur={(e) => {
            if (e.target.value !== setting.value) {
              handleSettingChange(key, e.target.value);
            }
          }}
        />
      );
    }

    // Булевые настройки
    if (['preview_enabled', 'email_notifications', 'notify_on_upload',
'notify_on_approval'].includes(key)) {
      return (
        <label className="flex items-center space-x-2 cursor-pointer">
          <input
            type="checkbox"
            checked={setting.value === 'true'}
            onChange={(e) => handleSettingChange(key,
e.target.checked.toString())}
            className="w-5 h-5 text-blue-600 rounded"
          />
          <span className="text-gray-600">{setting.value === 'true' ?
'Включено' : 'Выключено'}</span>
        </label>
      );
    }

    // Текстовые настройки
    return (
      <input
        type="text"
        defaultValue={setting.value}
```

```
        className="input"
        onBlur={(e) => {
          if (e.target.value !== setting.value) {
            handleSettingChange(key, e.target.value);
          }
        }}
      />
    );
  };

  const settingDescriptions = {
    app_name: 'Название приложения',
    company_name: 'Название организации',
    support_email: 'Email поддержки',
    max_file_size: 'Максимальный размер файла',
    allowed_extensions: 'Разрешенные расширения файлов',
    preview_enabled: 'Включить предпросмотр документов',
    session_timeout: 'Время сессии',
    max_login_attempts: 'Максимум попыток входа',
    password_min_length: 'Минимальная длина пароля',
    email_notifications: 'Email уведомления',
    notify_on_upload: 'Уведомлять при загрузке документа',
    notify_on_approval: 'Уведомлять при согласовании',
  };

  return (
    <Layout>
      {/* Уведомления */}
      {error && (
        <div className="mb-4 p-4 bg-red-100 border border-red-400 text-
red-700 rounded">
          {error}
          <button onClick={() => setError('')} className="float-
right">&times;</button>
        </div>
      )}
      {success && (
        <div className="mb-4 p-4 bg-green-100 border border-green-400
text-green-700 rounded">
          {success}
          <button onClick={() => setSuccess('')} className="float-
right">&times;</button>
        </div>
      )}

      {/* Табы с названиями вкладок */}
      <div className="flex flex-wrap gap-2 mb-6">
        {[
          { id: 'users', label: 'Пользователи' },
          { id: 'settings', label: 'Настройки' },
          { id: 'backups', label: 'Резервные копии' },
          { id: 'analytics', label: 'Аналитика' },
          { id: 'audit', label: 'Журнал действий' },
        ].map((tab) => (
```

```
          <button
            key={tab.id}
            onClick={() => setActiveTab(tab.id)}
            className={`px-4 py-2 rounded-lg font-medium ${
              activeTab === tab.id
                ? 'bg-blue-600 text-white'
                : 'bg-gray-200 text-gray-700 hover:bg-gray-300'
            }`}
          >
            {tab.label}
          </button>
        ))}
      </div>

      {loading && <div className="text-center py-8">Загрузка...</div>}

      {/* Пользователи */}
      {activeTab === 'users' && !loading && (
        <div className="card">
          <div className="flex flex-col sm:flex-row justify-between
items-start sm:items-center gap-4 mb-4">
            <div className="relative flex-1 max-w-md">
              <input
                type="text"
                placeholder="Поиск по имени, email или должности..."
                value={userSearch}
                onChange={(e) => setUserSearch(e.target.value)}
                className="input pl-10"
              />
              <svg
                className="absolute left-3 top-1/2 -translate-y-1/2 w-4
h-4 text-gray-400"
                fill="none"
                stroke="currentColor"
                viewBox="0 0 24 24"
              >
                <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z" />
              </svg>
            </div>

            <button
              onClick={() => {
                resetUserForm();
                setShowUserModal(true);
              }}
              className="btn btn-primary whitespace-nowrap"
            >
              Добавить пользователя
            </button>
          </div>

          <div className="overflow-x-auto">
            <table className="w-full">
```

```
              <thead className="bg-gray-50">
                <tr>
                  <th className="px-4 py-2 text-left text-sm font-medium
text-gray-600">ФИО</th>
                  <th className="px-4 py-2 text-left text-sm font-medium
text-gray-600">Email</th>
                  <th className="px-4 py-2 text-left text-sm font-medium
text-gray-600">Роль</th>
                  <th className="px-4 py-2 text-left text-sm font-medium
text-gray-600">Должность</th>
                  <th className="px-4 py-2 text-right text-sm font-medium
text-gray-600">Действия</th>
                </tr>
              </thead>
              <tbody>
                {filteredUsers.length === 0 ? (
                  <tr>
                    <td colSpan="5" className="px-4 py-8 text-center
text-gray-500">
                      {userSearch ? 'Пользователи не найдены' : 'Нет
пользователей'}
                    </td>
                  </tr>
                ) : (
                  filteredUsers.map((user) => (
                    <tr key={user.id} className="border-t">
                      <td className="px-4 py-3">{user.full_name}</td>
                      <td className="px-4 py-3">{user.email}</td>
                      <td className="px-4 py-
3">{roleNames[user.role]}</td>
                      <td className="px-4 py-3">{user.position || '-
'}</td>
                      <td className="px-4 py-3 text-right">
                        <button
                          onClick={() => handleEditUser(user)}
                          className="text-blue-600 hover:underline mr-3"
                        >
                          Изменить
                        </button>
                        <button
                          onClick={() => handleDeleteUser(user.id)}
                          className="text-red-600 hover:underline"
                        >
                          Удалить
                        </button>
                      </td>
                    </tr>
                  ))
                )}
              </tbody>
            </table>
          </div>

          {userSearch && (
```

```
            <div className="mt-4 text-sm text-gray-500">
              Найдено: {filteredUsers.length} из {users.length}
            </div>
          )}
        </div>
      )}

      {/* Настройки */}
      {activeTab === 'settings' && !loading && (
        <div className="space-y-6">
          {Object.entries(settingsGroups).map(([groupKey, group]) => {
            const groupSettings = settings.filter(s =>
group.keys.includes(s.key));
            if (groupSettings.length === 0 && groupKey !== 'general')
return null;

            return (
              <div key={groupKey} className="card">
                <h3 className="text-lg font-semibold text-gray-800 mb-
4">{group.title}</h3>
                <div className="space-y-4">
                  {group.keys.map(key => {
                    const setting = settings.find(s => s.key === key);
                    if (!setting) {
                      // Показываем поле для создания настройки
                      return (
                        <div key={key} className="border-b pb-4 opacity-
50">
                          <label className="block text-sm font-medium
text-gray-700 mb-1">
                            {settingDescriptions[key] || key}
                          </label>
                          <p className="text-sm text-gray-400">Настройка
не задана</p>
                        </div>
                      );
                    }

                    return (
                      <div key={key} className="border-b pb-4">
                        <label className="block text-sm font-medium text-
gray-700 mb-1">
                          {settingDescriptions[key] ||
setting.description || key}
                        </label>
                        {getSettingInput(setting)}
                      </div>
                    );
                  })}
                </div>
              </div>
            );
          })}
```

```
          {/* Добавление новой настройки */}
          <div className="card">
            <h3 className="text-lg font-semibold text-gray-800 mb-
4">Добавить настройку</h3>
            <form onSubmit={handleAddSetting} className="space-y-4">
              <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
                <div>
                  <label className="block text-sm font-medium text-gray-
700 mb-1">Ключ</label>
                  <input
                    type="text"
                    value={newSetting.key}
                    onChange={(e) => setNewSetting({ ...newSetting, key:
e.target.value })}
                    className="input"
                    placeholder="например: custom_setting"
                  />
                </div>
                <div>
                  <label className="block text-sm font-medium text-gray-
700 mb-1">Значение</label>
                  <input
                    type="text"
                    value={newSetting.value}
                    onChange={(e) => setNewSetting({ ...newSetting,
value: e.target.value })}
                    className="input"
                  />
                </div>
                <div>
                  <label className="block text-sm font-medium text-gray-
700 mb-1">Описание</label>
                  <input
                    type="text"
                    value={newSetting.description}
                    onChange={(e) => setNewSetting({ ...newSetting,
description: e.target.value })}
                    className="input"
                  />
                </div>
              </div>
              <button type="submit" className="btn btn-secondary">
                Добавить настройку
              </button>
            </form>
          </div>

          {/* Все остальные настройки */}
          {settings.filter(s => !Object.values(settingsGroups).flatMap(g
=> g.keys).includes(s.key) && s.key !== 'approval_stages').length > 0 &&
(
            <div className="card">
              <h3 className="text-lg font-semibold text-gray-800 mb-
4">Другие настройки</h3>
```

```
              <div className="space-y-4">
                {settings
                  .filter(s => !Object.values(settingsGroups).flatMap(g
=> g.keys).includes(s.key) && s.key !== 'approval_stages')
                  .map(setting => (
                    <div key={setting.key} className="border-b pb-4">
                      <label className="block text-sm font-medium text-
gray-700 mb-1">
                        {setting.description || setting.key}
                      </label>
                      {getSettingInput(setting)}
                    </div>
                  ))
                }
              </div>
            </div>
          )}
        </div>
      )}

      {/* Резервные копии */}
      {activeTab === 'backups' && !loading && (
        <div className="card">
          <div className="flex justify-between items-center mb-4">
            <span className="text-gray-600">Управление резервными
копиями</span>
            <button onClick={handleCreateBackup} className="btn btn-
primary">
              Создать резервную копию
            </button>
          </div>

          {backups.length === 0 ? (
            <div className="text-center py-8 text-gray-500">
              Нет резервных копий
            </div>
          ) : (
            <div className="space-y-3">
              {backups.map((backup) => (
                <div
                  key={backup.id}
                  className="flex items-center justify-between p-4 bg-
gray-50 rounded-lg"
                >
                  <div>
                    <div className="font-medium">{backup.filename}</div>
                    <div className="text-sm text-gray-500">
                      {formatDate(backup.created_at)} |
{formatFileSize(backup.size_bytes)}
                      {backup.created_by_name && ` |
${backup.created_by_name}`}
                    </div>
                  </div>
                  <div className="flex space-x-2">
```

```
                    <button
                      onClick={() =>
handleRestoreBackup(backup.filename)}
                      className="btn bg-green-600 text-white hover:bg-
green-700 text-sm"
                    >
                      Восстановить
                    </button>
                    <button
                      onClick={() =>
handleDownloadBackup(backup.filename)}
                      className="btn btn-secondary text-sm"
                    >
                      Скачать
                    </button>
                    <button
                      onClick={() => handleDeleteBackup(backup.id)}
                      className="btn bg-red-600 text-white hover:bg-red-
700 text-sm"
                    >
                      Удалить
                    </button>
                  </div>
                </div>
              ))}
            </div>
          )}
        </div>
      )}

      {/* Аналитика */}
      {activeTab === 'analytics' && !loading && analytics && (
        <div className="space-y-6">
          <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
            <div className="card text-center">
              <div className="text-3xl font-bold text-blue-
600">{analytics.total_documents || 0}</div>
              <div className="text-gray-600 text-sm">Всего
документов</div>
            </div>
            <div className="card text-center">
              <div className="text-3xl font-bold text-green-
600">{analytics.documents_in_library || 0}</div>
              <div className="text-gray-600 text-sm">В библиотеке</div>
            </div>
            <div className="card text-center">
              <div className="text-3xl font-bold text-yellow-
600">{analytics.documents_pending || 0}</div>
              <div className="text-gray-600 text-sm">На
согласовании</div>
            </div>
            <div className="card text-center">
              <div className="text-3xl font-bold text-gray-
600">{analytics.documents_archived || 0}</div>
```

```
              <div className="text-gray-600 text-sm">В архиве</div>
            </div>
          </div>

          <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
            <div className="card">
              <h3 className="font-semibold text-gray-800 mb-
4">Пользователи ({analytics.total_users || 0})</h3>
              <div className="space-y-2">
                <div className="flex justify-between">
                  <span className="text-gray-600">Администраторы</span>
                  <span className="font-medium">{analytics.admins ||
0}</span>
                </div>
                <div className="flex justify-between">
                  <span className="text-gray-600">Библиотекари</span>
                  <span className="font-medium">{analytics.librarians ||
0}</span>
                </div>
                <div className="flex justify-between">
                  <span className="text-gray-600">Руководители
отделов</span>
                  <span className="font-
medium">{analytics.department_heads || 0}</span>
                </div>
                <div className="flex justify-between">
                  <span className="text-gray-600">Технические
специалисты</span>
                  <span className="font-
medium">{analytics.technical_specialists || 0}</span>
                </div>
              </div>
            </div>

            <div className="card">
              <h3 className="font-semibold text-gray-800 mb-4">Тикеты
согласования ({analytics.total_tickets || 0})</h3>
              <div className="space-y-2">
                <div className="flex justify-between">
                  <span className="text-gray-600">Открытые</span>
                  <span className="font-medium text-blue-
600">{analytics.tickets_open || 0}</span>
                </div>
                <div className="flex justify-between">
                  <span className="text-gray-600">В работе</span>
                  <span className="font-medium text-yellow-
600">{analytics.tickets_in_progress || 0}</span>
                </div>
                <div className="flex justify-between">
                  <span className="text-gray-600">Согласованы</span>
                  <span className="font-medium text-green-
600">{analytics.tickets_resolved || 0}</span>
                </div>
                <div className="flex justify-between">
```

```
                  <span className="text-gray-600">Закрыты</span>
                  <span className="font-medium text-gray-
600">{analytics.tickets_closed || 0}</span>
                </div>
              </div>
            </div>
          </div>

          {analytics.docs_by_type && analytics.docs_by_type.length > 0 &&
(
            <div className="card">
              <h3 className="font-semibold text-gray-800 mb-4">Документы
по типам</h3>
              <div className="space-y-2">
                {analytics.docs_by_type.map((item) => (
                  <div key={item.type} className="flex items-center">
                    <div className="w-32 text-gray-
600">{typeNames[item.type] || item.type}</div>
                    <div className="flex-1 bg-gray-200 rounded-full h-4
mr-4">
                      <div
                        className="bg-blue-500 h-4 rounded-full"
                        style={{ width: `${Math.max((parseInt(item.count)
/ (analytics.total_documents || 1)) * 100, 5)}%` }}
                      />
                    </div>
                    <div className="w-12 text-right font-
medium">{item.count}</div>
                  </div>
                ))}
              </div>
            </div>
          )}

          {analytics.docs_by_category &&
analytics.docs_by_category.length > 0 && (
            <div className="card">
              <h3 className="font-semibold text-gray-800 mb-4">Документы
по категориям</h3>
              <div className="space-y-2">
                {analytics.docs_by_category.map((item, index) => (
                  <div key={index} className="flex items-center">
                    <div className="w-40 text-gray-600
truncate">{item.category}</div>
                    <div className="flex-1 bg-gray-200 rounded-full h-4
mr-4">
                      <div
                        className="bg-green-500 h-4 rounded-full"
                        style={{ width: `${Math.max((parseInt(item.count)
/ (analytics.total_documents || 1)) * 100, 5)}%` }}
                      />
                    </div>
                    <div className="w-12 text-right font-
medium">{item.count}</div>
```

```
                  </div>
                ))}
              </div>
            </div>
          )}

          {analytics.top_users && analytics.top_users.length > 0 && (
            <div class="card">
              <h3 class="font-semibold text-gray-800 mb-4">Топ
активных пользователей (за 30 дней)</h3>
              <div class="space-y-2">
                {analytics.top_users.map((user, index) => (
                  <div key={index} class="flex items-center justify-
between p-2 bg-gray-50 rounded">
                    <div class="flex items-center space-x-3">
                      <span class="w-6 h-6 bg-blue-100 text-blue-600
rounded-full flex items-center justify-center text-sm font-medium">
                        {index + 1}
                      </span>
                      <span>{user.full_name}</span>
                    </div>
                    <span class="font-medium">{user.actions}
действий</span>
                  </div>
                ))}
              </div>
            </div>
          )}

          <div class="grid grid-cols-2 md:grid-cols-4 gap-4">
            <div class="card text-center">
              <div class="text-2xl font-bold text-purple-
600">{analytics.total_categories || 0}</div>
              <div class="text-gray-600 text-sm">Категорий</div>
            </div>
            <div class="card text-center">
              <div class="text-2xl font-bold text-purple-
600">{analytics.total_tags || 0}</div>
              <div class="text-gray-600 text-sm">Тегов</div>
            </div>
            <div class="card text-center">
              <div class="text-2xl font-bold text-purple-
600">{formatFileSize(analytics.total_storage_bytes || 0)}</div>
              <div class="text-gray-600 text-sm">Размер файлов</div>
            </div>
            <div class="card text-center">
              <div class="text-2xl font-bold text-purple-
600">{analytics.actions_today || 0}</div>
              <div class="text-gray-600 text-sm">Действий
сегодня</div>
            </div>
          </div>
        </div>
      )}
```

```
      {/* Журнал действий */}
      {activeTab === 'audit' && !loading && (
        <div className="card">
          {auditLogs.length === 0 ? (
            <div className="text-center py-8 text-gray-500">
              Журнал пуст
            </div>
          ) : (
            <div className="overflow-x-auto">
              <table className="w-full">
                <thead className="bg-gray-50">
                  <tr>
                    <th className="px-4 py-2 text-left text-sm font-
medium text-gray-600">Дата</th>
                    <th className="px-4 py-2 text-left text-sm font-
medium text-gray-600">Пользователь</th>
                    <th className="px-4 py-2 text-left text-sm font-
medium text-gray-600">Действие</th>
                    <th className="px-4 py-2 text-left text-sm font-
medium text-gray-600">Объект</th>
                  </tr>
                </thead>
                <tbody>
                  {auditLogs.slice(0, 100).map((log) => (
                    <tr key={log.id} className="border-t">
                      <td className="px-4 py-2 text-
sm">{formatDate(log.created_at)}</td>
                      <td className="px-4 py-2 text-sm">{log.user_name ||
'Система'}</td>
                      <td className="px-4 py-2 text-
sm">{actionNames[log.action] || log.action}</td>
                      <td className="px-4 py-2 text-sm">{log.entity_type}
#{log.entity_id}</td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          )}
        </div>
      )}

      {/* Модальное окно пользователя */}
      {showUserModal && (
        <div className="fixed inset-0 bg-black bg-opacity-50 flex items-
center justify-center z-50">
          <div className="bg-white rounded-lg p-6 max-w-md w-full mx-4">
            <h3 className="text-lg font-semibold mb-4">
              {editingUser ? 'Редактирование пользователя' : 'Новый
пользователь'}
            </h3>

            <form onSubmit={handleUserSubmit} className="space-y-4">
```

```
              <div>
                <label className="block text-sm font-medium text-gray-700
mb-1">ФИО *</label>
                <input
                  type="text"
                  value={userForm.full_name}
                  onChange={(e) => setUserForm({ ...userForm, full_name:
e.target.value })}
                  className="input"
                  required
                />
              </div>

              <div>
                <label className="block text-sm font-medium text-gray-700
mb-1">Email *</label>
                <input
                  type="email"
                  value={userForm.email}
                  onChange={(e) => setUserForm({ ...userForm, email:
e.target.value })}
                  className="input"
                  required
                />
              </div>

              <div>
                <label className="block text-sm font-medium text-gray-700
mb-1">
                  Пароль {editingUser ? '(оставьте пустым, чтобы не
менять)' : '*'}
                </label>
                <input
                  type="password"
                  value={userForm.password}
                  onChange={(e) => setUserForm({ ...userForm, password:
e.target.value })}
                  className="input"
                  required={!editingUser}
                />
              </div>

              <div>
                <label className="block text-sm font-medium text-gray-700
mb-1">Должность</label>
                <input
                  type="text"
                  value={userForm.position}
                  onChange={(e) => setUserForm({ ...userForm, position:
e.target.value })}
                  className="input"
                />
              </div>
```

```
              <div>
                <label className="block text-sm font-medium text-gray-700
mb-1">Роль *</label>
                <select
                  value={userForm.role}
                  onChange={(e) => setUserForm({ ...userForm, role:
e.target.value })}
                  className="input"
                >
                  <option value="technical_specialist">Технический
специалист</option>
                  <option value="librarian">Библиотекарь</option>
                  <option value="department_head">Руководитель
отдела</option>
                  <option value="admin">Администратор</option>
                </select>
              </div>

              <div className="flex space-x-2 pt-4">
                <button type="submit" className="btn btn-primary flex-1">
                  {editingUser ? 'Сохранить' : 'Создать'}
                </button>
                <button
                  type="button"
                  onClick={() => setShowUserModal(false)}
                  className="btn btn-secondary flex-1"
                >
                  Отмена
                </button>
              </div>
            </form>
          </div>
        </div>
      )}
    </Layout>
  );
}
```

client/src/pages/ApprovalsPage.jsx

```
import { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import { documentsApi } from '../services/api';
import Layout from '../components/Layout';

export default function ApprovalsPage() {
  const [activeTab, setActiveTab] = useState('approval');
  const [pendingApproval, setPendingApproval] = useState([]);
  const [pendingDeletion, setPendingDeletion] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    loadDocuments();
```

```
}, []);

const loadDocuments = async () => {
  setLoading(true);
  setError('');
  try {
    const [approvalRes, deletionRes] = await Promise.all([
      documentsApi.search({ status: 'pending_approval' }),
      documentsApi.search({ status: 'pending_deletion' }),
    ]);
    setPendingApproval(approvalRes.data.documents || []);
    setPendingDeletion(deletionRes.data.documents || []);
  } catch (err) {
    setError('Ошибка загрузки документов');
  }
  setLoading(false);
};

// Утверждение документа
const handleApprove = async (id) => {
  const comment = prompt('Комментарий (необязательно):');
  try {
    await documentsApi.approve(id, comment || '');
    loadDocuments();
  } catch (err) {
    alert('Ошибка при утверждении');
  }
};

// Отклонение документа
const handleReject = async (id) => {
  const comment = prompt('Причина отклонения:');
  if (!comment) {
    alert('Укажите причину отклонения');
    return;
  }
  try {
    await documentsApi.reject(id, comment);
    loadDocuments();
  } catch (err) {
    alert('Ошибка при отклонении');
  }
};

// Согласование удаления
const handleApproveDeletion = async (id) => {
  if (!confirm('Подтвердить удаление документа?')) return;
  try {
    await documentsApi.approveDeletion(id);
    loadDocuments();
  } catch (err) {
    alert('Ошибка при удалении');
  }
};
```

```
// Отклонение удаления
const handleRejectDeletion = async (id) => {
  try {
    await documentsApi.rejectDeletion(id);
    loadDocuments();
  } catch (err) {
    alert('Ошибка');
  }
};

const formatDate = (dateString) => {
  return new Date(dateString).toLocaleDateString('ru-RU', {
    day: '2-digit',
    month: '2-digit',
    year: 'numeric',
  });
};

const typeNames = {
  drawing: 'Чертеж',
  standard: 'Стандарт',
  specification: 'Спецификация',
  instruction: 'Инструкция',
  manual: 'Руководство',
  other: 'Другое',
};

return (
  <Layout>
    <h1 className="text-2xl font-bold text-gray-800 mb-6">
      Согласование документов
    </h1>

    {/* Табы */}
    <div className="flex space-x-4 mb-6">
      <button
        onClick={() => setActiveTab('approval')}
        className={`px-4 py-2 rounded-lg font-medium ${
          activeTab === 'approval'
            ? 'bg-blue-600 text-white'
            : 'bg-gray-200 text-gray-700 hover:bg-gray-300'
        }`}
      >
        На утверждение ({pendingApproval.length})
      </button>
      <button
        onClick={() => setActiveTab('deletion')}
        className={`px-4 py-2 rounded-lg font-medium ${
          activeTab === 'deletion'
            ? 'bg-blue-600 text-white'
            : 'bg-gray-200 text-gray-700 hover:bg-gray-300'
        }`}
      >
```

```
          На удаление ({pendingDeletion.length})
        </button>
      </div>

      {loading && <div className="text-center py-8">Загрузка...</div>}

      {error && (
        <div className="mb-4 p-4 bg-red-100 border border-red-400 text-
red-700 rounded">
          {error}
        </div>
      )}

      {/* Документы на утверждение */}
      {!loading && activeTab === 'approval' && (
        <div className="card">
          {pendingApproval.length === 0 ? (
            <div className="text-center py-8 text-gray-500">
              Нет документов на утверждение
            </div>
          ) : (
            <div className="overflow-x-auto">
              <table className="w-full">
                <thead>
                  <tr className="border-b">
                    <th className="text-left py-3 px-4 font-medium text-
gray-600">Код</th>
                    <th className="text-left py-3 px-4 font-medium text-
gray-600">Название</th>
                    <th className="text-left py-3 px-4 font-medium text-
gray-600">Тип</th>
                    <th className="text-left py-3 px-4 font-medium text-
gray-600">Автор</th>
                    <th className="text-left py-3 px-4 font-medium text-
gray-600">Дата</th>
                    <th className="text-left py-3 px-4 font-medium text-
gray-600">Действия</th>
                  </tr>
                </thead>
                <tbody>
                  {pendingApproval.map((doc) => (
                    <tr key={doc.id} className="border-b hover:bg-gray-
50">
                      <td className="py-3 px-4 font-mono text-
sm">{doc.code}</td>
                      <td className="py-3 px-4">
                        <Link to={`/documents/${doc.id}`}
className="text-blue-600 hover:underline">
                          {doc.title}
                        </Link>
                      </td>
                      <td className="py-3 px-
4">{typeNames[doc.type]}</td>
                      <td className="py-3 px-4">{doc.author_name}</td>
```

```
                      <td className="py-3 px-4 text-
sm">{formatDate(doc.updated_at)}</td>
                      <td className="py-3 px-4">
                        <div className="flex space-x-2">
                          <button
                            onClick={() => handleApprove(doc.id)}
                            className="px-3 py-1 bg-green-600 text-white
text-sm rounded hover:bg-green-700"
                          >
                            Утвердить
                          </button>
                          <button
                            onClick={() => handleReject(doc.id)}
                            className="px-3 py-1 bg-red-600 text-white
text-sm rounded hover:bg-red-700"
                          >
                            Отклонить
                          </button>
                        </div>
                      </td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          )}
        </div>
      )}

      {/* Документы на удаление */}
      {!loading && activeTab === 'deletion' && (
        <div className="card">
          {pendingDeletion.length === 0 ? (
            <div className="text-center py-8 text-gray-500">
              Нет документов на удаление
            </div>
          ) : (
            <div className="overflow-x-auto">
              <table className="w-full">
                <thead>
                  <tr className="border-b">
                    <th className="text-left py-3 px-4 font-medium text-
gray-600">Код</th>
                    <th className="text-left py-3 px-4 font-medium text-
gray-600">Название</th>
                    <th className="text-left py-3 px-4 font-medium text-
gray-600">Тип</th>
                    <th className="text-left py-3 px-4 font-medium text-
gray-600">Действия</th>
                  </tr>
                </thead>
                <tbody>
                  {pendingDeletion.map((doc) => (
```

```
                    <tr key={doc.id} className="border-b hover:bg-gray-
50">
                      <td className="py-3 px-4 font-mono text-
sm">{doc.code}</td>
                      <td className="py-3 px-4">
                        <Link to={`/documents/${doc.id}`}
className="text-blue-600 hover:underline">
                          {doc.title}
                        </Link>
                      </td>
                      <td className="py-3 px-
4">{typeNames[doc.type]}</td>
                      <td className="py-3 px-4">
                        <div className="flex space-x-2">
                          <button
                            onClick={() => handleApproveDeletion(doc.id)}
                            className="px-3 py-1 bg-red-600 text-white
text-sm rounded hover:bg-red-700"
                          >
                            Удалить
                          </button>
                          <button
                            onClick={() => handleRejectDeletion(doc.id)}
                            className="px-3 py-1 bg-gray-600 text-white
text-sm rounded hover:bg-gray-700"
                          >
                            Отменить
                          </button>
                        </div>
                      </td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          )}
        </div>
      )}
    </Layout>
  );
}
```

client/src/pages/CategoriesPage.jsx

```
import { useState, useEffect } from 'react';
import { categoriesApi } from '../services/api';
import Layout from '../components/Layout';

export default function CategoriesPage() {
  const [categories, setCategories] = useState([]);
  const [tags, setTags] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState('');
```

```
// Category form
const [showCategoryModal, setShowCategoryModal] = useState(false);
const [editingCategory, setEditingCategory] = useState(null);
const [categoryName, setCategoryName] = useState('');
const [categoryDescription, setCategoryDescription] = useState('');

// Tag form
const [showTagModal, setShowTagModal] = useState(false);
const [tagName, setTagName] = useState('');

useEffect(() => {
  loadData();
}, []);

const loadData = async () => {
  setLoading(true);
  try {
    const [categoriesRes, tagsRes] = await Promise.all([
      categoriesApi.getAll(),
      categoriesApi.getAllTags(),
    ]);
    setCategories(categoriesRes.data);
    setTags(tagsRes.data);
  } catch (err) {
    setError('Ошибка загрузки данных');
  }
  setLoading(false);
};

// Category handlers
const handleCategorySubmit = async (e) => {
  e.preventDefault();
  setError('');

  if (!categoryName.trim()) {
    setError('Введите название категории');
    return;
  }

  try {
    if (editingCategory) {
      await categoriesApi.update(editingCategory.id, {
        name: categoryName,
        description: categoryDescription,
      });
      setSuccess('Категория обновлена');
    } else {
      await categoriesApi.create({
        name: categoryName,
        description: categoryDescription,
      });
      setSuccess('Категория создана');
    }
```

```
    setShowCategoryModal(false);
    resetCategoryForm();
    loadData();
  } catch (err) {
    setError(err.response?.data?.error || 'Ошибка сохранения');
  }
};

const handleEditCategory = (category) => {
  setEditingCategory(category);
  setCategoryName(category.name);
  setCategoryDescription(category.description || '');
  setShowCategoryModal(true);
};

const handleDeleteCategory = async (id) => {
  if (!confirm('Удалить эту категорию?')) return;

  try {
    await categoriesApi.delete(id);
    setSuccess('Категория удалена');
    loadData();
  } catch (err) {
    setError(err.response?.data?.error || 'Ошибка удаления');
  }
};

const resetCategoryForm = () => {
  setEditingCategory(null);
  setCategoryName('');
  setCategoryDescription('');
};

// Tag handlers
const handleTagSubmit = async (e) => {
  e.preventDefault();
  setError('');

  if (!tagName.trim()) {
    setError('Введите название тега');
    return;
  }

  try {
    await categoriesApi.createTag({ name: tagName });
    setSuccess('Тег создан');
    setShowTagModal(false);
    setTagName('');
    loadData();
  } catch (err) {
    setError(err.response?.data?.error || 'Ошибка создания тега');
  }
};
```

```
  const handleDeleteTag = async (id) => {
    if (!confirm('Удалить этот тег?')) return;

    try {
      await categoriesApi.deleteTag(id);
      setSuccess('Тег удален');
      loadData();
    } catch (err) {
      setError(err.response?.data?.error || 'Ошибка удаления');
    }
  };

  if (loading) {
    return (
      <Layout>
        <div className="text-center py-8">Загрузка...</div>
      </Layout>
    );
  }

  return (
    <Layout>
      <h1 className="text-2xl font-bold text-gray-800 mb-6">
        Управление категориями и тегами
      </h1>

      {/* Уведомления */}
      {error && (
        <div className="mb-4 p-4 bg-red-100 border border-red-400 text-
red-700 rounded">
          {error}
          <button onClick={() => setError('')} className="float-right">
            &times;
          </button>
        </div>
      )}
      {success && (
        <div className="mb-4 p-4 bg-green-100 border border-green-400
text-green-700 rounded">
          {success}
          <button onClick={() => setSuccess('')} className="float-right">
            &times;
          </button>
        </div>
      )}

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        {/* Категории */}
        <div className="card">
          <div className="flex justify-between items-center mb-4">
            <h2 className="text-lg font-semibold">Категории</h2>
            <button
              onClick={() => {
                resetCategoryForm();
```

```
                setShowCategoryModal(true);
              }}
              className="btn btn-primary"
            >
              Добавить
            </button>
          </div>

          {categories.length === 0 ? (
            <div className="text-center py-8 text-gray-500">Нет
категорий</div>
          ) : (
            <div className="space-y-3">
              {categories.map((category) => (
                <div
                  key={category.id}
                  className="flex items-center justify-between p-3 bg-
gray-50 rounded-lg"
                >
                  <div>
                    <div className="font-medium">{category.name}</div>
                    {category.description && (
                      <div className="text-sm text-gray-500">
                        {category.description}
                      </div>
                    )}
                  </div>
                  <div className="flex space-x-2">
                    <button
                      onClick={() => handleEditCategory(category)}
                      className="text-blue-600 hover:underline text-sm"
                    >
                      Изменить
                    </button>
                    <button
                      onClick={() => handleDeleteCategory(category.id)}
                      className="text-red-600 hover:underline text-sm"
                    >
                      Удалить
                    </button>
                  </div>
                </div>
              ))}
            </div>
          )}
        </div>

        {/* Теги */}
        <div className="card">
          <div className="flex justify-between items-center mb-4">
            <h2 className="text-lg font-semibold">Теги</h2>
            <button
              onClick={() => setShowTagModal(true)}
              className="btn btn-primary"
```

```
            >
              Добавить
            </button>
          </div>

          {tags.length === 0 ? (
            <div className="text-center py-8 text-gray-500">Нет
тегов</div>
          ) : (
            <div className="flex flex-wrap gap-2">
              {tags.map((tag) => (
                <div
                  key={tag.id}
                  className="flex items-center space-x-2 px-3 py-1.5 bg-
blue-100 text-blue-800 rounded-full"
                >
                  <span>{tag.name}</span>
                  <button
                    onClick={() => handleDeleteTag(tag.id)}
                    className="text-blue-600 hover:text-blue-800"
                  >
                    &times;
                  </button>
                </div>
              ))}
            </div>
          )}
        </div>
      </div>

      {/* Модальное окно категории */}
      {showCategoryModal && (
        <div className="fixed inset-0 bg-black bg-opacity-50 flex items-
center justify-center z-50">
          <div className="bg-white rounded-lg p-6 max-w-md w-full mx-4">
            <h3 className="text-lg font-semibold mb-4">
              {editingCategory ? 'Редактирование категории' : 'Новая
категория'}
            </h3>

            <form onSubmit={handleCategorySubmit} className="space-y-4">
              <div>
                <label className="block text-sm font-medium text-gray-700
mb-1">
                  Название *
                </label>
                <input
                  type="text"
                  value={categoryName}
                  onChange={(e) => setCategoryName(e.target.value)}
                  className="input"
                  required
                />
              </div>
```

```
              <div>
                <label className="block text-sm font-medium text-gray-700
mb-1">
                  Описание
                </label>
                <textarea
                  value={categoryDescription}
                  onChange={(e) =>
setCategoryDescription(e.target.value)}
                  className="input"
                  rows={3}
                />
              </div>

              <div className="flex space-x-2 pt-4">
                <button type="submit" className="btn btn-primary flex-1">
                  {editingCategory ? 'Сохранить' : 'Создать'}
                </button>
                <button
                  type="button"
                  onClick={() => setShowCategoryModal(false)}
                  className="btn btn-secondary flex-1"
                >
                  Отмена
                </button>
              </div>
            </form>
          </div>
        </div>
      )}

      {/* Модальное окно тега */}
      {showTagModal && (
        <div className="fixed inset-0 bg-black bg-opacity-50 flex items-
center justify-center z-50">
          <div className="bg-white rounded-lg p-6 max-w-md w-full mx-4">
            <h3 className="text-lg font-semibold mb-4">Новый тег</h3>

            <form onSubmit={handleTagSubmit} className="space-y-4">
              <div>
                <label className="block text-sm font-medium text-gray-700
mb-1">
                  Название *
                </label>
                <input
                  type="text"
                  value={tagName}
                  onChange={(e) => setTagName(e.target.value)}
                  className="input"
                  required
                />
              </div>
```

```
              <div className="flex space-x-2 pt-4">
                <button type="submit" className="btn btn-primary flex-1">
                  Создать
                </button>
                <button
                  type="button"
                  onClick={() => {
                    setShowTagModal(false);
                    setTagName('');
                  }}
                  className="btn btn-secondary flex-1"
                >
                  Отмена
                </button>
              </div>
            </form>
          </div>
        </div>
      )}
    </Layout>
  );
}
```

client/src/pages/DocumentPage.jsx

```
import { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { documentsApi, getApiUrl } from '../services/api';
import { useAuth } from '../contexts/AuthContext';
import Layout from '../components/Layout';

// Функция получения токена
const getToken = () => {
  return localStorage.getItem('token');
};

export default function DocumentPage() {
  const { id } = useParams();
  const navigate = useNavigate();
  const { user } = useAuth();

  const [document, setDocument] = useState(null);
  const [versions, setVersions] = useState([]);
  const [comments, setComments] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [isFavorite, setIsFavorite] = useState(false);
  const [newComment, setNewComment] = useState('');
  const [activeTab, setActiveTab] = useState('info');

  // Для обновления документа
  const [showUpdateModal, setShowUpdateModal] = useState(false);
  const [updateFile, setUpdateFile] = useState(null);
  const [updateComment, setUpdateComment] = useState('');
```

```
  const [updating, setUpdating] = useState(false);

  useEffect(() => {
    loadDocument();
  }, [id]);

  const loadDocument = async () => {
    setLoading(true);
    try {
      const [docRes, versionsRes, commentsRes, favoritesRes] = await
Promise.all([
        documentsApi.getById(id),
        documentsApi.getVersions(id),
        documentsApi.getComments(id),
        documentsApi.getFavorites(),
      ]);

      setDocument(docRes.data);
      setVersions(versionsRes.data);
      setComments(commentsRes.data);
      setIsFavorite(favoritesRes.data.some(f => f.id === parseInt(id)));
    } catch (err) {
      setError('Ошибка загрузки документа');
    }
    setLoading(false);
  };

  const handleDownload = async () => {
    try {
      const response = await documentsApi.download(id);
      const url = window.URL.createObjectURL(new Blob([response.data]));
      const link = window.document.createElement('a');
      link.href = url;
      link.download = document.file_name ||
`${document.code}.${document.file_path?.split('.').pop()}`;
      link.click();
      window.URL.revokeObjectURL(url);
    } catch (err) {
      setError('Ошибка скачивания');
    }
  };

  const handleDownloadVersion = async (versionId) => {
    try {
      const response = await documentsApi.downloadVersion(id, versionId);
      const url = window.URL.createObjectURL(new Blob([response.data]));
      const link = window.document.createElement('a');
      link.href = url;

      // Находим версию для имени файла
      const version = versions.find(v => v.id === versionId);
      const fileName = version?.file_name ||
`${document.code}_v${version?.version}.${version?.file_path?.split('.').p
op()}`;
```

```
    link.download = fileName;
    link.click();
    window.URL.revokeObjectURL(url);
  } catch (err) {
    setError('Ошибка скачивания версии');
  }
};

const handleToggleFavorite = async () => {
  try {
    if (isFavorite) {
      await documentsApi.removeFromFavorites(id);
    } else {
      await documentsApi.addToFavorites(id);
    }
    setIsFavorite(!isFavorite);
  } catch (err) {
    setError('Ошибка обновления избранного');
  }
};

const handleAddComment = async (e) => {
  e.preventDefault();
  if (!newComment.trim()) return;

  try {
    await documentsApi.addComment(id, newComment);
    setNewComment('');
    const commentsRes = await documentsApi.getComments(id);
    setComments(commentsRes.data);
  } catch (err) {
    setError('Ошибка добавления комментария');
  }
};

const handleDeleteComment = async (commentId) => {
  if (!confirm('Удалить комментарий?')) return;

  try {
    await documentsApi.deleteComment(id, commentId);
    setComments(comments.filter(c => c.id !== commentId));
  } catch (err) {
    setError('Ошибка удаления комментария');
  }
};

const handleUpdateDocument = async (e) => {
  e.preventDefault();
  if (!updateFile) {
    setError('Выберите файл для обновления');
    return;
  }
```

```
    setUpdating(true);
    try {
      const formData = new FormData();
      formData.append('file', updateFile);
      if (updateComment) {
        formData.append('comment', updateComment);
      }

      await documentsApi.update(id, formData);
      setShowUpdateModal(false);
      setUpdateFile(null);
      setUpdateComment('');
      loadDocument();
    } catch (err) {
      setError(err.response?.data?.error || 'Ошибка обновления
документа');
    }
    setUpdating(false);
  };

  const formatDate = (dateString) => {
    return new Date(dateString).toLocaleString('ru-RU');
  };

  const formatFileSize = (bytes) => {
    if (!bytes) return '-';
    const k = 1024;
    const sizes = ['Б', 'КБ', 'МБ', 'ГБ'];
    const i = Math.floor(Math.log(bytes) / Math.log(k));
    return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + ' ' +
sizes[i];
  };

  const typeNames = {
    drawing: 'Чертеж',
    standard: 'Стандарт',
    specification: 'Спецификация',
    instruction: 'Инструкция',
    manual: 'Руководство',
    other: 'Другое',
  };

  const statusNames = {
    pending_approval: 'На согласовании',
    in_library: 'В библиотеке',
    archived: 'В архиве',
    withdrawn: 'Изъят',
  };

  const statusColors = {
    pending_approval: 'bg-yellow-100 text-yellow-800',
    in_library: 'bg-green-100 text-green-800',
    archived: 'bg-gray-100 text-gray-800',
    withdrawn: 'bg-red-100 text-red-800',
```

```
  };

  // Проверяем, можно ли обновлять документ
  const canUpdate = user &&
    (user.role === 'librarian' || user.role === 'admin') &&
    document?.status === 'in_library';

  // Получаем расширение файла
  const getFileExtension = () => {
    if (!document?.file_path) return '';
    return document.file_path.split('.').pop()?.toLowerCase() || '';
  };

  // Проверяем, поддерживается ли предпросмотр
  const getPreviewType = () => {
    const ext = getFileExtension();

    // Изображения
    if (['jpg', 'jpeg', 'png', 'gif', 'webp', 'svg',
'bmp'].includes(ext)) {
      return 'image';
    }

    // PDF
    if (ext === 'pdf') {
      return 'pdf';
    }

    // Текстовые файлы
    if (['txt', 'md', 'json', 'xml', 'csv', 'log', 'ini', 'cfg', 'yaml',
'yml'].includes(ext)) {
      return 'text';
    }

    // Код
    if (['js', 'ts', 'jsx', 'tsx', 'html', 'css', 'scss', 'py', 'java',
'c', 'cpp', 'h', 'sql'].includes(ext)) {
      return 'code';
    }

    // Office документы
    if (['doc', 'docx', 'xls', 'xlsx', 'ppt', 'pptx', 'odt', 'ods',
'odp'].includes(ext)) {
      return 'office';
    }

    // Видео
    if (['mp4', 'webm', 'ogg', 'mov'].includes(ext)) {
      return 'video';
    }

    // Аудио
    if (['mp3', 'wav', 'ogg', 'aac', 'm4a'].includes(ext)) {
      return 'audio';
```

```
    }

    return 'unsupported';
  };

  const renderPreview = () => {
    const previewType = getPreviewType();
    const token = getToken();
    const previewUrl =
`${getApiUrl()}/api/documents/${id}/preview?token=${encodeURIComponent(to
ken || '')}`;

    switch (previewType) {
      case 'image':
        return (
          <div className="flex items-center justify-center p-4 bg-gray-
100 rounded-lg">
            <img
              src={previewUrl}
              alt={document.title}
              className="max-w-full max-h-[600px] object-contain"
            />
          </div>
        );

      case 'pdf':
        return (
          <iframe
            src={previewUrl}
            className="w-full h-[700px] border rounded-lg"
            title={document.title}
          />
        );

      case 'video':
        return (
          <video
            controls
            className="w-full max-h-[500px] rounded-lg"
            src={previewUrl}
          >
            Ваш браузер не поддерживает воспроизведение видео.
          </video>
        );

      case 'audio':
        return (
          <div className="p-8 bg-gray-100 rounded-lg">
            <audio controls className="w-full">
              <source src={previewUrl} />
              Ваш браузер не поддерживает воспроизведение аудио.
            </audio>
          </div>
        );
```

```
      case 'office':
        return (
          <div className="text-center py-12 bg-gray-50 rounded-lg">
            <svg className="w-16 h-16 text-gray-300 mx-auto mb-4"
fill="none" stroke="currentColor" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M9 12h6m-6 4h6m2 5H7a2 2 0 01-2-2V5a2 2 0 012-
2h5.586a1 1 0 01.707.293l5.414 5.414a1 1 0 01.293.707V19a2 2 0 01-2 2z"
/>
            </svg>
            <p className="text-gray-600 mb-4">
              Предпросмотр Office документов (.{getFileExtension()})
недоступен в браузере
            </p>
            <button onClick={handleDownload} className="btn btn-primary">
              Скачать документ
            </button>
          </div>
        );

      case 'text':
      case 'code':
        return (
          <iframe
            src={previewUrl}
            className="w-full h-[500px] bg-white border rounded-lg"
            title={document.title}
          />
        );

      default:
        return (
          <div className="text-center py-12 bg-gray-50 rounded-lg">
            <svg className="w-16 h-16 text-gray-300 mx-auto mb-4"
fill="none" stroke="currentColor" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M9 12h6m-6 4h6m2 5H7a2 2 0 01-2-2V5a2 2 0 012-
2h5.586a1 1 0 01.707.293l5.414 5.414a1 1 0 01.293.707V19a2 2 0 01-2 2z"
/>
            </svg>
            <p className="text-gray-500 mb-4">
              Предпросмотр для файлов .{getFileExtension()} не
поддерживается
            </p>
            <button onClick={handleDownload} className="btn btn-primary">
              Скачать документ
            </button>
          </div>
        );
    }
  };

  if (loading) {
```

```
    return (
      <Layout>
        <div className="text-center py-8">Загрузка...</div>
      </Layout>
    );
  }

  if (!document) {
    return (
      <Layout>
        <div className="text-center py-8 text-red-600">{error ||
'Документ не найден'}</div>
      </Layout>
    );
  }

  return (
    <Layout>
      {error && (
        <div className="mb-4 p-4 bg-red-100 border border-red-400 text-
red-700 rounded">
          {error}
          <button onClick={() => setError('')} className="float-
right">&times;</button>
        </div>
      )}

      {/* Заголовок */}
      <div className="mb-6">
        <div className="flex items-start justify-between">
          <div>
            <div className="flex items-center space-x-3 mb-2">
              <span className={`px-2 py-1 rounded text-sm
${statusColors[document.status]}`}>
                {statusNames[document.status]}
              </span>
              <span className="text-sm text-gray-
500">v{document.version}</span>
            </div>
            <h1 className="text-2xl font-bold text-gray-
800">{document.title}</h1>
            <p className="text-gray-600 font-mono">{document.code}</p>
          </div>

          <div className="flex items-center space-x-2">
            <button
              onClick={handleToggleFavorite}
              className={`p-2 rounded-lg ${isFavorite ? 'text-red-500 bg-
red-50' : 'text-gray-400 hover:text-red-500 hover:bg-red-50'}`}
              title={isFavorite ? 'Удалить из избранного' : 'Добавить в
избранное'}
            >
              <svg className="w-6 h-6" fill={isFavorite ? 'currentColor'
: 'none'} stroke="currentColor" viewBox="0 0 24 24">
```

```
                <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M4.318 6.318a4.5 4.5 0 000 6.364L12 20.364l7.682-
7.682a4.5 4.5 0 00-6.364-6.364L12 7.636l-1.318-1.318a4.5 4.5 0 00-6.364
0z" />
              </svg>
            </button>

            {canUpdate && (
              <button
                onClick={() => setShowUpdateModal(true)}
                className="btn btn-secondary"
              >
                Обновить документ
              </button>
            )}

            <button onClick={handleDownload} className="btn btn-primary">
              Скачать
            </button>
          </div>
        </div>
      </div>

      {/* Табы */}
      <div className="flex space-x-4 mb-6 border-b">
        {[
          { id: 'info', label: 'Информация' },
          { id: 'preview', label: 'Просмотр' },
          { id: 'versions', label: `Версии (${versions.length})` },
          { id: 'comments', label: `Комментарии (${comments.length})` },
        ].map((tab) => (
          <button
            key={tab.id}
            onClick={() => setActiveTab(tab.id)}
            className={`pb-2 px-1 border-b-2 transition-colors ${
              activeTab === tab.id
                ? 'border-blue-600 text-blue-600'
                : 'border-transparent text-gray-500 hover:text-gray-700'
            }`}
          >
            {tab.label}
          </button>
        ))}
      </div>

      {/* Информация */}
      {activeTab === 'info' && (
        <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
          <div className="card">
            <h3 className="text-lg font-semibold mb-4">Основная
информация</h3>
            <dl className="space-y-3">
              <div className="flex">
                <dt className="w-40 text-gray-500">Тип:</dt>
```

```
                <dd className="font-
medium">{typeNames[document.type]}</dd>
              </div>
              <div className="flex">
                <dt className="w-40 text-gray-500">Категория:</dt>
                <dd className="font-medium">{document.category_name || '-
'}</dd>
              </div>
              <div className="flex">
                <dt className="w-40 text-gray-500">Размер файла:</dt>
                <dd className="font-
medium">{formatFileSize(document.file_size)}</dd>
              </div>
              <div className="flex">
                <dt className="w-40 text-gray-500">Формат:</dt>
                <dd className="font-medium
uppercase">{getFileExtension()}</dd>
              </div>
              <div className="flex">
                <dt className="w-40 text-gray-500">Создан:</dt>
                <dd className="font-
medium">{formatDate(document.created_at)}</dd>
              </div>
              <div className="flex">
                <dt className="w-40 text-gray-500">Обновлен:</dt>
                <dd className="font-
medium">{formatDate(document.updated_at)}</dd>
              </div>
              <div className="flex">
                <dt className="w-40 text-gray-500">Автор:</dt>
                <dd className="font-medium">{document.created_by_name ||
'-'}</dd>
              </div>
            </dl>
          </div>

          <div className="card">
            <h3 className="text-lg font-semibold mb-4">Описание</h3>
            <p className="text-gray-700 whitespace-pre-wrap">
              {document.description || 'Описание отсутствует'}
            </p>

            {document.tags && document.tags.length > 0 && (
              <div className="mt-4">
                <h4 className="text-sm font-medium text-gray-500 mb-
2">Теги:</h4>
                <div className="flex flex-wrap gap-2">
                  {document.tags.map((tag, index) => (
                    <span key={tag.id || index} className="px-2 py-1 bg-
blue-100 text-blue-800 rounded text-sm">
                      {typeof tag === 'object' ? tag.name : tag}
                    </span>
                  ))}
                </div>
```

```
              </div>
            )}
          </div>
        </div>
      )}

      {/* Просмотр */}
      {activeTab === 'preview' && (
        <div className="card">
          {renderPreview()}
        </div>
      )}

      {/* Версии */}
      {activeTab === 'versions' && (
        <div className="card">
          {versions.length === 0 ? (
            <p className="text-gray-500 text-center py-8">История версий
пуста</p>
          ) : (
            <div className="space-y-4">
              {versions.map((version) => (
                <div key={version.id} className="flex items-center
justify-between p-4 bg-gray-50 rounded-lg">
                  <div>
                    <div className="font-medium flex items-center">
                      Версия {version.version}
                      {version.version === document.version && (
                        <span className="ml-2 text-xs bg-green-100 text-
green-800 px-2 py-1 rounded">
                          Текущая
                        </span>
                      )}
                    </div>
                    <div className="text-sm text-gray-500">
                      {formatDate(version.created_at)} |
{version.created_by_name || 'Неизвестно'}
                    </div>
                    {version.change_description && (
                      <div className="text-sm text-gray-600 mt-
1">{version.change_description}</div>
                    )}
                  </div>
                  <div className="flex items-center space-x-4">
                    <span className="text-sm text-gray-500">
                      {formatFileSize(version.file_size)}
                    </span>
                    <button
                      onClick={() => handleDownloadVersion(version.id)}
                      className="text-blue-600 hover:text-blue-800 text-
sm font-medium"
                    >
                      Скачать
                    </button>
```

```
                  </div>
                </div>
              ))}
            </div>
          )}
        </div>
      )}

      {/* Комментарии */}
      {activeTab === 'comments' && (
        <div className="card">
          {/* Форма добавления комментария */}
          <form onSubmit={handleAddComment} className="mb-6">
            <textarea
              value={newComment}
              onChange={(e) => setNewComment(e.target.value)}
              placeholder="Написать комментарий..."
              className="input mb-2"
              rows={3}
            />
            <button type="submit" className="btn btn-primary">
              Отправить
            </button>
          </form>

          {/* Список комментариев */}
          {comments.length === 0 ? (
            <p className="text-gray-500 text-center py-4">Комментариев
пока нет</p>
          ) : (
            <div className="space-y-4">
              {comments.map((comment) => (
                <div key={comment.id} className="border-b pb-4">
                  <div className="flex items-center justify-between mb-
2">
                    <div className="font-medium text-gray-
800">{comment.author_name}</div>
                    <div className="flex items-center space-x-2">
                      <span className="text-sm text-gray-
500">{formatDate(comment.created_at)}</span>
                      {(user?.id === comment.user_id || user?.role ===
'admin') && (
                        <button
                          onClick={() => handleDeleteComment(comment.id)}
                          className="text-red-500 hover:text-red-700
text-sm"
                        >
                          Удалить
                        </button>
                      )}
                    </div>
                  </div>
                  <p className="text-gray-700">{comment.content}</p>
                </div>
```

```
              ))}
            </div>
          )}
        </div>
      )}

      {/* Модальное окно обновления документа */}
      {showUpdateModal && (
        <div className="fixed inset-0 bg-black bg-opacity-50 flex items-
center justify-center z-50">
          <div className="bg-white rounded-lg p-6 max-w-md w-full mx-4">
            <h3 className="text-lg font-semibold mb-4">Обновить
документ</h3>
            <p className="text-sm text-gray-600 mb-4">
              Загрузите новую версию документа. Текущая версия будет
сохранена в истории.
            </p>

            <form onSubmit={handleUpdateDocument} className="space-y-4">
              <div>
                <label className="block text-sm font-medium text-gray-700
mb-1">
                  Новый файл *
                </label>
                <input
                  type="file"
                  onChange={(e) => setUpdateFile(e.target.files?.[0] ||
null)}
                  className="input"
                  required
                />
              </div>

              <div>
                <label className="block text-sm font-medium text-gray-700
mb-1">
                  Комментарий к версии
                </label>
                <textarea
                  value={updateComment}
                  onChange={(e) => setUpdateComment(e.target.value)}
                  className="input"
                  rows={3}
                  placeholder="Опишите изменения..."
                />
              </div>

              <div className="flex space-x-2 pt-4">
                <button
                  type="submit"
                  disabled={updating}
                  className="btn btn-primary flex-1"
                >
                  {updating ? 'Загрузка...' : 'Обновить'}
```

```
                </button>
                <button
                  type="button"
                  onClick={() => {
                    setShowUpdateModal(false);
                    setUpdateFile(null);
                    setUpdateComment('');
                  }}
                  className="btn btn-secondary flex-1"
                >
                  Отмена
                </button>
              </div>
            </form>
          </div>
        </div>
      )}
    </Layout>
  );
}
```

client/src/pages/FavoritesPage.jsx

```
import { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import { documentsApi } from '../services/api';
import Layout from '../components/Layout';

export default function FavoritesPage() {
  const [favorites, setFavorites] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    loadFavorites();
  }, []);

  const loadFavorites = async () => {
    try {
      const response = await documentsApi.getFavorites();
      setFavorites(response.data);
    } catch (err) {
      setError('Ошибка загрузки избранного');
    }
    setLoading(false);
  };

  const handleRemoveFromFavorites = async (docId) => {
    try {
      await documentsApi.removeFromFavorites(docId);
      setFavorites(favorites.filter(f => f.id !== docId));
    } catch (err) {
      setError('Ошибка удаления из избранного');
    }
```

```
  };

  const typeNames = {
    drawing: 'Чертеж',
    standard: 'Стандарт',
    specification: 'Спецификация',
    instruction: 'Инструкция',
    manual: 'Руководство',
    other: 'Другое',
  };

  if (loading) {
    return (
      <Layout>
        <div className="text-center py-8">Загрузка...</div>
      </Layout>
    );
  }

  return (
    <Layout>
      <div className="mb-6">
        <h1 className="text-2xl font-bold text-gray-800">Избранное</h1>
        <p className="text-gray-600 mt-1">Ваши сохраненные документы</p>
      </div>

      {error && (
        <div className="mb-4 p-4 bg-red-100 border border-red-400 text-
red-700 rounded">
          {error}
        </div>
      )}

      {favorites.length === 0 ? (
        <div className="card text-center py-12">
          <svg className="w-16 h-16 text-gray-300 mx-auto mb-4"
fill="none" stroke="currentColor" viewBox="0 0 24 24">
            <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M4.318 6.318a4.5 4.5 0 000 6.364L12 20.364l7.682-
7.682a4.5 4.5 0 00-6.364-6.364L12 7.636l-1.318-1.318a4.5 4.5 0 00-6.364
0z" />
          </svg>
          <h3 className="text-lg font-medium text-gray-600 mb-2">Нет
избранных документов</h3>
          <p className="text-gray-500 mb-4">Добавляйте документы в
избранное для быстрого доступа</p>
          <Link to="/search" className="btn btn-primary">
            Найти документы
          </Link>
        </div>
      ) : (
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3
gap-4">
          {favorites.map((doc) => (
```

```
            <div key={doc.id} className="card hover:shadow-md transition-
shadow">
              <div className="flex justify-between items-start mb-2">
                <Link
                  to={`/documents/${doc.id}`}
                  className="text-lg font-semibold text-gray-800
hover:text-blue-600"
                >
                  {doc.title}
                </Link>
                <button
                  onClick={() => handleRemoveFromFavorites(doc.id)}
                  className="text-red-500 hover:text-red-700 p-1"
                  title="Удалить из избранного"
                >
                  <svg className="w-5 h-5" fill="currentColor" viewBox="0
0 24 24">
                    <path d="M4.318 6.318a4.5 4.5 0 000 6.364L12
20.364l7.682-7.682a4.5 4.5 0 00-6.364-6.364L12 7.636l-1.318-1.318a4.5 4.5
0 00-6.364 0z" />
                  </svg>
                </button>
              </div>

              <div className="flex items-center space-x-2 text-sm text-
gray-500 mb-2">
                <span className="font-mono">{doc.code}</span>
                <span>|</span>
                <span>{typeNames[doc.type]}</span>
              </div>

              {doc.description && (
                <p className="text-sm text-gray-600 line-clamp-
2">{doc.description}</p>
              )}

              {doc.category_name && (
                <div className="mt-2">
                  <span className="px-2 py-1 bg-gray-100 text-gray-600
rounded text-xs">
                    {doc.category_name}
                  </span>
                </div>
              )}
            </div>
          ))}
        </div>
      )}
    </Layout>
  );
}
```

client/src/pages/HomePage.jsx

```
import { useState, useEffect } from 'react';
import { Link, Navigate } from 'react-router-dom';
import { documentsApi, ticketsApi } from '../services/api';
import { useAuth } from '../contexts/AuthContext';
import { useSettings } from '../contexts/SettingsContext';
import Layout from '../components/Layout';

export default function HomePage() {
  const { user } = useAuth();
  const { settings } = useSettings();
  const [recentDocuments, setRecentDocuments] = useState([]);
  const [favorites, setFavorites] = useState([]);
  const [pendingTickets, setPendingTickets] = useState([]);
  const [loading, setLoading] = useState(true);

  // Admin сразу перенаправляется на страницу администрирования
  if (user?.role === 'admin') {
    return <Navigate to="/admin" />;
  }

  useEffect(() => {
    loadData();
  }, [user]);

  const loadData = async () => {
    try {
      const promises = [
        documentsApi.search({ status: 'in_library' }),
        documentsApi.getFavorites(),
      ];

      // Для библиотекаря и руководителя загружаем тикеты
      if (user?.role === 'librarian' || user?.role === 'department_head')
{
        promises.push(ticketsApi.getAll());
      }

      const results = await Promise.all(promises);

      setRecentDocuments(results[0].data.documents?.slice(0, 5) || []);
      setFavorites(results[1].data?.slice(0, 5) || []);

      if (results[2]) {
        const openTickets = results[2].data.filter(t => ['open',
'in_progress'].includes(t.status));
        setPendingTickets(openTickets.slice(0, 5));
      }
    } catch (err) {
      console.error('Ошибка загрузки данных:', err);
    }
    setLoading(false);
  };

  const typeNames = {
```

```
    drawing: 'Чертеж',
    standard: 'Стандарт',
    specification: 'Спецификация',
    instruction: 'Инструкция',
    manual: 'Руководство',
    other: 'Другое',
  };

  const roleNames = {
    admin: 'Администратор',
    librarian: 'Библиотекарь',
    department_head: 'Руководитель отдела',
    technical_specialist: 'Технический специалист',
  };

  const statusNames = {
    open: 'Открыт',
    in_progress: 'В работе',
  };

  return (
    <Layout>
      {/* Приветствие */}
      <div className="mb-8">
        <h1 className="text-2xl font-bold text-gray-800">
          Добро пожаловать, {user?.full_name}!
        </h1>
        <p className="text-gray-600 mt-1">
          {settings.app_name || 'Электронная библиотека НТД'} |
{roleNames[user?.role]}
        </p>
      </div>

      {/* Быстрые действия */}
      <div className="grid grid-cols-1 md:grid-cols-3 gap-4 mb-8">
        <Link
          to="/search"
          className="card hover:shadow-md transition-shadow flex items-
center space-x-4"
        >
          <div className="w-12 h-12 bg-blue-100 rounded-lg flex items-
center justify-center">
            <svg className="w-6 h-6 text-blue-600" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z" />
            </svg>
          </div>
          <div>
            <h3 className="font-semibold text-gray-800">Поиск
документов</h3>
            <p className="text-sm text-gray-500">Найти нужный
документ</p>
          </div>
```

```
        </Link>

        {user?.role === 'librarian' && (
          <Link
            to="/upload"
            className="card hover:shadow-md transition-shadow flex items-
center space-x-4"
          >
            <div className="w-12 h-12 bg-green-100 rounded-lg flex items-
center justify-center">
              <svg className="w-6 h-6 text-green-600" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M12 4v16m8-8H4" />
              </svg>
            </div>
            <div>
              <h3 className="font-semibold text-gray-800">Загрузить
документ</h3>
              <p className="text-sm text-gray-500">Добавить новый
документ</p>
            </div>
          </Link>
        )}

        {(user?.role === 'librarian' || user?.role === 'department_head')
&& (
          <Link
            to="/tickets"
            className="card hover:shadow-md transition-shadow flex items-
center space-x-4"
          >
            <div className="w-12 h-12 bg-yellow-100 rounded-lg flex
items-center justify-center">
              <svg className="w-6 h-6 text-yellow-600" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M9 5H7a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002-2V7a2
2 0 00-2-2h-2M9 5a2 2 0 002 2h2a2 2 0 002-2M9 5a2 2 0 012-2h2a2 2 0 012
2" />
              </svg>
            </div>
            <div>
              <h3 className="font-semibold text-gray-
800">Согласование</h3>
              <p className="text-sm text-gray-500">
                {pendingTickets.length > 0 ? `${pendingTickets.length} на
рассмотрении` : 'Документы на рассмотрении'}
              </p>
            </div>
          </Link>
        )}
      </div>
```

```
      {loading ? (
        <div className="text-center py-8">Загрузка...</div>
      ) : (
        <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
          {/* Избранное */}
          <div className="card">
            <div className="flex justify-between items-center mb-4">
              <h2 className="text-lg font-semibold text-gray-
800">Избранное</h2>
            </div>

            {favorites.length === 0 ? (
              <div className="text-center py-8 text-gray-500">
                У вас пока нет избранных документов
              </div>
            ) : (
              <div className="space-y-3">
                {favorites.map((doc) => (
                  <Link
                    key={doc.id}
                    to={`/documents/${doc.id}`}
                    className="block p-3 bg-gray-50 rounded-lg hover:bg-
gray-100 transition-colors"
                  >
                    <div className="font-medium text-gray-
800">{doc.title}</div>
                    <div className="flex items-center space-x-2 text-sm
text-gray-500 mt-1">
                      <span className="font-mono">{doc.code}</span>
                      <span>•</span>
                      <span>{typeNames[doc.type]}</span>
                    </div>
                  </Link>
                ))}
              </div>
            )}
          </div>

          {/* Для библиотекаря/руководителя - тикеты на согласовании */}
          {(user?.role === 'librarian' || user?.role ===
'department_head') && pendingTickets.length > 0 ? (
            <div className="card">
              <div className="flex justify-between items-center mb-4">
                <h2 className="text-lg font-semibold text-gray-800">На
согласовании</h2>
                <Link to="/tickets" className="text-blue-600
hover:underline text-sm">
                  Все тикеты
                </Link>
              </div>

              <div className="space-y-3">
                {pendingTickets.map((ticket) => (
                  <div
```

```
                    key={ticket.id}
                    onClick={() => window.location.href =
`/tickets/${ticket.id}`}
                    className="block p-3 bg-yellow-50 rounded-lg
hover:bg-yellow-100 transition-colors cursor-pointer"
                  >
                    <div className="flex items-center justify-between">
                      <div className="font-medium text-gray-
800">{ticket.document_title}</div>
                      <span className="text-xs px-2 py-1 bg-yellow-200
text-yellow-800 rounded">
                        {statusNames[ticket.status]}
                      </span>
                    </div>
                    <div className="text-sm text-gray-500 mt-1">
                      {ticket.document_code} • От:
{ticket.created_by_name}
                    </div>
                  </div>
                ))}
              </div>
            </div>
          ) : (
            /* Последние документы */
            <div className="card">
              <div className="flex justify-between items-center mb-4">
                <h2 className="text-lg font-semibold text-gray-
800">Последние документы</h2>
                <Link to="/search" className="text-blue-600
hover:underline text-sm">
                  Все документы
                </Link>
              </div>

              {recentDocuments.length === 0 ? (
                <div className="text-center py-8 text-gray-500">
                  В библиотеке пока нет документов
                </div>
              ) : (
                <div className="space-y-3">
                  {recentDocuments.map((doc) => (
                    <Link
                      key={doc.id}
                      to={`/documents/${doc.id}`}
                      className="block p-3 bg-gray-50 rounded-lg
hover:bg-gray-100 transition-colors"
                    >
                      <div className="font-medium text-gray-
800">{doc.title}</div>
                      <div className="flex items-center space-x-2 text-sm
text-gray-500 mt-1">
                        <span className="font-mono">{doc.code}</span>
                        <span>•</span>
                        <span>{typeNames[doc.type]}</span>
```

```
                        {doc.category_name && (
                          <>
                            <span>•</span>
                            <span>{doc.category_name}</span>
                          </>
                        )}
                      </div>
                    </Link>
                  ))}
                </div>
              )}
            </div>
          )}
        </div>
      )}
    </Layout>
  );
}
```

client/src/pages/LibrarianPage.jsx

```
import { useState, useEffect } from 'react';
import { categoriesApi, documentsApi } from '../services/api';
import Layout from '../components/Layout';

export default function LibrarianPage() {
  const [activeTab, setActiveTab] = useState('categories');
  const [categories, setCategories] = useState([]);
  const [tags, setTags] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  // Модальное окно
  const [showModal, setShowModal] = useState(false);
  const [modalType, setModalType] = useState('');
  const [editingItem, setEditingItem] = useState(null);

  // Загрузка данных
  useEffect(() => {
    if (activeTab === 'categories') {
      loadCategories();
    } else if (activeTab === 'tags') {
      loadTags();
    }
  }, [activeTab]);

  const loadCategories = async () => {
    setLoading(true);
    setError('');
    try {
      const response = await categoriesApi.getAll();
      setCategories(response.data);
    } catch (err) {
      setError('Ошибка загрузки категорий');
```

```
    }
    setLoading(false);
  };

  const loadTags = async () => {
    setLoading(true);
    setError('');
    try {
      const response = await categoriesApi.getAllTags();
      setTags(response.data);
    } catch (err) {
      setError('Ошибка загрузки тегов');
    }
    setLoading(false);
  };

  // Категории
  const handleCreateCategory = () => {
    setEditingItem({ name: '', description: '' });
    setModalType('createCategory');
    setShowModal(true);
  };

  const handleEditCategory = (category) => {
    setEditingItem(category);
    setModalType('editCategory');
    setShowModal(true);
  };

  const handleDeleteCategory = async (id) => {
    if (!confirm('Вы уверены, что хотите удалить эту категорию?'))
return;
    try {
      await categoriesApi.delete(id);
      loadCategories();
    } catch (err) {
      alert(err.response?.data?.error || 'Ошибка удаления');
    }
  };

  // Теги
  const handleCreateTag = () => {
    setEditingItem({ name: '' });
    setModalType('createTag');
    setShowModal(true);
  };

  const handleEditTag = (tag) => {
    setEditingItem(tag);
    setModalType('editTag');
    setShowModal(true);
  };

  const handleDeleteTag = async (id) => {
```

```
    if (!confirm('Вы уверены, что хотите удалить этот тег?')) return;
    try {
      await categoriesApi.deleteTag(id);
      loadTags();
    } catch (err) {
      alert(err.response?.data?.error || 'Ошибка удаления');
    }
  };

  // Сохранение
  const handleSave = async () => {
    try {
      if (modalType === 'createCategory') {
        await categoriesApi.create(editingItem);
        loadCategories();
      } else if (modalType === 'editCategory') {
        await categoriesApi.update(editingItem.id, editingItem);
        loadCategories();
      } else if (modalType === 'createTag') {
        await categoriesApi.createTag(editingItem);
        loadTags();
      } else if (modalType === 'editTag') {
        await categoriesApi.updateTag(editingItem.id, editingItem);
        loadTags();
      }
      setShowModal(false);
      setEditingItem(null);
    } catch (err) {
      alert(err.response?.data?.error || 'Ошибка сохранения');
    }
  };

  // Формирование отчета
  const handleGenerateReport = async () => {
    try {
      const response = await documentsApi.generateReport();
      // Создаем ссылку для скачивания
      const blob = new Blob([response.data], { type:
'text/csv;charset=utf-8;' });
      const url = window.URL.createObjectURL(blob);
      const link = document.createElement('a');
      link.href = url;
      link.download = `report_${new
Date().toISOString().split('T')[0]}.csv`;
      link.click();
      window.URL.revokeObjectURL(url);
    } catch (err) {
      alert('Ошибка формирования отчета');
    }
  };

  return (
    <Layout>
      <h1 className="text-2xl font-bold text-gray-800 mb-6">
```

```
        Управление библиотекой
      </h1>

      {/* Табы */}
      <div className="flex space-x-4 mb-6">
        <button
          onClick={() => setActiveTab('categories')}
          className={`px-4 py-2 rounded-lg font-medium ${
            activeTab === 'categories'
              ? 'bg-blue-600 text-white'
              : 'bg-gray-200 text-gray-700 hover:bg-gray-300'
          }`}
        >
          Категории
        </button>
        <button
          onClick={() => setActiveTab('tags')}
          className={`px-4 py-2 rounded-lg font-medium ${
            activeTab === 'tags'
              ? 'bg-blue-600 text-white'
              : 'bg-gray-200 text-gray-700 hover:bg-gray-300'
          }`}
        >
          Теги
        </button>
        <button
          onClick={() => setActiveTab('reports')}
          className={`px-4 py-2 rounded-lg font-medium ${
            activeTab === 'reports'
              ? 'bg-blue-600 text-white'
              : 'bg-gray-200 text-gray-700 hover:bg-gray-300'
          }`}
        >
          Отчеты
        </button>
      </div>

      {error && (
        <div className="mb-4 p-4 bg-red-100 border border-red-400 text-
red-700 rounded">
          {error}
        </div>
      )}

      {loading && activeTab !== 'reports' && (
        <div className="text-center py-8">Загрузка...</div>
      )}

      {/* Категории */}
      {!loading && activeTab === 'categories' && (
        <div className="card">
          <div className="mb-4">
            <button onClick={handleCreateCategory} className="btn btn-
primary">
```

```
              Добавить категорию
            </button>
          </div>

          <div className="overflow-x-auto">
            <table className="w-full">
              <thead>
                <tr className="border-b">
                  <th className="text-left py-3 px-4 font-medium text-
gray-600">Название</th>
                  <th className="text-left py-3 px-4 font-medium text-
gray-600">Описание</th>
                  <th className="text-left py-3 px-4 font-medium text-
gray-600">Документов</th>
                  <th className="text-left py-3 px-4 font-medium text-
gray-600">Действия</th>
                </tr>
              </thead>
              <tbody>
                {categories.map((category) => (
                  <tr key={category.id} className="border-b hover:bg-
gray-50">
                    <td className="py-3 px-4 font-
medium">{category.name}</td>
                    <td className="py-3 px-4 text-gray-
600">{category.description || '-'}</td>
                    <td className="py-3 px-4">{category.document_count ||
0}</td>
                    <td className="py-3 px-4">
                      <div className="flex space-x-2">
                        <button
                          onClick={() => handleEditCategory(category)}
                          className="px-3 py-1 bg-blue-600 text-white
text-sm rounded hover:bg-blue-700"
                        >
                          Изменить
                        </button>
                        <button
                          onClick={() =>
handleDeleteCategory(category.id)}
                          className="px-3 py-1 bg-red-600 text-white
text-sm rounded hover:bg-red-700"
                        >
                          Удалить
                        </button>
                      </div>
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        </div>
      )}
```

```
      {/* Теги */}
      {!loading && activeTab === 'tags' && (
        <div className="card">
          <div className="mb-4">
            <button onClick={handleCreateTag} className="btn btn-
primary">
              Добавить тег
            </button>
          </div>

          <div className="overflow-x-auto">
            <table className="w-full">
              <thead>
                <tr className="border-b">
                  <th className="text-left py-3 px-4 font-medium text-
gray-600">Название</th>
                  <th className="text-left py-3 px-4 font-medium text-
gray-600">Документов</th>
                  <th className="text-left py-3 px-4 font-medium text-
gray-600">Действия</th>
                </tr>
              </thead>
              <tbody>
                {tags.map((tag) => (
                  <tr key={tag.id} className="border-b hover:bg-gray-50">
                    <td className="py-3 px-4">
                      <span className="px-3 py-1 bg-gray-100 text-gray-
700 rounded-full">
                        {tag.name}
                      </span>
                    </td>
                    <td className="py-3 px-4">{tag.document_count ||
0}</td>
                    <td className="py-3 px-4">
                      <div className="flex space-x-2">
                        <button
                          onClick={() => handleEditTag(tag)}
                          className="px-3 py-1 bg-blue-600 text-white
text-sm rounded hover:bg-blue-700"
                        >
                          Изменить
                        </button>
                        <button
                          onClick={() => handleDeleteTag(tag.id)}
                          className="px-3 py-1 bg-red-600 text-white
text-sm rounded hover:bg-red-700"
                        >
                          Удалить
                        </button>
                      </div>
                    </td>
                  </tr>
                ))}
```

```
              </tbody>
            </table>
          </div>
        </div>
      )}

      {/* Отчеты */}
      {activeTab === 'reports' && (
        <div className="card">
          <h2 className="text-lg font-semibold mb-4">Формирование
отчетов</h2>

          <div className="space-y-4">
            <div className="p-4 border rounded-lg">
              <h3 className="font-medium mb-2">Отчет по документам</h3>
              <p className="text-gray-600 text-sm mb-3">
                Список всех документов в библиотеке с информацией о
статусе, категории и авторе.
              </p>
              <button onClick={handleGenerateReport} className="btn btn-
primary">
                Сформировать отчет (CSV)
              </button>
            </div>
          </div>
        </div>
      )}

      {/* Модальное окно */}
      {showModal && (
        <div className="fixed inset-0 bg-black bg-opacity-50 flex items-
center justify-center z-50">
          <div className="bg-white rounded-lg p-6 w-full max-w-md">
            <h2 className="text-xl font-bold mb-4">
              {modalType === 'createCategory' && 'Новая категория'}
              {modalType === 'editCategory' && 'Редактирование
категории'}
              {modalType === 'createTag' && 'Новый тег'}
              {modalType === 'editTag' && 'Редактирование тега'}
            </h2>

            {/* Форма категории */}
            {(modalType === 'createCategory' || modalType ===
'editCategory') && editingItem && (
              <div className="space-y-4">
                <div>
                  <label className="block text-sm font-medium text-gray-
700 mb-1">Название</label>
                  <input
                    type="text"
                    value={editingItem.name}
                    onChange={(e) => setEditingItem({ ...editingItem,
name: e.target.value })}
                    className="input"
```

```
                />
              </div>
              <div>
                <label className="block text-sm font-medium text-gray-
700 mb-1">Описание</label>
                <textarea
                  value={editingItem.description || ''}
                  onChange={(e) => setEditingItem({ ...editingItem,
description: e.target.value })}
                  className="input resize-none h-24"
                />
              </div>
            </div>
          )}

          {/* Форма тега */}
          {(modalType === 'createTag' || modalType === 'editTag') &&
editingItem && (
            <div className="space-y-4">
              <div>
                <label className="block text-sm font-medium text-gray-
700 mb-1">Название тега</label>
                <input
                  type="text"
                  value={editingItem.name}
                  onChange={(e) => setEditingItem({ ...editingItem,
name: e.target.value })}
                  className="input"
                />
              </div>
            </div>
          )}

          <div className="flex space-x-4 mt-6">
            <button onClick={handleSave} className="btn btn-primary">
              Сохранить
            </button>
            <button
              onClick={() => {
                setShowModal(false);
                setEditingItem(null);
              }}
              className="btn btn-secondary"
            >
              Отмена
            </button>
          </div>
        </div>
      </div>
    )}
  </Layout>
);
}
```

client/src/pages/LoginPage.jsx

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../contexts/AuthContext';

export default function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setLoading(true);

    try {
      await login(email, password);
      navigate('/');
    } catch (err) {
      setError(err.response?.data?.error || 'Ошибка входа');
    }

    setLoading(false);
  };

  return (
    <div className="min-h-screen bg-gray-100 flex items-center justify-
center p-4">
      <div className="max-w-md w-full bg-white rounded-lg shadow-md p-8">
        <div className="text-center mb-8">
          <div className="w-16 h-16 bg-blue-100 rounded-full flex items-
center justify-center mx-auto mb-4">
            <svg className="w-8 h-8 text-blue-600" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M12 6.253v13m0-13C10.832 5.477 9.246 5 7.5 5S4.168
5.477 3 6.253v13C4.168 18.477 5.754 18 7.5 18s3.332.477 4.5 1.253m0-
13C13.168 5.477 14.754 5 16.5 5c1.747 0 3.332.477 4.5 1.253v13C19.832
18.477 18.247 18 16.5 18c-1.746 0-3.332.477-4.5 1.253" />
            </svg>
          </div>
          <h1 className="text-2xl font-bold text-gray-800">Техническая
библиотека</h1>
          <p className="text-gray-600 mt-1">Войдите в систему для
продолжения</p>
        </div>

        {error && (
          <div className="mb-4 p-3 bg-red-100 border border-red-400 text-
red-700 rounded text-sm">
```

```
            {error}
          </div>
        )}

        <form onSubmit={handleSubmit} className="space-y-4">
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-
1">
              Email
            </label>
            <input
              type="email"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              className="w-full px-3 py-2 border border-gray-300 rounded-
lg focus:ring-2 focus:ring-blue-500 focus:border-blue-500"
              required
            />
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-
1">
              Пароль
            </label>
            <input
              type="password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              className="w-full px-3 py-2 border border-gray-300 rounded-
lg focus:ring-2 focus:ring-blue-500 focus:border-blue-500"
              required
            />
          </div>

          <button
            type="submit"
            disabled={loading}
            className="w-full py-2 px-4 bg-blue-600 text-white font-
medium rounded-lg hover:bg-blue-700 disabled:opacity-50"
          >
            {loading ? 'Вход...' : 'Войти'}
          </button>
        </form>

        <p className="text-center mt-6 text-xs text-gray-500">
          Для получения доступа обратитесь к администратору
        </p>
      </div>
    </div>
  );
}
```

client/src/pages/ProfilePage.jsx

```
import { useState, useEffect, useRef } from 'react';
import { authApi, getApiUrl } from '../services/api';
import { useAuth } from '../contexts/AuthContext';
import Layout from '../components/Layout';

export default function ProfilePage() {
  const { user, logout } = useAuth();
  const fileInputRef = useRef(null);
  const [formData, setFormData] = useState({
    full_name: '',
    position: '',
    current_password: '',
    new_password: '',
    confirm_password: '',
  });
  const [currentUser, setCurrentUser] = useState(null);
  const [loading, setLoading] = useState(false);
  const [avatarLoading, setAvatarLoading] = useState(false);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState('');

  useEffect(() => {
    loadProfile();
  }, []);

  const loadProfile = async () => {
    try {
      const response = await authApi.getProfile();
      setCurrentUser(response.data);
      setFormData((prev) => ({
        ...prev,
        full_name: response.data.full_name || '',
        position: response.data.position || '',
      }));
    } catch (err) {
      console.error('Ошибка загрузки профиля:', err);
    }
  };

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData({ ...formData, [name]: value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setSuccess('');

    if (formData.new_password && formData.new_password !==
formData.confirm_password) {
      setError('Пароли не совпадают');
      return;
```

```
  }

  setLoading(true);

  try {
    const updateData = {
      full_name: formData.full_name,
      position: formData.position,
    };

    if (formData.new_password) {
      updateData.current_password = formData.current_password;
      updateData.new_password = formData.new_password;
    }

    await authApi.updateProfile(updateData);
    setSuccess('Профиль успешно обновлен');

    setFormData((prev) => ({
      ...prev,
      current_password: '',
      new_password: '',
      confirm_password: '',
    }));

    loadProfile();
  } catch (err) {
    setError(err.response?.data?.error || 'Ошибка обновления профиля');
  }

  setLoading(false);
};

const handleAvatarClick = () => {
  fileInputRef.current?.click();
};

const handleAvatarChange = async (e) => {
  const file = e.target.files?.[0];
  if (!file) return;

  if (file.size > 5 * 1024 * 1024) {
    setError('Размер файла не должен превышать 5 МБ');
    return;
  }

  setAvatarLoading(true);
  setError('');

  try {
    const formData = new FormData();
    formData.append('avatar', file);

    const response = await authApi.uploadAvatar(formData);
```

```
    setCurrentUser(response.data);
    setSuccess('Фото профиля обновлено');
  } catch (err) {
    setError(err.response?.data?.error || 'Ошибка загрузки фото');
  }

  setAvatarLoading(false);
  e.target.value = '';
};

const handleDeleteAvatar = async () => {
  if (!confirm('Удалить фото профиля?')) return;

  setAvatarLoading(true);
  setError('');

  try {
    const response = await authApi.deleteAvatar();
    setCurrentUser(response.data);
    setSuccess('Фото профиля удалено');
  } catch (err) {
    setError(err.response?.data?.error || 'Ошибка удаления фото');
  }

  setAvatarLoading(false);
};

const roleNames = {
  admin: 'Администратор',
  librarian: 'Библиотекарь',
  department_head: 'Руководитель отдела',
  technical_specialist: 'Технический специалист',
};

const getInitials = (name) => {
  if (!name) return 'U';
  const parts = name.split(' ');
  if (parts.length >= 2) {
    return (parts[0][0] + parts[1][0]).toUpperCase();
  }
  return name.substring(0, 2).toUpperCase();
};

const getAvatarColor = (name) => {
  if (!name) return 'bg-gray-400';
  const colors = [
    'bg-blue-500',
    'bg-green-500',
    'bg-yellow-500',
    'bg-red-500',
    'bg-purple-500',
    'bg-pink-500',
    'bg-indigo-500',
    'bg-teal-500',
```

```
    ];
    const index = name.charCodeAt(0) % colors.length;
    return colors[index];
  };

  const getAvatarUrl = (avatarPath) => {
    if (!avatarPath) return null;
    const apiUrl = getApiUrl();
    const baseUrl = apiUrl.replace('/api', '');
    return `${baseUrl}${avatarPath}`;
  };

  const displayUser = currentUser || user;

  return (
    <Layout>
      <div className="max-w-2xl mx-auto">
        <h1 className="text-2xl font-bold text-gray-800 mb-
6">Профиль</h1>

        {/* Карточка профиля */}
        <div className="card mb-6">
          <div className="flex items-center space-x-4">
            {/* Аватар */}
            <div className="relative">
              <input
                ref={fileInputRef}
                type="file"
                accept="image/jpeg,image/png,image/gif,image/webp"
                onChange={handleAvatarChange}
                className="hidden"
              />

              {displayUser?.avatar_url ? (
                <img
                  src={getAvatarUrl(displayUser.avatar_url)}
                  alt="Аватар"
                  className="w-20 h-20 rounded-full object-cover cursor-
pointer hover:opacity-80 transition-opacity"
                  onClick={handleAvatarClick}
                />
              ) : (
                <div
                  className={`w-20 h-20 rounded-full flex items-center
justify-center text-white text-2xl font-bold cursor-pointer
hover:opacity-80 transition-opacity
${getAvatarColor(displayUser?.full_name)}`}
                  onClick={handleAvatarClick}
                >
                  {avatarLoading ? (
                    <svg className="animate-spin h-6 w-6" viewBox="0 0 24
24">
                      <circle className="opacity-25" cx="12" cy="12"
r="10" stroke="currentColor" strokeWidth="4" fill="none" />
```

```
                      <path className="opacity-75" fill="currentColor"
d="M4 12a8 8 0 018-8V0C5.373 0 0 5.373 0 12h4zm2 5.291A7.962 7.962 0 014
12H0c0 3.042 1.135 5.824 3 7.938l3-2.647z" />
                    </svg>
                  ) : (
                    getInitials(displayUser?.full_name)
                  )}
                </div>
              )}

              {/* Кнопка удаления аватара */}
              {displayUser?.avatar_url && (
                <button
                  onClick={handleDeleteAvatar}
                  disabled={avatarLoading}
                  className="absolute -bottom-1 -right-1 w-6 h-6 bg-red-
500 text-white rounded-full flex items-center justify-center hover:bg-
red-600 text-sm"
                  title="Удалить фото"
                >
                  &times;
                </button>
              )}
            </div>

            <div className="flex-1">
              <h2 className="text-xl font-semibold text-gray-
800">{displayUser?.full_name}</h2>
              <p className="text-gray-600">{displayUser?.email}</p>
              <div className="flex items-center space-x-2 mt-1">
                <span className="px-2 py-1 bg-blue-100 text-blue-800
rounded text-sm">
                  {roleNames[displayUser?.role]}
                </span>
                {displayUser?.position && (
                  <span className="text-gray-500 text-
sm">{displayUser?.position}</span>
                )}
              </div>
              <p className="text-xs text-gray-400 mt-2">
                Нажмите на аватар, чтобы изменить фото
              </p>
            </div>
          </div>
        </div>

        {/* Форма редактирования */}
        <div className="card">
          <h2 className="text-lg font-semibold text-gray-800 mb-
4">Редактирование профиля</h2>

          {error && (
            <div className="mb-4 p-3 bg-red-100 border border-red-400
text-red-700 rounded text-sm">
```

```
              {error}
            </div>
          )}

          {success && (
            <div className="mb-4 p-3 bg-green-100 border border-green-400
text-green-700 rounded text-sm">
              {success}
            </div>
          )}

          <form onSubmit={handleSubmit} className="space-y-4">
            <div>
              <label className="block text-sm font-medium text-gray-700
mb-1">
                ФИО
              </label>
              <input
                type="text"
                name="full_name"
                value={formData.full_name}
                onChange={handleChange}
                className="input"
              />
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700
mb-1">
                Должность
              </label>
              <input
                type="text"
                name="position"
                value={formData.position}
                onChange={handleChange}
                className="input"
              />
            </div>

            <hr className="my-6" />

            <h3 className="text-md font-medium text-gray-800">Изменение
пароля</h3>
            <p className="text-sm text-gray-500 mb-4">
              Оставьте поля пустыми, если не хотите менять пароль
            </p>

            <div>
              <label className="block text-sm font-medium text-gray-700
mb-1">
                Текущий пароль
              </label>
              <input
```

```
                type="password"
                name="current_password"
                value={formData.current_password}
                onChange={handleChange}
                className="input"
              />
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700
mb-1">
                Новый пароль
              </label>
              <input
                type="password"
                name="new_password"
                value={formData.new_password}
                onChange={handleChange}
                className="input"
              />
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700
mb-1">
                Подтверждение пароля
              </label>
              <input
                type="password"
                name="confirm_password"
                value={formData.confirm_password}
                onChange={handleChange}
                className="input"
              />
            </div>

            <button
              type="submit"
              disabled={loading}
              className="btn btn-primary"
            >
              {loading ? 'Сохранение...' : 'Сохранить изменения'}
            </button>
          </form>
        </div>
      </div>
    </Layout>
  );
}
```

client/src/pages/RegisterPage.jsx

```
import { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';
```

```
import { useAuth } from '../contexts/AuthContext';

export default function RegisterPage() {
  const [formData, setFormData] = useState({
    email: '',
    password: '',
    confirmPassword: '',
    full_name: '',
    position: '',
  });
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const { register } = useAuth();
  const navigate = useNavigate();

  const handleChange = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value,
    });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');

    // Проверка паролей
    if (formData.password !== formData.confirmPassword) {
      setError('Пароли не совпадают');
      return;
    }

    if (formData.password.length < 6) {
      setError('Пароль должен быть не менее 6 символов');
      return;
    }

    setLoading(true);

    const result = await register({
      email: formData.email,
      password: formData.password,
      full_name: formData.full_name,
      position: formData.position,
      role: 'technical_specialist', // По умолчанию - технический
специалист
    });

    if (result.success) {
      navigate('/login');
    } else {
      setError(result.error);
    }
```

```
    setLoading(false);
  };

  return (
    <div className="min-h-screen bg-gray-100 flex items-center justify-
center px-4 py-8">
      <div className="max-w-md w-full">
        {/* Заголовок */}
        <div className="text-center mb-8">
          <h1 className="text-3xl font-bold text-gray-800">
            Регистрация
          </h1>
          <p className="text-gray-600 mt-2">
            Создайте аккаунт для доступа к библиотеке
          </p>
        </div>

        {/* Форма */}
        <div className="card">
          <form onSubmit={handleSubmit}>
            {/* Сообщение об ошибке */}
            {error && (
              <div className="mb-4 p-3 bg-red-100 border border-red-400
text-red-700 rounded">
                {error}
              </div>
            )}

            {/* ФИО */}
            <div className="mb-4">
              <label className="block text-sm font-medium text-gray-700
mb-1">
                ФИО
              </label>
              <input
                type="text"
                name="full_name"
                value={formData.full_name}
                onChange={handleChange}
                className="input"
                placeholder="Иванов Иван Иванович"
                required
              />
            </div>

            {/* Email */}
            <div className="mb-4">
              <label className="block text-sm font-medium text-gray-700
mb-1">
                Email
              </label>
              <input
                type="email"
```

```
                name="email"
                value={formData.email}
                onChange={handleChange}
                className="input"
                placeholder="example@library.ru"
                required
              />
            </div>

            {/* Должность */}
            <div className="mb-4">
              <label className="block text-sm font-medium text-gray-700
mb-1">
                Должность
              </label>
              <input
                type="text"
                name="position"
                value={formData.position}
                onChange={handleChange}
                className="input"
                placeholder="Инженер-конструктор"
              />
            </div>

            {/* Пароль */}
            <div className="mb-4">
              <label className="block text-sm font-medium text-gray-700
mb-1">
                Пароль
              </label>
              <input
                type="password"
                name="password"
                value={formData.password}
                onChange={handleChange}
                className="input"
                placeholder="Минимум 6 символов"
                required
              />
            </div>

            {/* Подтверждение пароля */}
            <div className="mb-6">
              <label className="block text-sm font-medium text-gray-700
mb-1">
                Подтверждение пароля
              </label>
              <input
                type="password"
                name="confirmPassword"
                value={formData.confirmPassword}
                onChange={handleChange}
                className="input"
```

```
                placeholder="Повторите пароль"
                required
              />
            </div>

            {/* Кнопка регистрации */}
            <button
              type="submit"
              disabled={loading}
              className="w-full btn btn-primary py-3 text-lg"
            >
              {loading ? 'Регистрация...' : 'Зарегистрироваться'}
            </button>
          </form>

          {/* Ссылка на вход */}
          <div className="mt-4 text-center text-sm text-gray-600">
            Уже есть аккаунт?{' '}
            <Link to="/login" className="text-blue-600 hover:underline">
              Войти
            </Link>
          </div>
        </div>
      </div>
    </div>
  );
}
```

client/src/pages/SearchPage.jsx

```
import { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import { documentsApi, categoriesApi } from '../services/api';
import Layout from '../components/Layout';

export default function SearchPage() {
  const [documents, setDocuments] = useState([]);
  const [categories, setCategories] = useState([]);
  const [tags, setTags] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  // Фильтры
  const [filters, setFilters] = useState({
    query: '',
    category_id: '',
    type: '',
    tag_id: '',
  });

  useEffect(() => {
    loadInitialData();
  }, []);
```

```
const loadInitialData = async () => {
  try {
    const [categoriesRes, tagsRes] = await Promise.all([
      categoriesApi.getAll(),
      categoriesApi.getAllTags(),
    ]);
    setCategories(categoriesRes.data);
    setTags(tagsRes.data);

    // Загружаем документы
    handleSearch();
  } catch (err) {
    console.error('Ошибка загрузки данных:', err);
  }
};

const handleSearch = async (e) => {
  if (e) e.preventDefault();

  setLoading(true);
  setError('');

  try {
    const params = {};
    if (filters.query) params.query = filters.query;
    if (filters.category_id) params.category_id = filters.category_id;
    if (filters.type) params.type = filters.type;
    if (filters.tag_id) params.tag_id = filters.tag_id;

    const response = await documentsApi.search(params);
    setDocuments(response.data.documents);
  } catch (err) {
    setError('Ошибка поиска');
  }
  setLoading(false);
};

const handleFilterChange = (field, value) => {
  setFilters({ ...filters, [field]: value });
};

const handleReset = () => {
  setFilters({
    query: '',
    category_id: '',
    type: '',
    tag_id: '',
  });
};

const formatDate = (dateString) => {
  return new Date(dateString).toLocaleDateString('ru-RU');
};
```

```
  const typeNames = {
    drawing: 'Чертеж',
    standard: 'Стандарт',
    specification: 'Спецификация',
    instruction: 'Инструкция',
    manual: 'Руководство',
    other: 'Другое',
  };

  const statusNames = {
    draft: 'Черновик',
    pending_approval: 'На согласовании',
    in_library: 'В библиотеке',
    pending_deletion: 'На удаление',
  };

  const statusColors = {
    draft: 'bg-gray-100 text-gray-800',
    pending_approval: 'bg-yellow-100 text-yellow-800',
    in_library: 'bg-green-100 text-green-800',
    pending_deletion: 'bg-red-100 text-red-800',
  };

  return (
    <Layout>
      <h1 className="text-2xl font-bold text-gray-800 mb-6">Поиск
документов</h1>

      {/* Форма поиска */}
      <form onSubmit={handleSearch} className="card mb-6">
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-5
gap-4">
          {/* Поисковая строка */}
          <div className="lg:col-span-2">
            <label className="block text-sm font-medium text-gray-700 mb-
1">
              Поиск
            </label>
            <input
              type="text"
              value={filters.query}
              onChange={(e) => handleFilterChange('query',
e.target.value)}
              placeholder="Название, код или описание..."
              className="input"
            />
          </div>

          {/* Категория */}
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-
1">
              Категория
            </label>
```

```
            <select
              value={filters.category_id}
              onChange={(e) => handleFilterChange('category_id',
e.target.value)}
              className="input"
            >
              <option value="">Все категории</option>
              {categories.map((cat) => (
                <option key={cat.id} value={cat.id}>{cat.name}</option>
              ))}
            </select>
          </div>

          {/* Тип */}
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-
1">
              Тип документа
            </label>
            <select
              value={filters.type}
              onChange={(e) => handleFilterChange('type',
e.target.value)}
              className="input"
            >
              <option value="">Все типы</option>
              <option value="drawing">Чертеж</option>
              <option value="standard">Стандарт</option>
              <option value="specification">Спецификация</option>
              <option value="instruction">Инструкция</option>
              <option value="manual">Руководство</option>
              <option value="other">Другое</option>
            </select>
          </div>

          {/* Тег */}
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-
1">
              Тег
            </label>
            <select
              value={filters.tag_id}
              onChange={(e) => handleFilterChange('tag_id',
e.target.value)}
              className="input"
            >
              <option value="">Все теги</option>
              {tags.map((tag) => (
                <option key={tag.id} value={tag.id}>{tag.name}</option>
              ))}
            </select>
          </div>
        </div>
```

```
        <div className="flex space-x-2 mt-4">
          <button type="submit" className="btn btn-primary">
            Найти
          </button>
          <button type="button" onClick={handleReset} className="btn btn-
secondary">
            Сбросить
          </button>
        </div>
      </form>

      {/* Ошибка */}
      {error && (
        <div className="mb-4 p-4 bg-red-100 border border-red-400 text-
red-700 rounded">
          {error}
        </div>
      )}

      {/* Результаты */}
      {loading ? (
        <div className="text-center py-8">Загрузка...</div>
      ) : documents.length === 0 ? (
        <div className="card text-center py-8 text-gray-500">
          Документы не найдены
        </div>
      ) : (
        <div className="space-y-4">
          <div className="text-sm text-gray-500 mb-2">
            Найдено документов: {documents.length}
          </div>

          {documents.map((doc) => (
            <Link
              key={doc.id}
              to={`/documents/${doc.id}`}
              className="card block hover:shadow-md transition-shadow"
            >
              <div className="flex items-start justify-between">
                <div className="flex-1">
                  <div className="flex items-center space-x-3 mb-1">
                    <span className="font-mono text-sm text-gray-
500">{doc.code}</span>
                    <span className={`px-2 py-0.5 rounded text-xs
${statusColors[doc.status]}`}>
                      {statusNames[doc.status]}
                    </span>
                  </div>

                  <h3 className="text-lg font-semibold text-gray-800 mb-
1">
                    {doc.title}
                  </h3>
```

```
                  <div className="flex flex-wrap items-center gap-x-4
gap-y-1 text-sm text-gray-600">
                    <span>Тип: {typeNames[doc.type]}</span>
                    {doc.category_name && <span>Категория:
{doc.category_name}</span>}
                    <span>Автор: {doc.author_name}</span>
                    <span>Версия: {doc.current_version}</span>
                  </div>

                  {/* Теги */}
                  {doc.tag_names && doc.tag_names.length > 0 && (
                    <div className="flex flex-wrap gap-1 mt-2">
                      {doc.tag_names.map((tagName, index) => (
                        <span
                          key={index}
                          className="px-2 py-0.5 bg-blue-100 text-blue-
700 rounded text-xs"
                        >
                          {tagName}
                        </span>
                      ))}
                    </div>
                  )}
                </div>

                <div className="text-right text-sm text-gray-500 ml-4">
                  <div>Обновлен: {formatDate(doc.updated_at)}</div>
                </div>
              </div>
            </Link>
          ))}
        </div>
      )}
    </Layout>
  );
}
```

client/src/pages/TicketDetailPage.jsx

```
import { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { ticketsApi, documentsApi, getApiUrl } from '../services/api';
import { useAuth } from '../contexts/AuthContext';
import Layout from '../components/Layout';

export default function TicketDetailPage() {
  const { id } = useParams();
  const navigate = useNavigate();
  const { user } = useAuth();

  const [ticket, setTicket] = useState(null);
  const [messages, setMessages] = useState([]);
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState('');
  const [newMessage, setNewMessage] = useState('');
  const [actionComment, setActionComment] = useState('');
  const [showActionModal, setShowActionModal] = useState(null);
  const [processing, setProcessing] = useState(false);

  useEffect(() => {
    loadTicket();
  }, [id]);

  const loadTicket = async () => {
    setLoading(true);
    try {
      const response = await ticketsApi.getById(id);
      setTicket(response.data.ticket);
      setMessages(response.data.messages || []);
    } catch (err) {
      setError('Ошибка загрузки тикета');
    }
    setLoading(false);
  };

  const handleSendMessage = async (e) => {
    e.preventDefault();
    if (!newMessage.trim()) return;

    try {
      await ticketsApi.addMessage(id, newMessage);
      setNewMessage('');
      loadTicket();
    } catch (err) {
      setError('Ошибка отправки сообщения');
    }
  };

  const handleAction = async (action) => {
    setProcessing(true);
    try {
      if (action === 'approve') {
        await ticketsApi.approve(id, actionComment);
      } else if (action === 'reject') {
        await ticketsApi.reject(id, actionComment);
      } else if (action === 'request_changes') {
        await ticketsApi.requestChanges(id, actionComment);
      }

      setShowActionModal(null);
      setActionComment('');
      loadTicket();
    } catch (err) {
      setError(err.response?.data?.error || 'Ошибка выполнения
действия');
    }
    setProcessing(false);
```

```
  };

  const handleDownload = async () => {
    try {
      const response = await documentsApi.download(ticket.document_id);
      const url = window.URL.createObjectURL(new Blob([response.data]));
      const link = window.document.createElement('a');
      link.href = url;
      link.download = ticket.file_name || ticket.document_title ||
'document';
      link.click();
      window.URL.revokeObjectURL(url);
    } catch (err) {
      setError('Ошибка скачивания');
    }
  };

  const formatDate = (dateString) => {
    return new Date(dateString).toLocaleString('ru-RU');
  };

  const formatFileSize = (bytes) => {
    if (!bytes) return '-';
    const k = 1024;
    const sizes = ['Б', 'КБ', 'МБ', 'ГБ'];
    const i = Math.floor(Math.log(bytes) / Math.log(k));
    return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + ' ' +
sizes[i];
  };

  const statusNames = {
    open: 'Открыт',
    in_progress: 'В работе',
    resolved: 'Согласован',
    closed: 'Закрыт',
  };

  const statusColors = {
    open: 'bg-blue-100 text-blue-800',
    in_progress: 'bg-yellow-100 text-yellow-800',
    resolved: 'bg-green-100 text-green-800',
    closed: 'bg-gray-100 text-gray-800',
  };

  const typeNames = {
    drawing: 'Чертеж',
    standard: 'Стандарт',
    specification: 'Спецификация',
    instruction: 'Инструкция',
    manual: 'Руководство',
    other: 'Другое',
  };

  const canApprove = user?.role === 'department_head' &&
```

```
    ['open', 'in_progress'].includes(ticket?.status);

  // Получаем расширение файла
  const getFileExtension = () => {
    if (!ticket?.file_path && !ticket?.file_name) return '';
    const fileName = ticket.file_name || ticket.file_path || '';
    return fileName.split('.').pop()?.toLowerCase() || '';
  };

  // Функция рендеринга предпросмотра
  const renderPreview = () => {
    if (!ticket?.document_id) {
      return (
        <div className="text-center py-8 bg-gray-50 rounded-lg">
          <p className="text-gray-500">Документ не найден</p>
        </div>
      );
    }

    const token = localStorage.getItem('token');
    const previewUrl =
`${getApiUrl()}/api/documents/${ticket.document_id}/preview?token=${encod
eURIComponent(token || '')}`;
    const ext = getFileExtension();

    // Изображения
    if (['jpg', 'jpeg', 'png', 'gif', 'webp', 'svg', 'bmp', 'ico',
'tiff', 'tif', 'avif'].includes(ext)) {
      return (
        <div className="flex items-center justify-center p-4 bg-gray-100
rounded-lg">
          <img
            src={previewUrl}
            alt={ticket.document_title}
            className="max-w-full max-h-[500px] object-contain"
          />
        </div>
      );
    }

    // PDF
    if (ext === 'pdf') {
      return (
        <iframe
          src={previewUrl}
          className="w-full h-[600px] border rounded-lg"
          title={ticket.document_title}
        />
      );
    }

    // Видео
    if (['mp4', 'webm', 'ogg', 'ogv', 'mov', 'avi', 'mkv',
'm4v'].includes(ext)) {
```

```
      return (
        <video
          controls
          className="w-full max-h-[500px] rounded-lg"
          src={previewUrl}
        >
          Ваш браузер не поддерживает воспроизведение видео.
        </video>
      );
    }

    // Аудио
    if (['mp3', 'wav', 'oga', 'm4a', 'aac', 'flac', 'wma', 'ogg',
'opus'].includes(ext)) {
      return (
        <div className="p-6 bg-gray-100 rounded-lg">
          <div className="text-center mb-4">
            <svg className="w-12 h-12 text-gray-400 mx-auto" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M9 19V6l12-3v13M9 19c0 1.105-1.343 2-3 2s-3-.895-3-2
1.343-2 3-2 3 .895 3 2zm12-3c0 1.105-1.343 2-3 2s-3-.895-3-2 1.343-2 3-2
3 .895 3 2zM9 10l12-3" />
            </svg>
            <p className="mt-2 text-gray-600 text-
sm">{ticket.file_name}</p>
          </div>
          <audio controls className="w-full">
            <source src={previewUrl} />
            Ваш браузер не поддерживает воспроизведение аудио.
          </audio>
        </div>
      );
    }

    // Текст и код
    if (['txt', 'md', 'markdown', 'json', 'xml', 'csv', 'log', 'ini',
'cfg', 'yaml', 'yml', 'toml', 'env', 'js', 'ts', 'jsx', 'tsx', 'html',
'htm', 'css', 'scss', 'py', 'java', 'c', 'cpp', 'h', 'cs', 'go', 'rs',
'rb', 'php', 'sql', 'sh', 'bat'].includes(ext)) {
      return (
        <iframe
          src={previewUrl}
          className="w-full h-[400px] bg-white border rounded-lg font-
mono"
          title={ticket.document_title}
        />
      );
    }

    // Office документы
    if (['doc', 'docx', 'xls', 'xlsx', 'ppt', 'pptx', 'odt', 'ods',
'odp', 'rtf'].includes(ext)) {
      return (
```

```
        <div className="text-center py-8 bg-gray-50 rounded-lg">
          <svg className="w-12 h-12 text-blue-400 mx-auto mb-3"
fill="none" stroke="currentColor" viewBox="0 0 24 24">
            <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M9 12h6m-6 4h6m2 5H7a2 2 0 01-2-2V5a2 2 0 012-
2h5.586a1 1 0 01.707.293l5.414 5.414a1 1 0 01.293.707V19a2 2 0 01-2 2z"
/>
          </svg>
          <p className="text-gray-600 mb-1 font-
medium">{ticket.file_name}</p>
          <p className="text-gray-500 text-sm mb-3">
            Предпросмотр Office документов недоступен
          </p>
          <button onClick={handleDownload} className="btn btn-primary
btn-sm">
            Скачать документ
          </button>
        </div>
      );
    }

    // CAD файлы
    if (['dwg', 'dxf', 'dwf', 'step', 'stp', 'iges', 'igs',
'stl'].includes(ext)) {
      return (
        <div className="text-center py-8 bg-gray-50 rounded-lg">
          <svg className="w-12 h-12 text-orange-400 mx-auto mb-3"
fill="none" stroke="currentColor" viewBox="0 0 24 24">
            <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M4 5a1 1 0 011-1h14a1 1 0 011 1v2a1 1 0 01-1 1H5a1 1 0
01-1-1V5zM4 13a1 1 0 011-1h6a1 1 0 011 1v6a1 1 0 01-1 1H5a1 1 0 01-1-1v-
6zM16 13a1 1 0 011-1h2a1 1 0 011 1v6a1 1 0 01-1 1h-2a1 1 0 01-1-1v-6z" />
          </svg>
          <p className="text-gray-600 mb-1 font-
medium">{ticket.file_name}</p>
          <p className="text-gray-500 text-sm mb-3">
            CAD файлы требуют специализированного ПО
          </p>
          <button onClick={handleDownload} className="btn btn-primary
btn-sm">
            Скачать документ
          </button>
        </div>
      );
    }

    // Архивы
    if (['zip', 'rar', '7z', 'tar', 'gz', 'bz2'].includes(ext)) {
      return (
        <div className="text-center py-8 bg-gray-50 rounded-lg">
          <svg className="w-12 h-12 text-yellow-500 mx-auto mb-3"
fill="none" stroke="currentColor" viewBox="0 0 24 24">
```

```
            <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M5 8h14M5 8a2 2 0 110-4h14a2 2 0 110 4M5 8v10a2 2 0
002 2h10a2 2 0 002-2V8m-9 4h4" />
          </svg>
          <p className="text-gray-600 mb-1 font-
medium">{ticket.file_name}</p>
          <p className="text-gray-500 text-sm mb-3">
            Архивы нужно скачать и распаковать
          </p>
          <button onClick={handleDownload} className="btn btn-primary
btn-sm">
            Скачать архив
          </button>
        </div>
      );
    }

    // Не поддерживается
    return (
      <div className="text-center py-8 bg-gray-50 rounded-lg">
        <svg className="w-12 h-12 text-gray-300 mx-auto mb-3" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
          <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M9 12h6m-6 4h6m2 5H7a2 2 0 01-2-2V5a2 2 0 012-
2h5.586a1 1 0 01.707.293l5.414 5.414a1 1 0 01.293.707V19a2 2 0 01-2 2z"
/>
        </svg>
        <p className="text-gray-600 mb-1 font-
medium">{ticket.file_name}</p>
        <p className="text-gray-500 text-sm mb-3">
          Предпросмотр для .{ext} не поддерживается
        </p>
        <button onClick={handleDownload} className="btn btn-primary btn-
sm">
          Скачать документ
        </button>
      </div>
    );
  };

  // Получаем пометку типа сообщения
  const getMessageTypeLabel = (msgType) => {
    switch (msgType) {
      case 'approve':
        return { text: 'Согласовано', className: 'bg-green-100 text-
green-800 border border-green-300' };
      case 'reject':
        return { text: 'Отклонено', className: 'bg-red-100 text-red-800
border border-red-300' };
      case 'request_changes':
        return { text: 'Запрос на доработку', className: 'bg-yellow-100
text-yellow-800 border border-yellow-300' };
      default:
        return null;
```

```
    }
  };

  // Получаем стиль карточки сообщения
  const getMessageCardStyle = (msgType) => {
    switch (msgType) {
      case 'approve':
        return 'bg-green-50 border-l-4 border-green-500';
      case 'reject':
        return 'bg-red-50 border-l-4 border-red-500';
      case 'request_changes':
        return 'bg-yellow-50 border-l-4 border-yellow-500';
      default:
        return 'bg-gray-50 border-l-4 border-gray-300';
    }
  };

  if (loading) {
    return (
      <Layout>
        <div className="text-center py-8">Загрузка...</div>
      </Layout>
    );
  }

  if (!ticket) {
    return (
      <Layout>
        <div className="text-center py-8 text-red-600">{error || 'Тикет
не найден'}</div>
      </Layout>
    );
  }

  return (
    <Layout>
      {error && (
        <div className="mb-4 p-4 bg-red-100 border border-red-400 text-
red-700 rounded">
          {error}
          <button onClick={() => setError('')} className="float-
right">&times;</button>
        </div>
      )}

      {/* Заголовок */}
      <div className="mb-6">
        <button
          onClick={() => navigate('/tickets')}
          className="text-blue-600 hover:underline mb-2 flex items-
center"
        >
          <svg className="w-4 h-4 mr-1" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
```

```
            <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M15 19l-7-7 7-7" />
          </svg>
          Назад к списку
        </button>

        <div className="flex items-start justify-between">
          <div>
            <div className="flex items-center space-x-3 mb-2">
              <span className={`px-2 py-1 rounded text-sm
${statusColors[ticket.status]}`}>
                {statusNames[ticket.status]}
              </span>
              <span className="text-sm text-gray-500">Тикет
#{ticket.id}</span>
            </div>
            <h1 className="text-2xl font-bold text-gray-
800">{ticket.document_title}</h1>
            <p className="text-gray-600 font-
mono">{ticket.document_code}</p>
          </div>

          {canApprove && (
            <div className="flex space-x-2">
              <button
                onClick={() => setShowActionModal('approve')}
                className="btn bg-green-600 text-white hover:bg-green-
700"
              >
                Согласовать
              </button>
              <button
                onClick={() => setShowActionModal('request_changes')}
                className="btn bg-yellow-600 text-white hover:bg-yellow-
700"
              >
                На доработку
              </button>
              <button
                onClick={() => setShowActionModal('reject')}
                className="btn bg-red-600 text-white hover:bg-red-700"
              >
                Отклонить
              </button>
            </div>
          )}
        </div>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
        {/* Информация о документе */}
        <div className="lg:col-span-1">
          <div className="card mb-6">
```

```
            <h3 className="text-lg font-semibold mb-4">Информация о
документе</h3>
            <dl className="space-y-3">
              <div>
                <dt className="text-sm text-gray-500">Тип документа</dt>
                <dd className="font-
medium">{typeNames[ticket.document_type] || ticket.document_type}</dd>
              </div>
              <div>
                <dt className="text-sm text-gray-500">Формат</dt>
                <dd className="font-medium
uppercase">{getFileExtension()}</dd>
              </div>
              {ticket.file_size && (
                <div>
                  <dt className="text-sm text-gray-500">Размер</dt>
                  <dd className="font-
medium">{formatFileSize(ticket.file_size)}</dd>
                </div>
              )}
              <div>
                <dt className="text-sm text-gray-500">Автор заявки</dt>
                <dd className="font-medium">{ticket.created_by_name ||
ticket.creator_name}</dd>
              </div>
              <div>
                <dt className="text-sm text-gray-500">Создан</dt>
                <dd className="font-
medium">{formatDate(ticket.created_at)}</dd>
              </div>
              <div>
                <dt className="text-sm text-gray-500">Обновлен</dt>
                <dd className="font-
medium">{formatDate(ticket.updated_at)}</dd>
              </div>
            </dl>

            {ticket.document_description && (
              <div className="mt-4 pt-4 border-t">
                <dt className="text-sm text-gray-500 mb-1">Описание</dt>
                <dd className="text-gray-
700">{ticket.document_description}</dd>
              </div>
            )}

            <div className="mt-4 pt-4 border-t">
              <button
                onClick={handleDownload}
                className="btn btn-primary w-full"
              >
                Скачать документ
              </button>
            </div>
          </div>
```

```
        </div>

        {/* Предпросмотр и переписка */}
        <div className="lg:col-span-2 space-y-6">
          {/* Предпросмотр документа */}
          <div className="card">
            <h3 className="text-lg font-semibold mb-4">Предпросмотр
документа</h3>
            {renderPreview()}
          </div>

          {/* Переписка */}
          <div className="card">
            <h3 className="text-lg font-semibold mb-4">
              Обсуждение
              <span className="text-gray-400 font-normal ml-
2">({messages.length})</span>
            </h3>

            {/* Список сообщений */}
            <div className="space-y-4 mb-6 max-h-[500px] overflow-y-
auto">
              {messages.length === 0 ? (
                <p className="text-gray-500 text-center py-8">Сообщений
пока нет</p>
              ) : (
                messages.map((msg) => {
                  const msgType = msg.message_type || msg.type ||
'message';
                  const typeLabel = getMessageTypeLabel(msgType);
                  const cardStyle = getMessageCardStyle(msgType);

                  return (
                    <div key={msg.id} className={`p-4 rounded-lg
${cardStyle}`}>
                      {/* Шапка сообщения */}
                      <div className="flex items-center justify-between
mb-2">
                        <div className="flex items-center space-x-3">
                          {/* ФИО автора */}
                          <span className="font-bold text-gray-900">
                            {msg.author_name || msg.full_name ||
'Неизвестный пользователь'}
                          </span>

                          {/* Пометка типа сообщения */}
                          {typeLabel && (
                            <span className={`px-2 py-0.5 rounded text-xs
font-medium ${typeLabel.className}`}>
                              {typeLabel.text}
                            </span>
                          )}
                        </div>
```

```
                    {/* Дата и время */}
                    <span className="text-sm text-gray-500">
                      {formatDate(msg.created_at)}
                    </span>
                  </div>

                  {/* Текст сообщения */}
                  <p className="text-gray-700 whitespace-pre-wrap">
                    {msg.message || msg.content}
                  </p>
                </div>
              );
            })
          )}
        </div>

        {/* Форма отправки сообщения */}
        {['open', 'in_progress'].includes(ticket.status) && (
          <form onSubmit={handleSendMessage} className="border-t pt-
4">
            <div className="flex space-x-2">
              <input
                type="text"
                value={newMessage}
                onChange={(e) => setNewMessage(e.target.value)}
                placeholder="Написать сообщение..."
                className="input flex-1"
              />
              <button type="submit" className="btn btn-primary">
                Отправить
              </button>
            </div>
          </form>
        )}

        {/* Статус закрыт */}
        {['resolved', 'closed'].includes(ticket.status) && (
          <div className="border-t pt-4 text-center text-gray-500">
            Тикет закрыт. Новые сообщения недоступны.
          </div>
        )}
      </div>
    </div>
  </div>

  {/* Модальное окно действия */}
  {showActionModal && (
    <div className="fixed inset-0 bg-black bg-opacity-50 flex items-
center justify-center z-50">
      <div className="bg-white rounded-lg p-6 max-w-md w-full mx-4">
        <h3 className="text-lg font-semibold mb-4">
          {showActionModal === 'approve' && 'Согласование документа'}
          {showActionModal === 'reject' && 'Отклонение документа'}
```

```
              {showActionModal === 'request_changes' && 'Запрос на
доработку'}
            </h3>

            <div className="mb-4">
              <label className="block text-sm font-medium text-gray-700
mb-1">
                Комментарий {showActionModal !== 'approve' && <span
className="text-red-500">*</span>}
              </label>
              <textarea
                value={actionComment}
                onChange={(e) => setActionComment(e.target.value)}
                className="input"
                rows={4}
                placeholder={
                  showActionModal === 'approve'
                    ? 'Необязательный комментарий...'
                    : showActionModal === 'request_changes'
                    ? 'Опишите, что нужно исправить...'
                    : 'Укажите причину отклонения...'
                }
                required={showActionModal !== 'approve'}
              />
            </div>

            <div className="flex space-x-2">
              <button
                onClick={() => handleAction(showActionModal)}
                disabled={processing || (showActionModal !== 'approve' &&
!actionComment.trim())}
                className={`btn flex-1 text-white ${
                  showActionModal === 'approve'
                    ? 'bg-green-600 hover:bg-green-700'
                    : showActionModal === 'reject'
                    ? 'bg-red-600 hover:bg-red-700'
                    : 'bg-yellow-600 hover:bg-yellow-700'
                }`}
              >
                {processing ? 'Обработка...' : 'Подтвердить'}
              </button>
              <button
                onClick={() => {
                  setShowActionModal(null);
                  setActionComment('');
                }}
                className="btn btn-secondary flex-1"
              >
                Отмена
              </button>
            </div>
          </div>
        </div>
      )}
```

```
    </Layout>
  );
}
```

client/src/pages/TicketsPage.jsx

```
import { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { ticketsApi } from '../services/api';
import { useAuth } from '../contexts/AuthContext';
import Layout from '../components/Layout';

export default function TicketsPage() {
  const { user } = useAuth();
  const navigate = useNavigate();
  const [tickets, setTickets] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [filter, setFilter] = useState('all');

  useEffect(() => {
    loadTickets();
  }, []);

  const loadTickets = async () => {
    setLoading(true);
    try {
      const response = await ticketsApi.getAll();
      setTickets(response.data);
    } catch (err) {
      setError('Ошибка загрузки тикетов');
    }
    setLoading(false);
  };

  const formatDate = (dateString) => {
    return new Date(dateString).toLocaleString('ru-RU', {
      day: '2-digit',
      month: '2-digit',
      year: 'numeric',
      hour: '2-digit',
      minute: '2-digit',
    });
  };

  const statusNames = {
    open: 'Открыт',
    in_progress: 'В работе',
    resolved: 'Согласован',
    closed: 'Закрыт',
  };

  const statusColors = {
    open: 'bg-blue-100 text-blue-800',
```

```
    in_progress: 'bg-yellow-100 text-yellow-800',
    resolved: 'bg-green-100 text-green-800',
    closed: 'bg-gray-100 text-gray-800',
  };

  const typeNames = {
    drawing: 'Чертеж',
    standard: 'Стандарт',
    specification: 'Спецификация',
    instruction: 'Инструкция',
    manual: 'Руководство',
    other: 'Другое',
  };

  const filteredTickets = tickets.filter(ticket => {
    if (filter === 'all') return true;
    return ticket.status === filter;
  });

  const counts = {
    all: tickets.length,
    open: tickets.filter(t => t.status === 'open').length,
    in_progress: tickets.filter(t => t.status === 'in_progress').length,
    resolved: tickets.filter(t => t.status === 'resolved').length,
    closed: tickets.filter(t => t.status === 'closed').length,
  };

  const handleTicketClick = (ticketId) => {
    navigate(`/tickets/${ticketId}`);
  };

  if (loading) {
    return (
      <Layout>
        <div className="text-center py-8">Загрузка...</div>
      </Layout>
    );
  }

  return (
    <Layout>
      <div className="mb-6">
        <h1 className="text-2xl font-bold text-gray-800">
          {user?.role === 'librarian' ? 'Мои документы на согласовании' :
'Документы на согласование'}
        </h1>
        <p className="text-gray-600 mt-1">
          {user?.role === 'librarian'
            ? 'Отслеживайте статус своих документов'
            : 'Согласуйте или отклоните документы'
          }
        </p>
      </div>
```

```
      {error && (
        <div className="mb-4 p-4 bg-red-100 border border-red-400 text-
red-700 rounded">
          {error}
        </div>
      )}

      {/* Фильтры */}
      <div className="flex flex-wrap gap-2 mb-6">
        {[
          { id: 'all', label: 'Все' },
          { id: 'open', label: 'Открытые' },
          { id: 'in_progress', label: 'В работе' },
          { id: 'resolved', label: 'Согласованы' },
          { id: 'closed', label: 'Закрыты' },
        ].map((item) => (
          <button
            key={item.id}
            onClick={() => setFilter(item.id)}
            className={`px-4 py-2 rounded-lg font-medium ${
              filter === item.id
                ? 'bg-blue-600 text-white'
                : 'bg-gray-200 text-gray-700 hover:bg-gray-300'
            }`}
          >
            {item.label} ({counts[item.id]})
          </button>
        ))}
      </div>

      {/* Список тикетов */}
      {filteredTickets.length === 0 ? (
        <div className="card text-center py-12 text-gray-500">
          {filter === 'all' ? 'Нет документов на согласовании' : 'Нет
документов с таким статусом'}
        </div>
      ) : (
        <div className="space-y-4">
          {filteredTickets.map((ticket) => (
            <div
              key={ticket.id}
              onClick={() => handleTicketClick(ticket.id)}
              className="card hover:shadow-md transition-shadow cursor-
pointer"
            >
              <div className="flex items-start justify-between">
                <div className="flex-1">
                  <div className="flex items-center space-x-3 mb-2">
                    <span className={`px-2 py-1 rounded text-sm
${statusColors[ticket.status]}`}>
                      {statusNames[ticket.status]}
                    </span>
                    <span className="text-sm text-gray-500">
                      #{ticket.id}
```

```
                  </span>
                </div>

                <h3 className="text-lg font-semibold text-gray-800 mb-
1">
                  {ticket.document_title}
                </h3>

                <div className="flex flex-wrap items-center gap-x-4
gap-y-1 text-sm text-gray-600">
                  <span className="font-
mono">{ticket.document_code}</span>
                  <span>{typeNames[ticket.document_type]}</span>
                  <span>От: {ticket.created_by_name}</span>
                </div>

                {ticket.document_description && (
                  <p className="text-sm text-gray-500 mt-2 line-clamp-
2">
                    {ticket.document_description}
                  </p>
                )}
              </div>

              <div className="text-right text-sm text-gray-500 ml-4">
                <div>Создан: {formatDate(ticket.created_at)}</div>
                <div>Обновлен: {formatDate(ticket.updated_at)}</div>
              </div>
            </div>
          </div>
        ))}
      </div>
    )}
  </Layout>
);
}
```

client/src/pages/UploadPage.jsx

```
import { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { documentsApi, categoriesApi, usersApi } from '../services/api';
import Layout from '../components/Layout';

export default function UploadPage() {
  const navigate = useNavigate();

  const [formData, setFormData] = useState({
    title: '',
    code: '',
    type: 'drawing',
    category_id: '',
    description: '',
    tags: [],
```

```
  approver_ids: [],
});
const [file, setFile] = useState(null);

const [categories, setCategories] = useState([]);
const [tags, setTags] = useState([]);
const [approvers, setApprovers] = useState([]);

const [loading, setLoading] = useState(false);
const [error, setError] = useState('');

useEffect(() => {
  loadData();
}, []);

const loadData = async () => {
  try {
    const [categoriesRes, tagsRes, approversRes] = await Promise.all([
      categoriesApi.getAll(),
      categoriesApi.getAllTags(),
      usersApi.getApprovers(),
    ]);
    setCategories(categoriesRes.data);
    setTags(tagsRes.data);
    setApprovers(approversRes.data);
  } catch (err) {
    console.error('Ошибка загрузки данных:', err);
  }
};

const handleChange = (e) => {
  const { name, value } = e.target;
  setFormData({ ...formData, [name]: value });
};

const handleTagToggle = (tagId) => {
  const newTags = formData.tags.includes(tagId)
    ? formData.tags.filter((id) => id !== tagId)
    : [...formData.tags, tagId];
  setFormData({ ...formData, tags: newTags });
};

const handleApproverToggle = (approverId) => {
  const newApprovers = formData.approver_ids.includes(approverId)
    ? formData.approver_ids.filter((id) => id !== approverId)
    : [...formData.approver_ids, approverId];
  setFormData({ ...formData, approver_ids: newApprovers });
};

const handleFileChange = (e) => {
  if (e.target.files[0]) {
    setFile(e.target.files[0]);
  }
};
```

```
const handleSubmit = async (e) => {
  e.preventDefault();
  setError('');

  if (!formData.title || !formData.code || !formData.type) {
    setError('Заполните обязательные поля: Название, Код и Тип');
    return;
  }

  if (formData.approver_ids.length === 0) {
    setError('Выберите хотя бы одного руководителя для согласования');
    return;
  }

  if (!file) {
    setError('Выберите файл для загрузки');
    return;
  }

  setLoading(true);

  try {
    const data = new FormData();
    data.append('title', formData.title);
    data.append('code', formData.code);
    data.append('type', formData.type);
    data.append('category_id', formData.category_id);
    data.append('description', formData.description);
    data.append('tags', JSON.stringify(formData.tags));
    data.append('approver_ids', JSON.stringify(formData.approver_ids));
    data.append('file', file);

    await documentsApi.create(data);

    alert('Документ загружен и отправлен на согласование');
    navigate('/tickets');
  } catch (err) {
    setError(err.response?.data?.error || 'Ошибка загрузки документа');
  }

  setLoading(false);
};

const typeOptions = [
  { value: 'drawing', label: 'Чертеж' },
  { value: 'standard', label: 'Стандарт' },
  { value: 'specification', label: 'Спецификация' },
  { value: 'instruction', label: 'Инструкция' },
  { value: 'manual', label: 'Руководство' },
  { value: 'other', label: 'Другое' },
];

return (
```

```
    <Layout>
      <div className="max-w-3xl mx-auto">
        <h1 className="text-2xl font-bold text-gray-800 mb-6">
          Загрузка нового документа
        </h1>

        {error && (
          <div className="mb-4 p-4 bg-red-100 border border-red-400 text-
red-700 rounded">
            {error}
          </div>
        )}

        <form onSubmit={handleSubmit} className="card">
          {/* Основная информация */}
          <h2 className="text-lg font-semibold mb-4">Основная
информация</h2>

          <div className="grid grid-cols-1 md:grid-cols-2 gap-4 mb-6">
            <div>
              <label className="block text-sm font-medium text-gray-700
mb-1">
                Название документа *
              </label>
              <input
                type="text"
                name="title"
                value={formData.title}
                onChange={handleChange}
                className="input"
                placeholder="Введите название"
                required
              />
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700
mb-1">
                Код документа *
              </label>
              <input
                type="text"
                name="code"
                value={formData.code}
                onChange={handleChange}
                className="input"
                placeholder="Например: DOC-001"
                required
              />
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700
mb-1">
```

```
                Тип документа *
              </label>
              <select
                name="type"
                value={formData.type}
                onChange={handleChange}
                className="input"
                required
              >
                {typeOptions.map((opt) => (
                  <option key={opt.value} value={opt.value}>
                    {opt.label}
                  </option>
                ))}
              </select>
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700
mb-1">
                Категория
              </label>
              <select
                name="category_id"
                value={formData.category_id}
                onChange={handleChange}
                className="input"
              >
                <option value="">Без категории</option>
                {categories.map((cat) => (
                  <option key={cat.id} value={cat.id}>
                    {cat.name}
                  </option>
                ))}
              </select>
            </div>
          </div>

          {/* Описание */}
          <div className="mb-6">
            <label className="block text-sm font-medium text-gray-700 mb-
1">
              Описание документа
            </label>
            <textarea
              name="description"
              value={formData.description}
              onChange={handleChange}
              className="input"
              rows={4}
              placeholder="Введите описание документа..."
            />
          </div>
```

```
          {/* Теги */}
          {tags.length > 0 && (
            <div className="mb-6">
              <label className="block text-sm font-medium text-gray-700
mb-2">
                Теги
              </label>
              <div className="flex flex-wrap gap-2">
                {tags.map((tag) => (
                  <button
                    key={tag.id}
                    type="button"
                    onClick={() => handleTagToggle(tag.id)}
                    className={`px-3 py-1.5 rounded-full text-sm ${
                      formData.tags.includes(tag.id)
                        ? 'bg-blue-600 text-white'
                        : 'bg-gray-200 text-gray-700 hover:bg-gray-300'
                    }`}
                  >
                    {tag.name}
                  </button>
                ))}
              </div>
            </div>
          )}

          {/* Файл */}
          <div className="mb-6">
            <label className="block text-sm font-medium text-gray-700 mb-
1">
              Файл документа *
            </label>
            <input
              type="file"
              onChange={handleFileChange}
              className="input"
              accept=".pdf,.doc,.docx,.xls,.xlsx,.dwg,.dxf"
              required
            />
            {file && (
              <div className="mt-2 text-sm text-gray-600">
                Выбран файл: {file.name} ({(file.size / 1024 /
1024).toFixed(2)} МБ)
              </div>
            )}
          </div>

          {/* Согласующие руководители */}
          <div className="mb-6">
            <label className="block text-sm font-medium text-gray-700 mb-
2">
              Согласующие руководители *
            </label>
            <p className="text-sm text-gray-500 mb-3">
```

```
              Выберите руководителей, которые должны согласовать документ
            </p>

            {approvers.length === 0 ? (
              <div className="p-4 bg-yellow-50 border border-yellow-200
rounded text-yellow-800">
                Нет доступных руководителей. Обратитесь к администратору.
              </div>
            ) : (
              <div className="space-y-2">
                {approvers.map((approver) => (
                  <label
                    key={approver.id}
                    className={`flex items-center p-3 rounded-lg border
cursor-pointer ${
                      formData.approver_ids.includes(approver.id)
                        ? 'border-blue-500 bg-blue-50'
                        : 'border-gray-200 hover:border-gray-300'
                    }`}
                  >
                    <input
                      type="checkbox"

checked={formData.approver_ids.includes(approver.id)}
                      onChange={() => handleApproverToggle(approver.id)}
                      className="h-4 w-4 text-blue-600 rounded"
                    />
                    <div className="ml-3">
                      <div className="font-medium text-gray-800">
                        {approver.full_name}
                      </div>
                      {approver.position && (
                        <div className="text-sm text-gray-500">
                          {approver.position}
                        </div>
                      )}
                    </div>
                  </label>
                ))}
              </div>
            )}
          </div>

          {/* Кнопки */}
          <div className="flex space-x-4">
            <button
              type="submit"
              disabled={loading}
              className="btn btn-primary"
            >
              {loading ? 'Загрузка...' : 'Отправить на согласование'}
            </button>
            <button
              type="button"
```

```
              onClick={() => navigate(-1)}
              className="btn btn-secondary"
            >
              Отмена
            </button>
          </div>
        </form>
      </div>
    </Layout>
  );
}
```

client/src/services/api.js

```
import axios from 'axios';

const API_URL = import.meta.env.VITE_API_URL ||
'http://localhost:5000/api';

// Функция для получения базового URL (без /api)
export const getApiUrl = () => API_URL.replace('/api', '');

const api = axios.create({
  baseURL: API_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Добавляем токен к каждому запросу
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Обработка ошибок авторизации
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      localStorage.removeItem('user');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

// Auth API
export const authApi = {
  login: (data) => api.post('/auth/login', data),
```

```
  register: (data) => api.post('/auth/register', data),
  getProfile: () => api.get('/auth/profile'),
  updateProfile: (data) => api.put('/auth/profile', data),
};

// Users API
export const usersApi = {
  getAll: () => api.get('/users'),
  getById: (id) => api.get(`/users/${id}`),
  create: (data) => api.post('/users', data),
  update: (id, data) => api.put(`/users/${id}`, data),
  delete: (id) => api.delete(`/users/${id}`),
  getAuditLogs: () => api.get('/users/audit'),
  getAnalytics: () => api.get('/users/analytics'),
  getApprovers: () => api.get('/users/approvers'),
};

// Documents API
export const documentsApi = {
  search: (params) => api.get('/documents/search', { params }),
  getAll: (params) => api.get('/documents', { params }),
  getById: (id) => api.get(`/documents/${id}`),
  create: (formData) => api.post('/documents', formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
  }),
  update: (id, formData) => api.put(`/documents/${id}`, formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
  }),
  delete: (id) => api.delete(`/documents/${id}`),
  download: (id) => api.get(`/documents/${id}/download`, { responseType:
'blob' }),
  getPreviewUrl: (id) => `${API_URL}/documents/${id}/preview`,
  getVersions: (id) => api.get(`/documents/${id}/versions`),
  downloadVersion: (documentId, versionId) =>
api.get(`/documents/${documentId}/versions/${versionId}/download`, {
    responseType: 'blob'
  }),
  getComments: (id) => api.get(`/documents/${id}/comments`),
  addComment: (id, content) => api.post(`/documents/${id}/comments`, {
content }),
  deleteComment: (id, commentId) =>
api.delete(`/documents/${id}/comments/${commentId}`),
  addToFavorites: (id) => api.post(`/documents/${id}/favorite`),
  removeFromFavorites: (id) => api.delete(`/documents/${id}/favorite`),
  getFavorites: () => api.get('/documents/favorites'),
  generateReport: () => api.get('/documents/report', { responseType:
'blob' }),
  approveDeletion: (id) => api.post(`/documents/${id}/approve-deletion`),
  rejectDeletion: (id) => api.post(`/documents/${id}/reject-deletion`),
};

// Categories API
export const categoriesApi = {
  getAll: () => api.get('/categories'),
```

```
  create: (data) => api.post('/categories', data),
  update: (id, data) => api.put(`/categories/${id}`, data),
  delete: (id) => api.delete(`/categories/${id}`),
  getAllTags: () => api.get('/categories/tags'),
  createTag: (data) => api.post('/categories/tags', data),
  deleteTag: (id) => api.delete(`/categories/tags/${id}`),
};

// Tickets API
export const ticketsApi = {
  getAll: () => api.get('/tickets'),
  getById: (id) => api.get(`/tickets/${id}`),
  addMessage: (id, message) => api.post(`/tickets/${id}/message`, {
message }),
  approve: (id, comment) => api.post(`/tickets/${id}/approve`, { comment
}),
  reject: (id, comment) => api.post(`/tickets/${id}/reject`, { comment
}),
  requestChanges: (id, comment) => api.post(`/tickets/${id}/request-
changes`, { comment }),
};

// Settings API
export const settingsApi = {
  getAll: () => api.get('/settings'),
  getPublic: () => api.get('/settings/public'),
  update: (key, value) => api.put('/settings', { key, value }),
};

// Backup API
export const backupApi = {
  getAll: () => api.get('/backup'),
  create: () => api.post('/backup'),
  restore: (filename) => api.post(`/backup/restore/${filename}`),
  download: (filename) => api.get(`/backup/download/${filename}`, {
responseType: 'blob' }),
  delete: (id) => api.delete(`/backup/${id}`),
};

export default api;
```

client/src/App.jsx

```
import { Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider, useAuth } from './contexts/AuthContext';
import { SettingsProvider } from './contexts/SettingsContext';

import LoginPage from './pages/LoginPage';
import HomePage from './pages/HomePage';
import SearchPage from './pages/SearchPage';
import FavoritesPage from './pages/FavoritesPage';
import DocumentPage from './pages/DocumentPage';
import UploadPage from './pages/UploadPage';
import TicketsPage from './pages/TicketsPage';
```

```
import TicketDetailPage from './pages/TicketDetailPage';
import CategoriesPage from './pages/CategoriesPage';
import ProfilePage from './pages/ProfilePage';
import AdminPage from './pages/AdminPage';

function PrivateRoute({ children, roles }) {
  const { user, loading } = useAuth();

  if (loading) {
    return <div className="min-h-screen flex items-center justify-
center">Загрузка...</div>;
  }

  if (!user) {
    return <Navigate to="/login" />;
  }

  if (roles && !roles.includes(user.role)) {
    return <Navigate to="/" />;
  }

  return children;
}

function AppRoutes() {
  const { user } = useAuth();

  return (
    <Routes>
      <Route path="/login" element={<LoginPage />} />

      <Route path="/" element={
        <PrivateRoute>
          {user?.role === 'admin' ? <Navigate to="/admin" /> : <HomePage
/>}
        </PrivateRoute>
      } />

      <Route path="/search" element={
        <PrivateRoute>
          <SearchPage />
        </PrivateRoute>
      } />

      <Route path="/favorites" element={
        <PrivateRoute>
          <FavoritesPage />
        </PrivateRoute>
      } />

      <Route path="/documents/:id" element={
        <PrivateRoute>
          <DocumentPage />
        </PrivateRoute>
```

```
      } />

      <Route path="/upload" element={
        <PrivateRoute roles={['librarian', 'admin']}>
          <UploadPage />
        </PrivateRoute>
      } />

      <Route path="/tickets" element={
        <PrivateRoute roles={['librarian', 'department_head', 'admin']}>
          <TicketsPage />
        </PrivateRoute>
      } />

      <Route path="/tickets/:id" element={
        <PrivateRoute roles={['librarian', 'department_head', 'admin']}>
          <TicketDetailPage />
        </PrivateRoute>
      } />

      <Route path="/categories" element={
        <PrivateRoute roles={['librarian', 'admin']}>
          <CategoriesPage />
        </PrivateRoute>
      } />

      <Route path="/profile" element={
        <PrivateRoute>
          <ProfilePage />
        </PrivateRoute>
      } />

      <Route path="/admin" element={
        <PrivateRoute roles={['admin']}>
          <AdminPage />
        </PrivateRoute>
      } />

      <Route path="*" element={<Navigate to="/" />} />
    </Routes>
  );
}

export default function App() {
  return (
    <AuthProvider>
      <SettingsProvider>
        <AppRoutes />
      </SettingsProvider>
    </AuthProvider>
  );
}
```

server/src/config/database.js

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
});

// Проверка подключения
pool.query('SELECT NOW()', (err, res) => {
  if (err) {
    console.error('Ошибка подключения к базе данных:', err);
  } else {
    console.log('Успешное подключение к базе данных');
  }
});

module.exports = pool;
```

server/src/controllers/authController.js

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const pool = require('../config/database');
const path = require('path');
const fs = require('fs');

const authController = {
  // Регистрация
  async register(req, res) {
    try {
      const { email, password, full_name, position } = req.body;

      if (!email || !password || !full_name) {
        return res.status(400).json({ error: 'Заполните все обязательные
поля' });
      }

      const existingUser = await pool.query(
        'SELECT id FROM users WHERE email = $1',
        [email.toLowerCase()]
      );

      if (existingUser.rows.length > 0) {
        return res.status(400).json({ error: 'Пользователь с таким email
уже существует' });
      }

      const salt = await bcrypt.genSalt(10);
      const passwordHash = await bcrypt.hash(password, salt);
```

```
      const result = await pool.query(
        `INSERT INTO users (email, password_hash, full_name, position,
role)
         VALUES ($1, $2, $3, $4, 'technical_specialist')
         RETURNING id, email, full_name, position, role, avatar_url`,
        [email.toLowerCase(), passwordHash, full_name, position || null]
      );

      const user = result.rows[0];

      const jwtSecret = process.env.JWT_SECRET || 'your-secret-key-
change-in-production';
      const token = jwt.sign({ userId: user.id }, jwtSecret, { expiresIn:
'7d' });

      res.status(201).json({
        token,
        user: {
          id: user.id,
          email: user.email,
          full_name: user.full_name,
          position: user.position,
          role: user.role,
          avatar_url: user.avatar_url,
        },
      });
    } catch (error) {
      console.error('Ошибка регистрации:', error);
      res.status(500).json({ error: 'Ошибка сервера при регистрации' });
    }
  },

  // Вход
  async login(req, res) {
    try {
      const { email, password } = req.body;

      if (!email || !password) {
        return res.status(400).json({ error: 'Введите email и пароль' });
      }

      const result = await pool.query(
        'SELECT * FROM users WHERE email = $1',
        [email.toLowerCase()]
      );

      if (result.rows.length === 0) {
        return res.status(401).json({ error: 'Неверный email или пароль'
});
      }

      const user = result.rows[0];
```

```
      const isValidPassword = await bcrypt.compare(password,
user.password_hash);

      if (!isValidPassword) {
        return res.status(401).json({ error: 'Неверный email или пароль'
});
      }

      const jwtSecret = process.env.JWT_SECRET || 'your-secret-key-
change-in-production';
      const token = jwt.sign({ userId: user.id }, jwtSecret, { expiresIn:
'7d' });

      try {
        await pool.query(
          'INSERT INTO audit_log (user_id, action, entity_type,
entity_id) VALUES ($1, $2, $3, $4)',
          [user.id, 'user_login', 'user', user.id]
        );
      } catch (auditError) {
        console.log('Audit log error (non-critical):',
auditError.message);
      }

      res.json({
        token,
        user: {
          id: user.id,
          email: user.email,
          full_name: user.full_name,
          position: user.position,
          role: user.role,
          avatar_url: user.avatar_url,
        },
      });
    } catch (error) {
      console.error('Ошибка входа:', error);
      res.status(500).json({ error: 'Ошибка сервера при входе' });
    }
  },

  // Получение профиля
  async getProfile(req, res) {
    try {
      const result = await pool.query(
        'SELECT id, email, full_name, position, role, avatar_url,
created_at FROM users WHERE id = $1',
        [req.user.id]
      );

      if (result.rows.length === 0) {
        return res.status(404).json({ error: 'Пользователь не найден' });
      }
```

```
      res.json(result.rows[0]);
    } catch (error) {
      console.error('Ошибка получения профиля:', error);
      res.status(500).json({ error: 'Ошибка сервера' });
    }
  },

  // Обновление профиля
  async updateProfile(req, res) {
    try {
      const { full_name, position, current_password, new_password } =
req.body;

      if (new_password) {
        if (!current_password) {
          return res.status(400).json({ error: 'Введите текущий пароль'
});
        }

        const userResult = await pool.query(
          'SELECT password_hash FROM users WHERE id = $1',
          [req.user.id]
        );

        const isValidPassword = await bcrypt.compare(
          current_password,
          userResult.rows[0].password_hash
        );

        if (!isValidPassword) {
          return res.status(400).json({ error: 'Неверный текущий пароль'
});
        }

        const salt = await bcrypt.genSalt(10);
        const newPasswordHash = await bcrypt.hash(new_password, salt);

        await pool.query(
          'UPDATE users SET password_hash = $1, updated_at =
CURRENT_TIMESTAMP WHERE id = $2',
          [newPasswordHash, req.user.id]
        );
      }

      const result = await pool.query(
        `UPDATE users
         SET full_name = COALESCE($1, full_name),
             position = COALESCE($2, position),
             updated_at = CURRENT_TIMESTAMP
         WHERE id = $3
         RETURNING id, email, full_name, position, role, avatar_url`,
        [full_name, position, req.user.id]
      );
```

```
      res.json(result.rows[0]);
    } catch (error) {
      console.error('Ошибка обновления профиля:', error);
      res.status(500).json({ error: 'Ошибка сервера' });
    }
  },

  // Загрузка аватара
  async uploadAvatar(req, res) {
    try {
      if (!req.file) {
        return res.status(400).json({ error: 'Файл не загружен' });
      }

      const avatarUrl = `/uploads/avatars/${req.file.filename}`;

      // Удаляем старый аватар
      const oldUser = await pool.query('SELECT avatar_url FROM users
WHERE id = $1', [req.user.id]);
      if (oldUser.rows[0]?.avatar_url) {
        const oldPath = path.join(__dirname, '../../',
oldUser.rows[0].avatar_url);
        if (fs.existsSync(oldPath)) {
          fs.unlinkSync(oldPath);
        }
      }

      const result = await pool.query(
        `UPDATE users SET avatar_url = $1, updated_at = CURRENT_TIMESTAMP
         WHERE id = $2
         RETURNING id, email, full_name, position, role, avatar_url`,
        [avatarUrl, req.user.id]
      );

      res.json(result.rows[0]);
    } catch (error) {
      console.error('Ошибка загрузки аватара:', error);
      res.status(500).json({ error: 'Ошибка загрузки аватара' });
    }
  },

  // Удаление аватара
  async deleteAvatar(req, res) {
    try {
      const oldUser = await pool.query('SELECT avatar_url FROM users
WHERE id = $1', [req.user.id]);
      if (oldUser.rows[0]?.avatar_url) {
        const oldPath = path.join(__dirname, '../../',
oldUser.rows[0].avatar_url);
        if (fs.existsSync(oldPath)) {
          fs.unlinkSync(oldPath);
        }
      }
```

```
      const result = await pool.query(
        `UPDATE users SET avatar_url = NULL, updated_at =
CURRENT_TIMESTAMP
         WHERE id = $1
         RETURNING id, email, full_name, position, role, avatar_url`,
        [req.user.id]
      );

      res.json(result.rows[0]);
    } catch (error) {
      console.error('Ошибка удаления аватара:', error);
      res.status(500).json({ error: 'Ошибка удаления аватара' });
    }
  },
};

module.exports = authController;
```

server/src/controllers/backupController.js

```
const pool = require('../config/database');
const path = require('path');
const fs = require('fs');

const backupController = {
  // Получить список резервных копий
  async getAll(req, res) {
    try {
      const result = await pool.query(`
        SELECT b.*, u.full_name as created_by_name
        FROM backups b
        LEFT JOIN users u ON b.created_by = u.id
        ORDER BY b.created_at DESC
      `);
      res.json(result.rows);
    } catch (error) {
      console.error('Ошибка получения списка резервных копий:', error);
      res.status(500).json({ error: 'Ошибка получения списка' });
    }
  },

  // Создать резервную копию
  async create(req, res) {
    try {
      const timestamp = new Date().toISOString().replace(/[:.]/g, '-');
      const filename = `backup_${timestamp}.json`;
      const backupDir = path.join(__dirname, '../../backups');
      const filePath = path.join(backupDir, filename);

      if (!fs.existsSync(backupDir)) {
        fs.mkdirSync(backupDir, { recursive: true });
      }

      // Экспортируем данные из всех таблиц
```

```
      const tables = ['categories', 'tags', 'documents', 'document_tags',
                      'document_versions', 'comments', 'favorites',
                      'approval_tickets', 'ticket_messages',
'system_settings'];

      const backupData = {
        created_at: new Date().toISOString(),
        version: '1.0',
        tables: {}
      };

      for (const table of tables) {
        try {
          const result = await pool.query(`SELECT * FROM ${table}`);
          backupData.tables[table] = result.rows;
        } catch (err) {
          console.log(`Таблица ${table} не найдена, пропускаем`);
        }
      }

      fs.writeFileSync(filePath, JSON.stringify(backupData, null, 2),
'utf-8');
      const stats = fs.statSync(filePath);

      await pool.query(`
        INSERT INTO backups (filename, size_bytes, created_by)
        VALUES ($1, $2, $3)
      `, [filename, stats.size, req.user.id]);

      res.json({ message: 'Резервная копия создана', filename });
    } catch (error) {
      console.error('Ошибка создания резервной копии:', error);
      res.status(500).json({ error: 'Ошибка создания резервной копии: ' +
error.message });
    }
  },

  // Восстановить из резервной копии
  async restore(req, res) {
    try {
      const { filename } = req.params;

      const safeName = path.basename(filename);
      const filePath = path.join(__dirname, '../../backups', safeName);

      if (!fs.existsSync(filePath)) {
        return res.status(404).json({ error: 'Файл резервной копии не
найден' });
      }

      const backupContent = fs.readFileSync(filePath, 'utf-8');
      const backupData = JSON.parse(backupContent);

      if (!backupData.tables) {
```

```
        return res.status(400).json({ error: 'Неверный формат резервной
копии' });
      }

      // Начинаем транзакцию
      const client = await pool.connect();

      try {
        await client.query('BEGIN');

        // Порядок важен из-за foreign keys - сначала удаляем зависимые
        const deleteOrder = [
          'ticket_messages', 'approval_tickets', 'favorites', 'comments',
          'document_versions', 'document_tags', 'documents', 'tags',
'categories'
        ];

        for (const table of deleteOrder) {
          try {
            await client.query(`DELETE FROM ${table}`);
          } catch (err) {
            console.log(`Не удалось очистить ${table}:`, err.message);
          }
        }

        // Восстанавливаем в обратном порядке
        const restoreOrder = [
          'categories', 'tags', 'documents', 'document_tags',
          'document_versions', 'comments', 'favorites',
          'approval_tickets', 'ticket_messages', 'system_settings'
        ];

        for (const table of restoreOrder) {
          const rows = backupData.tables[table];
          if (!rows || rows.length === 0) continue;

          for (const row of rows) {
            const columns = Object.keys(row);
            const values = Object.values(row);
            const placeholders = columns.map((_, i) => `$${i +
1}`).join(', ');

            try {
              await client.query(
                `INSERT INTO ${table} (${columns.join(', ')}) VALUES
(${placeholders}) ON CONFLICT DO NOTHING`,
                values
              );
            } catch (err) {
              console.log(`Ошибка вставки в ${table}:`, err.message);
            }
          }

          // Сбрасываем sequence для serial полей
```

```
          try {
            await client.query(`
              SELECT setval(pg_get_serial_sequence('${table}', 'id'),
                COALESCE((SELECT MAX(id) FROM ${table}), 1))
            `);
          } catch (err) {
            // Игнорируем если нет sequence
          }
        }

        await client.query('COMMIT');

        res.json({ message: 'Данные успешно восстановлены из резервной
копии' });
      } catch (err) {
        await client.query('ROLLBACK');
        throw err;
      } finally {
        client.release();
      }
    } catch (error) {
      console.error('Ошибка восстановления:', error);
      res.status(500).json({ error: 'Ошибка восстановления: ' +
error.message });
    }
  },

  // Скачать резервную копию
  async download(req, res) {
    try {
      const { filename } = req.params;
      const safeName = path.basename(filename);
      const filePath = path.join(__dirname, '../../backups', safeName);

      if (!fs.existsSync(filePath)) {
        return res.status(404).json({ error: 'Файл не найден' });
      }

      res.download(filePath, safeName);
    } catch (error) {
      console.error('Ошибка скачивания:', error);
      res.status(500).json({ error: 'Ошибка скачивания' });
    }
  },

  // Удалить резервную копию
  async delete(req, res) {
    try {
      const { id } = req.params;

      const result = await pool.query('SELECT filename FROM backups WHERE
id = $1', [id]);

      if (result.rows.length === 0) {
```

```
        return res.status(404).json({ error: 'Резервная копия не найдена'
});
      }

      const filePath = path.join(__dirname, '../../backups',
result.rows[0].filename);

      if (fs.existsSync(filePath)) {
        fs.unlinkSync(filePath);
      }

      await pool.query('DELETE FROM backups WHERE id = $1', [id]);

      res.json({ message: 'Резервная копия удалена' });
    } catch (error) {
      console.error('Ошибка удаления:', error);
      res.status(500).json({ error: 'Ошибка удаления' });
    }
  }
};

module.exports = backupController;
```

server/src/controllers/categoryController.js

```
const pool = require('../config/database');

const categoryController = {
  // Получение всех категорий
  async getAll(req, res) {
    try {
      const result = await pool.query(`
        SELECT c.*,
               p.name as parent_name,
               COUNT(d.id) as document_count
        FROM categories c
        LEFT JOIN categories p ON c.parent_id = p.id
        LEFT JOIN documents d ON d.category_id = c.id
        GROUP BY c.id, p.name
        ORDER BY c.name
      `);

      res.json(result.rows);
    } catch (error) {
      console.error('Ошибка получения категорий:', error);
      res.status(500).json({ error: 'Ошибка получения категорий' });
    }
  },

  // Создание категории
  async create(req, res) {
    try {
      const { name, parent_id, description } = req.body;
```

```
    const result = await pool.query(`
      INSERT INTO categories (name, parent_id, description)
      VALUES ($1, $2, $3)
      RETURNING *
    `, [name, parent_id, description]);

    res.status(201).json(result.rows[0]);
  } catch (error) {
    console.error('Ошибка создания категории:', error);
    res.status(500).json({ error: 'Ошибка создания категории' });
  }
},

// Обновление категории
async update(req, res) {
  try {
    const { id } = req.params;
    const { name, parent_id, description } = req.body;

    const result = await pool.query(`
      UPDATE categories
      SET name = COALESCE($1, name),
          parent_id = $2,
          description = COALESCE($3, description)
      WHERE id = $4
      RETURNING *
    `, [name, parent_id, description, id]);

    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Категория не найдена' });
    }

    res.json(result.rows[0]);
  } catch (error) {
    console.error('Ошибка обновления категории:', error);
    res.status(500).json({ error: 'Ошибка обновления категории' });
  }
},

// Удаление категории
async delete(req, res) {
  try {
    const { id } = req.params;

    // Проверяем, есть ли документы в категории
    const docsCheck = await pool.query(
      'SELECT COUNT(*) FROM documents WHERE category_id = $1',
      [id]
    );

    if (parseInt(docsCheck.rows[0].count) > 0) {
      return res.status(400).json({
        error: 'Невозможно удалить категорию с документами'
      });
```

```
      }

      await pool.query('DELETE FROM categories WHERE id = $1', [id]);

      res.json({ message: 'Категория удалена' });
    } catch (error) {
      console.error('Ошибка удаления категории:', error);
      res.status(500).json({ error: 'Ошибка удаления категории' });
    }
  },

  // Получение всех тегов
  async getAllTags(req, res) {
    try {
      const result = await pool.query(`
        SELECT t.*,
               COUNT(dt.document_id) as document_count
        FROM tags t
        LEFT JOIN document_tags dt ON t.id = dt.tag_id
        GROUP BY t.id
        ORDER BY t.name
      `);

      res.json(result.rows);
    } catch (error) {
      console.error('Ошибка получения тегов:', error);
      res.status(500).json({ error: 'Ошибка получения тегов' });
    }
  },

  // Создание тега
  async createTag(req, res) {
    try {
      const { name } = req.body;

      if (!name || !name.trim()) {
        return res.status(400).json({ error: 'Название тега обязательно'
});
      }

      const result = await pool.query(`
        INSERT INTO tags (name)
        VALUES ($1)
        RETURNING *
      `, [name.trim()]);

      res.status(201).json(result.rows[0]);
    } catch (error) {
      if (error.code === '23505') {
        return res.status(400).json({ error: 'Тег с таким названием уже
существует' });
      }
      console.error('Ошибка создания тега:', error);
      res.status(500).json({ error: 'Ошибка создания тега' });
```

```
    }
  },

  // Обновление тега
  async updateTag(req, res) {
    try {
      const { id } = req.params;
      const { name } = req.body;

      if (!name || !name.trim()) {
        return res.status(400).json({ error: 'Название тега обязательно'
});
      }

      const result = await pool.query(`
        UPDATE tags
        SET name = $1
        WHERE id = $2
        RETURNING *
      `, [name.trim(), id]);

      if (result.rows.length === 0) {
        return res.status(404).json({ error: 'Тег не найден' });
      }

      res.json(result.rows[0]);
    } catch (error) {
      if (error.code === '23505') {
        return res.status(400).json({ error: 'Тег с таким названием уже
существует' });
      }
      console.error('Ошибка обновления тега:', error);
      res.status(500).json({ error: 'Ошибка обновления тега' });
    }
  },

  // Удаление тега
  async deleteTag(req, res) {
    try {
      const { id } = req.params;

      // Удаляем связи с документами
      await pool.query('DELETE FROM document_tags WHERE tag_id = $1',
[id]);

      // Удаляем сам тег
      await pool.query('DELETE FROM tags WHERE id = $1', [id]);

      res.json({ message: 'Тег удален' });
    } catch (error) {
      console.error('Ошибка удаления тега:', error);
      res.status(500).json({ error: 'Ошибка удаления тега' });
    }
  }
```

```
};

module.exports = categoryController;
```

server/src/controllers/documentController.js

```
const pool = require('../config/database');
const path = require('path');
const fs = require('fs');

const auditService = {
  async log(userId, action, entityType, entityId) {
    try {
      await pool.query(
        'INSERT INTO audit_log (user_id, action, entity_type, entity_id)
VALUES ($1, $2, $3, $4)',
        [userId, action, entityType, entityId]
      );
    } catch (error) {
      console.error('Ошибка записи в аудит:', error);
    }
  }
};

const documentController = {
  // Поиск документов
  async search(req, res) {
    try {
      const { query, category_id, type, status, tag_id } = req.query;

      let sql = `
        SELECT DISTINCT d.*,
               c.name as category_name,
               u.full_name as author_name,
               array_agg(DISTINCT t.name) FILTER (WHERE t.name IS NOT
NULL) as tag_names
        FROM documents d
        LEFT JOIN categories c ON d.category_id = c.id
        LEFT JOIN users u ON d.author_id = u.id
        LEFT JOIN document_tags dt ON d.id = dt.document_id
        LEFT JOIN tags t ON dt.tag_id = t.id
        WHERE 1=1
      `;

      const params = [];
      let paramIndex = 1;

      // Фильтр по роли (убрали draft)
      if (req.user.role === 'technical_specialist') {
        sql += ` AND d.status = 'in_library'`;
      } else if (req.user.role === 'librarian') {
        sql += ` AND d.status IN ('in_library', 'pending_approval')`;
      } else if (req.user.role === 'department_head') {
```

```
        sql += ` AND d.status IN ('in_library', 'pending_approval',
'pending_deletion')`;
      }

      if (query) {
        sql += ` AND (d.title ILIKE $${paramIndex} OR d.code ILIKE
$${paramIndex} OR d.description ILIKE $${paramIndex})`;
        params.push(`%${query}%`);
        paramIndex++;
      }

      if (category_id) {
        sql += ` AND d.category_id = $${paramIndex++}`;
        params.push(category_id);
      }

      if (type) {
        sql += ` AND d.type = $${paramIndex++}`;
        params.push(type);
      }

      if (status) {
        sql += ` AND d.status = $${paramIndex++}`;
        params.push(status);
      }

      if (tag_id) {
        sql += ` AND dt.tag_id = $${paramIndex++}`;
        params.push(tag_id);
      }

      sql += ` GROUP BY d.id, c.name, u.full_name`;
      sql += ` ORDER BY d.updated_at DESC`;

      const result = await pool.query(sql, params);

      res.json({ documents: result.rows });
    } catch (error) {
      console.error('Ошибка поиска документов:', error);
      res.status(500).json({ error: 'Ошибка поиска документов' });
    }
  },

  // Получение документа по ID
  async getById(req, res) {
    try {
      const { id } = req.params;

      const result = await pool.query(`
        SELECT d.*,
               c.name as category_name,
               u.full_name as author_name
        FROM documents d
        LEFT JOIN categories c ON d.category_id = c.id
```

```
        LEFT JOIN users u ON d.author_id = u.id
        WHERE d.id = $1
      `, [id]);

      if (result.rows.length === 0) {
        return res.status(404).json({ error: 'Документ не найден' });
      }

      const document = result.rows[0];

      if (req.user.role === 'technical_specialist' && document.status !==
'in_library') {
        return res.status(403).json({ error: 'Нет доступа к этому
документу' });
      }

      const tagsResult = await pool.query(`
        SELECT t.* FROM tags t
        JOIN document_tags dt ON t.id = dt.tag_id
        WHERE dt.document_id = $1
      `, [id]);

      document.tags = tagsResult.rows;

      await auditService.log(req.user.id, 'document_view', 'document',
id);

      res.json(document);
    } catch (error) {
      console.error('Ошибка получения документа:', error);
      res.status(500).json({ error: 'Ошибка получения документа' });
    }
  },

  // Просмотр файла (для отображения в браузере)
  async preview(req, res) {
    try {
      const { id } = req.params;

      const result = await pool.query('SELECT * FROM documents WHERE id =
$1', [id]);

      if (result.rows.length === 0) {
        return res.status(404).json({ error: 'Документ не найден' });
      }

      const document = result.rows[0];

      if (!document.file_path) {
        return res.status(404).json({ error: 'У документа нет файла' });
      }

      const uploadsDir = path.join(__dirname, '../../uploads');
      const filePath = path.join(uploadsDir, document.file_path);
```

```
      if (!fs.existsSync(filePath)) {
        return res.status(404).json({ error: 'Файл не найден на сервере'
});
      }

      const ext = path.extname(document.file_path).toLowerCase();

      // Определяем Content-Type
      const mimeTypes = {
        '.pdf': 'application/pdf',
        '.doc': 'application/msword',
        '.docx': 'application/vnd.openxmlformats-
officedocument.wordprocessingml.document',
        '.xls': 'application/vnd.ms-excel',
        '.xlsx': 'application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet',
        '.png': 'image/png',
        '.jpg': 'image/jpeg',
        '.jpeg': 'image/jpeg',
        '.gif': 'image/gif',
      };

      const contentType = mimeTypes[ext] || 'application/octet-stream';

      res.setHeader('Content-Type', contentType);
      res.setHeader('Content-Disposition', 'inline');

      const fileStream = fs.createReadStream(filePath);
      fileStream.pipe(res);
    } catch (error) {
      console.error('Ошибка просмотра файла:', error);
      res.status(500).json({ error: 'Ошибка просмотра файла' });
    }
  },

  // Создание документа (сразу на согласование)
  async create(req, res) {
    try {
      const { title, code, type, category_id, description, tags,
approver_ids } = req.body;
      const file = req.file;

      if (!title || !code || !type) {
        return res.status(400).json({ error: 'Заполните обязательные
поля' });
      }

      if (!approver_ids) {
        return res.status(400).json({ error: 'Выберите согласующих
руководителей' });
      }
```

```
      const codeCheck = await pool.query('SELECT id FROM documents WHERE
code = $1', [code]);
      if (codeCheck.rows.length > 0) {
        return res.status(400).json({ error: 'Документ с таким кодом уже
существует' });
      }

      const filePath = file ? file.filename : null;
      const fileSize = file ? file.size : null;

      // Создаем документ сразу со статусом pending_approval
      const result = await pool.query(`
        INSERT INTO documents (title, code, type, category_id,
description, file_path, file_size, author_id, status)
        VALUES ($1, $2, $3, $4, $5, $6, $7, $8, 'pending_approval')
        RETURNING *
      `, [title, code, type, category_id || null, description || null,
filePath, fileSize, req.user.id]);

      const documentId = result.rows[0].id;

      // Парсим список согласующих
      const approvers = typeof approver_ids === 'string' ?
JSON.parse(approver_ids) : approver_ids;

      // Создаем тикет на согласование (один этап)
      const ticketResult = await pool.query(`
        INSERT INTO approval_tickets (document_id, created_by, stage,
total_stages, status)
        VALUES ($1, $2, 1, 1, 'open')
        RETURNING id
      `, [documentId, req.user.id]);

      const ticketId = ticketResult.rows[0].id;

      // Записываем согласующих в отдельную таблицу
      for (const approverId of approvers) {
        await pool.query(`
          INSERT INTO ticket_approvers (ticket_id, user_id, status)
          VALUES ($1, $2, 'pending')
          ON CONFLICT DO NOTHING
        `, [ticketId, approverId]);
      }

      // Добавляем теги
      if (tags) {
        const tagIds = typeof tags === 'string' ? JSON.parse(tags) :
tags;
        if (Array.isArray(tagIds)) {
          for (const tagId of tagIds) {
            await pool.query(
              'INSERT INTO document_tags (document_id, tag_id) VALUES
($1, $2) ON CONFLICT DO NOTHING',
              [documentId, tagId]
```

```
            );
          }
        }
      }

      await auditService.log(req.user.id, 'document_create', 'document',
documentId);

      res.status(201).json(result.rows[0]);
    } catch (error) {
      console.error('Ошибка создания документа:', error);
      res.status(500).json({ error: 'Ошибка создания документа: ' +
error.message });
    }
  },

  // Обновление документа (версия увеличивается только для in_library)
  async update(req, res) {
    try {
      const { id } = req.params;
      const { title, code, type, category_id, description, tags } =
req.body;
      const file = req.file;

      const docCheck = await pool.query('SELECT * FROM documents WHERE id
= $1', [id]);
      if (docCheck.rows.length === 0) {
        return res.status(404).json({ error: 'Документ не найден' });
      }

      const currentDoc = docCheck.rows[0];
      const isInLibrary = currentDoc.status === 'in_library';

      let updateFields = [];
      let params = [];
      let paramIndex = 1;

      if (title) {
        updateFields.push(`title = $${paramIndex++}`);
        params.push(title);
      }
      if (code) {
        updateFields.push(`code = $${paramIndex++}`);
        params.push(code);
      }
      if (type) {
        updateFields.push(`type = $${paramIndex++}`);
        params.push(type);
      }
      updateFields.push(`category_id = $${paramIndex++}`);
      params.push(category_id || null);

      if (description !== undefined) {
        updateFields.push(`description = $${paramIndex++}`);
```

```
        params.push(description);
      }

      if (file) {
        updateFields.push(`file_path = $${paramIndex++}`);
        params.push(file.filename);
        updateFields.push(`file_size = $${paramIndex++}`);
        params.push(file.size);

        // Увеличиваем версию только если документ уже в библиотеке
        if (isInLibrary) {
          updateFields.push(`current_version = current_version + 1`);
        }
      }

      updateFields.push(`updated_at = CURRENT_TIMESTAMP`);
      params.push(id);

      const result = await pool.query(`
        UPDATE documents SET ${updateFields.join(', ')}
        WHERE id = $${paramIndex}
        RETURNING *
      `, params);

      // Обновляем теги
      if (tags !== undefined) {
        await pool.query('DELETE FROM document_tags WHERE document_id =
$1', [id]);
        const tagIds = typeof tags === 'string' ? JSON.parse(tags) :
tags;
        if (Array.isArray(tagIds)) {
          for (const tagId of tagIds) {
            await pool.query(
              'INSERT INTO document_tags (document_id, tag_id) VALUES
($1, $2) ON CONFLICT DO NOTHING',
              [id, tagId]
            );
          }
        }
      }

      // Создаем версию только если документ уже в библиотеке и загружен
новый файл
      if (file && isInLibrary) {
        await pool.query(`
          INSERT INTO document_versions (document_id, version_number,
file_path, file_size, created_by, comment)
          VALUES ($1, $2, $3, $4, $5, 'Обновление документа')
        `, [id, result.rows[0].current_version, file.filename, file.size,
req.user.id]);
      }

      await auditService.log(req.user.id, 'document_update', 'document',
id);
```

```
      res.json(result.rows[0]);
    } catch (error) {
      console.error('Ошибка обновления документа:', error);
      res.status(500).json({ error: 'Ошибка обновления документа' });
    }
  },

  // Скачивание файла
  async download(req, res) {
    try {
      const { id } = req.params;

      const result = await pool.query('SELECT * FROM documents WHERE id =
$1', [id]);

      if (result.rows.length === 0) {
        return res.status(404).json({ error: 'Документ не найден' });
      }

      const document = result.rows[0];

      if (!document.file_path) {
        return res.status(404).json({ error: 'У документа нет файла' });
      }

      const uploadsDir = path.join(__dirname, '../../uploads');
      const filePath = path.join(uploadsDir, document.file_path);

      if (!fs.existsSync(filePath)) {
        return res.status(404).json({ error: 'Файл не найден на сервере'
});
      }

      await auditService.log(req.user.id, 'document_download',
'document', id);

      const ext = path.extname(document.file_path);
      const downloadName = `${document.code}${ext}`;

      res.download(filePath, downloadName);
    } catch (error) {
      console.error('Ошибка скачивания файла:', error);
      res.status(500).json({ error: 'Ошибка скачивания файла' });
    }
  },

  // Удаление документа
  async delete(req, res) {
    try {
      const { id } = req.params;

      const docCheck = await pool.query('SELECT * FROM documents WHERE id
= $1', [id]);
```

```
      if (docCheck.rows.length === 0) {
        return res.status(404).json({ error: 'Документ не найден' });
      }

      const document = docCheck.rows[0];

      if (document.status === 'in_library') {
        await pool.query(
          `UPDATE documents SET status = 'pending_deletion', updated_at =
CURRENT_TIMESTAMP WHERE id = $1`,
          [id]
        );
        await auditService.log(req.user.id, 'document_delete_request',
'document', id);
        return res.json({ message: 'Запрос на удаление отправлен
руководителю' });
      }

      // Удаляем связанные записи
      await pool.query('DELETE FROM ticket_approvers WHERE ticket_id IN
(SELECT id FROM approval_tickets WHERE document_id = $1)', [id]);
      await pool.query('DELETE FROM ticket_messages WHERE ticket_id IN
(SELECT id FROM approval_tickets WHERE document_id = $1)', [id]);
      await pool.query('DELETE FROM document_versions WHERE document_id =
$1', [id]);
      await pool.query('DELETE FROM document_tags WHERE document_id =
$1', [id]);
      await pool.query('DELETE FROM comments WHERE document_id = $1',
[id]);
      await pool.query('DELETE FROM favorites WHERE document_id = $1',
[id]);
      await pool.query('DELETE FROM approval_tickets WHERE document_id =
$1', [id]);
      await pool.query('DELETE FROM documents WHERE id = $1', [id]);

      await auditService.log(req.user.id, 'document_delete', 'document',
id);

      res.json({ message: 'Документ удален' });
    } catch (error) {
      console.error('Ошибка удаления документа:', error);
      res.status(500).json({ error: 'Ошибка удаления документа' });
    }
  },

  // Получение заметок
  async getComments(req, res) {
    try {
      const { id } = req.params;

      const result = await pool.query(`
        SELECT c.*, u.full_name as author_name
        FROM comments c
```

```
        JOIN users u ON c.user_id = u.id
        WHERE c.document_id = $1 AND c.user_id = $2
        ORDER BY c.created_at DESC
      `, [id, req.user.id]);

      res.json(result.rows);
    } catch (error) {
      console.error('Ошибка получения заметок:', error);
      res.status(500).json({ error: 'Ошибка получения заметок' });
    }
  },

  // Добавление заметки
  async addComment(req, res) {
    try {
      const { id } = req.params;
      const { content } = req.body;

      if (!content || !content.trim()) {
        return res.status(400).json({ error: 'Введите текст заметки' });
      }

      const result = await pool.query(`
        INSERT INTO comments (document_id, user_id, content)
        VALUES ($1, $2, $3)
        RETURNING *
      `, [id, req.user.id, content.trim()]);

      res.status(201).json(result.rows[0]);
    } catch (error) {
      console.error('Ошибка добавления заметки:', error);
      res.status(500).json({ error: 'Ошибка добавления заметки' });
    }
  },

  // Удаление заметки
  async deleteComment(req, res) {
    try {
      const { id, commentId } = req.params;

      const check = await pool.query(
        'SELECT id FROM comments WHERE id = $1 AND user_id = $2',
        [commentId, req.user.id]
      );

      if (check.rows.length === 0) {
        return res.status(403).json({ error: 'Нет доступа к этой заметке'
});
      }

      await pool.query('DELETE FROM comments WHERE id = $1',
[commentId]);

      res.json({ message: 'Заметка удалена' });
```

```
  } catch (error) {
    console.error('Ошибка удаления заметки:', error);
    res.status(500).json({ error: 'Ошибка удаления заметки' });
  }
},

// Избранное
async addToFavorites(req, res) {
  try {
    const { id } = req.params;

    await pool.query(`
      INSERT INTO favorites (user_id, document_id)
      VALUES ($1, $2)
      ON CONFLICT DO NOTHING
    `, [req.user.id, id]);

    res.json({ message: 'Добавлено в избранное' });
  } catch (error) {
    res.status(500).json({ error: 'Ошибка добавления в избранное' });
  }
},

async removeFromFavorites(req, res) {
  try {
    const { id } = req.params;

    await pool.query(
      'DELETE FROM favorites WHERE user_id = $1 AND document_id = $2',
      [req.user.id, id]
    );

    res.json({ message: 'Удалено из избранного' });
  } catch (error) {
    res.status(500).json({ error: 'Ошибка удаления из избранного' });
  }
},

async getFavorites(req, res) {
  try {
    const result = await pool.query(`
      SELECT d.*, c.name as category_name
      FROM favorites f
      JOIN documents d ON f.document_id = d.id
      LEFT JOIN categories c ON d.category_id = c.id
      WHERE f.user_id = $1 AND d.status = 'in_library'
      ORDER BY f.created_at DESC
    `, [req.user.id]);

    res.json(result.rows);
  } catch (error) {
    res.status(500).json({ error: 'Ошибка получения избранного' });
  }
},
```

```
  // Версии
  async getVersions(req, res) {
    try {
      const { id } = req.params;

      const result = await pool.query(`
        SELECT v.*, u.full_name as created_by_name
        FROM document_versions v
        LEFT JOIN users u ON v.created_by = u.id
        WHERE v.document_id = $1
        ORDER BY v.version_number DESC
      `, [id]);

      res.json(result.rows);
    } catch (error) {
      res.status(500).json({ error: 'Ошибка получения версий' });
    }
  },

  // Отчет
  async generateReport(req, res) {
    try {
      const result = await pool.query(`
        SELECT
          d.code, d.title, d.type, d.status,
          c.name as category_name,
          u.full_name as author_name,
          d.current_version, d.created_at, d.updated_at
        FROM documents d
        LEFT JOIN categories c ON d.category_id = c.id
        LEFT JOIN users u ON d.author_id = u.id
        ORDER BY d.created_at DESC
      `);

      const typeNames = {
        drawing: 'Чертеж', standard: 'Стандарт', specification:
'Спецификация',
        instruction: 'Инструкция', manual: 'Руководство', other: 'Другое'
      };

      const statusNames = {
        pending_approval: 'На согласовании', in_library: 'В библиотеке',
        archived: 'В архиве', withdrawn: 'Отозван', pending_deletion: 'На
удаление'
      };

      let csv =
'Код;Название;Тип;Статус;Категория;Автор;Версия;Создан;Обновлен\n';

      result.rows.forEach(row => {
        const createdAt = new
Date(row.created_at).toLocaleDateString('ru-RU');
```

```
        const updatedAt = new
Date(row.updated_at).toLocaleDateString('ru-RU');

        csv += `${row.code};${row.title};${typeNames[row.type] ||
row.type};${statusNames[row.status] || row.status};${row.category_name ||
'-'};${row.author_name || '-
'};${row.current_version};${createdAt};${updatedAt}\n`;
      });

      res.setHeader('Content-Type', 'text/csv; charset=utf-8');
      res.setHeader('Content-Disposition', 'attachment;
filename=report.csv');
      res.send('\uFEFF' + csv);
    } catch (error) {
      res.status(500).json({ error: 'Ошибка формирования отчета' });
    }
  },

  // Согласование удаления
  async approveDeletion(req, res) {
    try {
      const { id } = req.params;

      const docCheck = await pool.query('SELECT file_path FROM documents
WHERE id = $1', [id]);

      if (docCheck.rows.length === 0) {
        return res.status(404).json({ error: 'Документ не найден' });
      }

      if (docCheck.rows[0].file_path) {
        const filePath = path.join(__dirname, '../../uploads',
docCheck.rows[0].file_path);
        if (fs.existsSync(filePath)) {
          fs.unlinkSync(filePath);
        }
      }

      await pool.query('DELETE FROM ticket_approvers WHERE ticket_id IN
(SELECT id FROM approval_tickets WHERE document_id = $1)', [id]);
      await pool.query('DELETE FROM ticket_messages WHERE ticket_id IN
(SELECT id FROM approval_tickets WHERE document_id = $1)', [id]);
      await pool.query('DELETE FROM document_versions WHERE document_id =
$1', [id]);
      await pool.query('DELETE FROM document_tags WHERE document_id =
$1', [id]);
      await pool.query('DELETE FROM comments WHERE document_id = $1',
[id]);
      await pool.query('DELETE FROM favorites WHERE document_id = $1',
[id]);
      await pool.query('DELETE FROM approval_tickets WHERE document_id =
$1', [id]);
      await pool.query('DELETE FROM documents WHERE id = $1', [id]);
```

```
      await auditService.log(req.user.id, 'document_delete_approved',
'document', id);

      res.json({ message: 'Документ удален' });
    } catch (error) {
      res.status(500).json({ error: 'Ошибка удаления документа' });
    }
  },

  async rejectDeletion(req, res) {
    try {
      const { id } = req.params;

      await pool.query(
        `UPDATE documents SET status = 'in_library', updated_at =
CURRENT_TIMESTAMP WHERE id = $1`,
        [id]
      );

      await auditService.log(req.user.id, 'document_delete_rejected',
'document', id);

      res.json({ message: 'Запрос на удаление отменен' });
    } catch (error) {
      res.status(500).json({ error: 'Ошибка' });
    }
  }
};

module.exports = documentController;
```

server/src/controllers/settingsController.js

```
const pool = require('../config/database');

const settingsController = {
  // Получить все настройки (для админа)
  async getAll(req, res) {
    try {
      const result = await pool.query(`
        SELECT * FROM system_settings
        ORDER BY key
      `);
      res.json(result.rows);
    } catch (error) {
      console.error('Ошибка получения настроек:', error);
      res.status(500).json({ error: 'Ошибка получения настроек' });
    }
  },

  // Получить публичные настройки (для всех авторизованных)
  async getPublic(req, res) {
    try {
      const result = await pool.query(`
```

```
        SELECT key, value FROM system_settings
        WHERE key IN ('app_name', 'max_file_size', 'allowed_extensions')
      `);

      const settings = {};
      result.rows.forEach(row => {
        settings[row.key] = row.value;
      });

      res.json(settings);
    } catch (error) {
      console.error('Ошибка получения настроек:', error);
      res.status(500).json({ error: 'Ошибка получения настроек' });
    }
  },

  // Обновить настройку
  async update(req, res) {
    try {
      const { key, value } = req.body;

      if (!key) {
        return res.status(400).json({ error: 'Не указан ключ настройки'
});
      }

      await pool.query(`
        UPDATE system_settings
        SET value = $1, updated_at = CURRENT_TIMESTAMP
        WHERE key = $2
      `, [value, key]);

      res.json({ message: 'Настройка обновлена' });
    } catch (error) {
      console.error('Ошибка обновления настройки:', error);
      res.status(500).json({ error: 'Ошибка обновления настройки' });
    }
  }
};

module.exports = settingsController;
```

server/src/controllers/ticketController.js

```
const pool = require('../config/database');

// Получение всех тикетов
exports.getAll = async (req, res) => {
  try {
    const userId = req.user.id;
    const userRole = req.user.role;

    let query = `
      SELECT t.*,
```

```
              d.title as document_title,
              d.code as document_code,
              d.type as document_type,
              u.full_name as created_by_name
      FROM approval_tickets t
      JOIN documents d ON t.document_id = d.id
      JOIN users u ON t.created_by = u.id
    `;

    // Руководители и админы видят все тикеты, остальные - только свои
    if (userRole !== 'department_head' && userRole !== 'admin') {
      query += ` WHERE t.created_by = $1`;
    }

    query += ` ORDER BY t.created_at DESC`;

    const params = (userRole !== 'department_head' && userRole !==
'admin') ? [userId] : [];
    const result = await pool.query(query, params);

    res.json(result.rows);
  } catch (error) {
    console.error('Ошибка получения тикетов:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
};

// Получение тикета по ID с сообщениями
exports.getById = async (req, res) => {
  try {
    const { id } = req.params;

    // Получаем тикет
    const ticketResult = await pool.query(`
      SELECT t.*,
              d.title as document_title,
              d.code as document_code,
              d.type as document_type,
              d.description as document_description,
              d.file_path,
              d.file_name,
              d.file_size,
              creator.full_name as created_by_name
      FROM approval_tickets t
      JOIN documents d ON t.document_id = d.id
      JOIN users creator ON t.created_by = creator.id
      WHERE t.id = $1
    `, [id]);

    if (ticketResult.rows.length === 0) {
      return res.status(404).json({ error: 'Тикет не найден' });
    }

    // Получаем сообщения с ФИО авторов и типом сообщения
```

```
    const messagesResult = await pool.query(`
      SELECT
        tm.id,
        tm.ticket_id,
        tm.user_id,
        tm.message,
        tm.message_type,
        tm.created_at,
        u.full_name as author_name
      FROM ticket_messages tm
      JOIN users u ON tm.user_id = u.id
      WHERE tm.ticket_id = $1
      ORDER BY tm.created_at ASC
    `, [id]);

    res.json({
      ticket: ticketResult.rows[0],
      messages: messagesResult.rows
    });
  } catch (error) {
    console.error('Ошибка получения тикета:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
};

// Добавить сообщение
exports.addMessage = async (req, res) => {
  try {
    const { id } = req.params;
    const { message } = req.body;
    const userId = req.user.id;

    if (!message || !message.trim()) {
      return res.status(400).json({ error: 'Сообщение не может быть
пустым' });
    }

    // Добавляем сообщение с типом 'message'
    const result = await pool.query(
      `INSERT INTO ticket_messages (ticket_id, user_id, message,
message_type)
       VALUES ($1, $2, $3, 'message')
       RETURNING *`,
      [id, userId, message.trim()]
    );

    // Обновляем статус тикета на "в работе" если он был открыт
    await pool.query(
      `UPDATE approval_tickets
       SET status = CASE WHEN status = 'open' THEN 'in_progress' ELSE
status END,
           updated_at = CURRENT_TIMESTAMP
       WHERE id = $1`,
      [id]
```

```
    );

    res.status(201).json(result.rows[0]);
  } catch (error) {
    console.error('Ошибка добавления сообщения:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
};

// Согласовать
exports.approve = async (req, res) => {
  try {
    const { id } = req.params;
    const { comment } = req.body;
    const userId = req.user.id;

    // Проверяем права
    if (req.user.role !== 'department_head' && req.user.role !== 'admin')
{
      return res.status(403).json({ error: 'Недостаточно прав' });
    }

    // Получаем тикет
    const ticketResult = await pool.query('SELECT * FROM approval_tickets
WHERE id = $1', [id]);
    if (ticketResult.rows.length === 0) {
      return res.status(404).json({ error: 'Тикет не найден' });
    }

    const ticket = ticketResult.rows[0];

    // Обновляем статус тикета
    await pool.query(
      `UPDATE approval_tickets
       SET status = 'resolved', updated_at = CURRENT_TIMESTAMP
       WHERE id = $1`,
      [id]
    );

    // Обновляем статус документа
    await pool.query(
      `UPDATE documents SET status = 'in_library', updated_at =
CURRENT_TIMESTAMP WHERE id = $1`,
      [ticket.document_id]
    );

    // Добавляем сообщение с типом 'approve'
    const messageText = (comment && comment.trim()) ? comment.trim() :
'Документ согласован';
    await pool.query(
      `INSERT INTO ticket_messages (ticket_id, user_id, message,
message_type)
       VALUES ($1, $2, $3, 'approve')`,
      [id, userId, messageText]
```

```
    );

    res.json({ message: 'Документ согласован' });
  } catch (error) {
    console.error('Ошибка согласования:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
};

// Отклонить
exports.reject = async (req, res) => {
  try {
    const { id } = req.params;
    const { comment } = req.body;
    const userId = req.user.id;

    // Проверяем права
    if (req.user.role !== 'department_head' && req.user.role !== 'admin')
{
      return res.status(403).json({ error: 'Недостаточно прав' });
    }

    if (!comment || !comment.trim()) {
      return res.status(400).json({ error: 'Укажите причину отклонения'
});
    }

    // Получаем тикет
    const ticketResult = await pool.query('SELECT * FROM approval_tickets
WHERE id = $1', [id]);
    if (ticketResult.rows.length === 0) {
      return res.status(404).json({ error: 'Тикет не найден' });
    }

    const ticket = ticketResult.rows[0];

    // Обновляем статус тикета
    await pool.query(
      `UPDATE approval_tickets
       SET status = 'closed', updated_at = CURRENT_TIMESTAMP
       WHERE id = $1`,
      [id]
    );

    // Обновляем статус документа
    await pool.query(
      `UPDATE documents SET status = 'rejected', updated_at =
CURRENT_TIMESTAMP WHERE id = $1`,
      [ticket.document_id]
    );

    // Добавляем сообщение с типом 'reject'
    await pool.query(
```

```
      `INSERT INTO ticket_messages (ticket_id, user_id, message,
message_type)
       VALUES ($1, $2, $3, 'reject')`,
      [id, userId, comment.trim()]
    );

    res.json({ message: 'Документ отклонен' });
  } catch (error) {
    console.error('Ошибка отклонения:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
};

// Запрос на доработку
exports.requestChanges = async (req, res) => {
  try {
    const { id } = req.params;
    const { comment } = req.body;
    const userId = req.user.id;

    // Проверяем права
    if (req.user.role !== 'department_head' && req.user.role !== 'admin')
{
      return res.status(403).json({ error: 'Недостаточно прав' });
    }

    if (!comment || !comment.trim()) {
      return res.status(400).json({ error: 'Укажите что нужно исправить'
});
    }

    // Обновляем статус тикета
    await pool.query(
      `UPDATE approval_tickets
       SET status = 'in_progress', updated_at = CURRENT_TIMESTAMP
       WHERE id = $1`,
      [id]
    );

    // Добавляем сообщение с типом 'request_changes'
    await pool.query(
      `INSERT INTO ticket_messages (ticket_id, user_id, message,
message_type)
       VALUES ($1, $2, $3, 'request_changes')`,
      [id, userId, comment.trim()]
    );

    res.json({ message: 'Запрос на доработку отправлен' });
  } catch (error) {
    console.error('Ошибка запроса доработки:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
};
```

server/src/controllers/userController.js

```
const pool = require('../config/database');
const bcrypt = require('bcryptjs');

const userController = {
  // Получить всех пользователей
  async getAll(req, res) {
    try {
      const result = await pool.query(
        'SELECT id, email, full_name, position, role, avatar_url,
created_at FROM users ORDER BY created_at DESC'
      );
      res.json(result.rows);
    } catch (error) {
      console.error('Ошибка получения пользователей:', error);
      res.status(500).json({ error: 'Ошибка сервера' });
    }
  },

  // Получить пользователя по ID
  async getById(req, res) {
    try {
      const { id } = req.params;
      const result = await pool.query(
        'SELECT id, email, full_name, position, role, avatar_url,
created_at FROM users WHERE id = $1',
        [id]
      );

      if (result.rows.length === 0) {
        return res.status(404).json({ error: 'Пользователь не найден' });
      }

      res.json(result.rows[0]);
    } catch (error) {
      console.error('Ошибка получения пользователя:', error);
      res.status(500).json({ error: 'Ошибка сервера' });
    }
  },

  // Создать пользователя
  async create(req, res) {
    try {
      const { email, password, full_name, position, role } = req.body;

      if (!email || !password || !full_name || !role) {
        return res.status(400).json({ error: 'Заполните все обязательные
поля' });
      }

      // Проверяем существование
      const existing = await pool.query('SELECT id FROM users WHERE email
= $1', [email.toLowerCase()]);
```

```
      if (existing.rows.length > 0) {
        return res.status(400).json({ error: 'Пользователь с таким email
уже существует' });
      }

      // Хешируем пароль
      const salt = await bcrypt.genSalt(10);
      const passwordHash = await bcrypt.hash(password, salt);

      const result = await pool.query(
        `INSERT INTO users (email, password_hash, full_name, position,
role)
         VALUES ($1, $2, $3, $4, $5)
         RETURNING id, email, full_name, position, role, created_at`,
        [email.toLowerCase(), passwordHash, full_name, position || null,
role]
      );

      res.status(201).json(result.rows[0]);
    } catch (error) {
      console.error('Ошибка создания пользователя:', error);
      res.status(500).json({ error: 'Ошибка сервера' });
    }
  },

  // Обновить пользователя
  async update(req, res) {
    try {
      const { id } = req.params;
      const { email, password, full_name, position, role } = req.body;

      let query = `
        UPDATE users
        SET email = COALESCE($1, email),
            full_name = COALESCE($2, full_name),
            position = COALESCE($3, position),
            role = COALESCE($4, role),
            updated_at = CURRENT_TIMESTAMP
      `;
      let params = [email?.toLowerCase(), full_name, position, role];

      // Если передан новый пароль
      if (password) {
        const salt = await bcrypt.genSalt(10);
        const passwordHash = await bcrypt.hash(password, salt);
        query += `, password_hash = $${params.length + 1}`;
        params.push(passwordHash);
      }

      query += ` WHERE id = $${params.length + 1} RETURNING id, email,
full_name, position, role`;
      params.push(id);

      const result = await pool.query(query, params);
```

```
      if (result.rows.length === 0) {
        return res.status(404).json({ error: 'Пользователь не найден' });
      }

      res.json(result.rows[0]);
    } catch (error) {
      console.error('Ошибка обновления пользователя:', error);
      res.status(500).json({ error: 'Ошибка сервера' });
    }
  },

  // Удалить пользователя
  async delete(req, res) {
    try {
      const { id } = req.params;

      // Нельзя удалить самого себя
      if (parseInt(id) === req.user.id) {
        return res.status(400).json({ error: 'Нельзя удалить собственный
аккаунт' });
      }

      const result = await pool.query('DELETE FROM users WHERE id = $1
RETURNING id', [id]);

      if (result.rows.length === 0) {
        return res.status(404).json({ error: 'Пользователь не найден' });
      }

      res.json({ message: 'Пользователь удален' });
    } catch (error) {
      console.error('Ошибка удаления пользователя:', error);
      res.status(500).json({ error: 'Ошибка сервера' });
    }
  },

  // Получить список согласующих (руководителей)
  async getApprovers(req, res) {
    try {
      const result = await pool.query(
        "SELECT id, full_name, position FROM users WHERE role =
'department_head' ORDER BY full_name"
      );
      res.json(result.rows);
    } catch (error) {
      console.error('Ошибка получения согласующих:', error);
      res.status(500).json({ error: 'Ошибка сервера' });
    }
  },

  // Получить журнал аудита
  async getAuditLogs(req, res) {
    try {
```

```
      const result = await pool.query(`
        SELECT
          al.id,
          al.action,
          al.entity_type,
          al.entity_id,
          al.created_at,
          u.full_name as user_name
        FROM audit_log al
        LEFT JOIN users u ON al.user_id = u.id
        ORDER BY al.created_at DESC
        LIMIT 500
      `);
      res.json(result.rows);
    } catch (error) {
      console.error('Ошибка получения аудита:', error);
      res.status(500).json({ error: 'Ошибка сервера' });
    }
  },

  // Получить расширенную аналитику
  async getAnalytics(req, res) {
    try {
      // Общая статистика документов
      let docsStats = { rows: [{ total_documents: 0,
documents_in_library: 0, documents_pending: 0, documents_archived: 0,
documents_withdrawn: 0 }] };
      try {
        docsStats = await pool.query(`
          SELECT
            COUNT(*) as total_documents,
            COUNT(*) FILTER (WHERE status = 'in_library') as
documents_in_library,
            COUNT(*) FILTER (WHERE status = 'pending_approval') as
documents_pending,
            COUNT(*) FILTER (WHERE status = 'archived') as
documents_archived,
            COUNT(*) FILTER (WHERE status = 'withdrawn') as
documents_withdrawn
          FROM documents
        `);
      } catch (e) {
        console.log('Таблица documents не найдена');
      }

      // Статистика пользователей
      let usersStats = { rows: [{ total_users: 0, admins: 0, librarians:
0, department_heads: 0, technical_specialists: 0 }] };
      try {
        usersStats = await pool.query(`
          SELECT
            COUNT(*) as total_users,
            COUNT(*) FILTER (WHERE role = 'admin') as admins,
            COUNT(*) FILTER (WHERE role = 'librarian') as librarians,
```

```
            COUNT(*) FILTER (WHERE role = 'department_head') as
department_heads,
            COUNT(*) FILTER (WHERE role = 'technical_specialist') as
technical_specialists
          FROM users
        `);
      } catch (e) {
        console.log('Таблица users не найдена');
      }

      // Статистика тикетов
      let ticketsStats = { rows: [{ total_tickets: 0, tickets_open: 0,
tickets_in_progress: 0, tickets_resolved: 0, tickets_closed: 0 }] };
      try {
        ticketsStats = await pool.query(`
          SELECT
            COUNT(*) as total_tickets,
            COUNT(*) FILTER (WHERE status = 'open') as tickets_open,
            COUNT(*) FILTER (WHERE status = 'in_progress') as
tickets_in_progress,
            COUNT(*) FILTER (WHERE status = 'resolved') as
tickets_resolved,
            COUNT(*) FILTER (WHERE status = 'closed') as tickets_closed
          FROM approval_tickets
        `);
      } catch (e) {
        console.log('Таблица approval_tickets не найдена');
      }

      // Документы по типам
      let docsByType = { rows: [] };
      try {
        docsByType = await pool.query(`
          SELECT type, COUNT(*) as count
          FROM documents
          GROUP BY type
          ORDER BY count DESC
        `);
      } catch (e) {
        console.log('Ошибка получения документов по типам');
      }

      // Документы по категориям
      let docsByCategory = { rows: [] };
      try {
        docsByCategory = await pool.query(`
          SELECT
            COALESCE(c.name, 'Без категории') as category,
            COUNT(*) as count
          FROM documents d
          LEFT JOIN categories c ON d.category_id = c.id
          GROUP BY c.name
          ORDER BY count DESC
          LIMIT 10
```

```
        `);
      } catch (e) {
        console.log('Ошибка получения документов по категориям');
      }

      // Топ пользователей по активности
      let topUsers = { rows: [] };
      try {
        topUsers = await pool.query(`
          SELECT
            u.full_name,
            COUNT(*) as actions
          FROM audit_log al
          JOIN users u ON al.user_id = u.id
          WHERE al.created_at >= CURRENT_DATE - INTERVAL '30 days'
          GROUP BY u.id, u.full_name
          ORDER BY actions DESC
          LIMIT 5
        `);
      } catch (e) {
        console.log('Ошибка получения топ пользователей');
      }

      // Общий размер файлов
      let storageStats = { rows: [{ total_size_bytes: 0, files_count: 0
}] };
      try {
        storageStats = await pool.query(`
          SELECT
            COALESCE(SUM(file_size), 0) as total_size_bytes,
            COUNT(*) FILTER (WHERE file_path IS NOT NULL) as files_count
          FROM documents
        `);
      } catch (e) {
        console.log('Ошибка получения статистики хранилища');
      }

      // Категории и теги
      let categoriesCount = { rows: [{ count: 0 }] };
      let tagsCount = { rows: [{ count: 0 }] };
      try {
        categoriesCount = await pool.query('SELECT COUNT(*) FROM
categories');
        tagsCount = await pool.query('SELECT COUNT(*) FROM tags');
      } catch (e) {
        console.log('Ошибка получения категорий/тегов');
      }

      // Действия за сегодня
      let actionsToday = { rows: [{ count: 0 }] };
      try {
        actionsToday = await pool.query(`
          SELECT COUNT(*) FROM audit_log
          WHERE created_at >= CURRENT_DATE
```

```
        `);
      } catch (e) {
        console.log('Ошибка получения действий за сегодня');
      }

      res.json({
        // Документы
        total_documents: parseInt(docsStats.rows[0]?.total_documents ||
0),
        documents_in_library:
parseInt(docsStats.rows[0]?.documents_in_library || 0),
        documents_pending: parseInt(docsStats.rows[0]?.documents_pending
|| 0),
        documents_archived:
parseInt(docsStats.rows[0]?.documents_archived || 0),
        documents_withdrawn:
parseInt(docsStats.rows[0]?.documents_withdrawn || 0),

        // Пользователи
        total_users: parseInt(usersStats.rows[0]?.total_users || 0),
        admins: parseInt(usersStats.rows[0]?.admins || 0),
        librarians: parseInt(usersStats.rows[0]?.librarians || 0),
        department_heads: parseInt(usersStats.rows[0]?.department_heads
|| 0),
        technical_specialists:
parseInt(usersStats.rows[0]?.technical_specialists || 0),

        // Тикеты
        total_tickets: parseInt(ticketsStats.rows[0]?.total_tickets ||
0),
        tickets_open: parseInt(ticketsStats.rows[0]?.tickets_open || 0),
        tickets_in_progress:
parseInt(ticketsStats.rows[0]?.tickets_in_progress || 0),
        tickets_resolved: parseInt(ticketsStats.rows[0]?.tickets_resolved
|| 0),
        tickets_closed: parseInt(ticketsStats.rows[0]?.tickets_closed ||
0),

        // Дополнительная статистика
        docs_by_type: docsByType.rows,
        docs_by_category: docsByCategory.rows,
        top_users: topUsers.rows,

        // Категории и теги
        total_categories: parseInt(categoriesCount.rows[0]?.count || 0),
        total_tags: parseInt(tagsCount.rows[0]?.count || 0),

        // Хранилище
        total_storage_bytes:
parseInt(storageStats.rows[0]?.total_size_bytes || 0),
        total_files: parseInt(storageStats.rows[0]?.files_count || 0),

        // Активность
        actions_today: parseInt(actionsToday.rows[0]?.count || 0),
```

```
      });
    } catch (error) {
      console.error('Ошибка получения аналитики:', error);
      res.status(500).json({ error: 'Ошибка сервера: ' + error.message
});
    }
  },
};

module.exports = userController;
```

server/src/middleware/auth.js

```
const jwt = require('jsonwebtoken');
const pool = require('../config/database');

const authMiddleware = async (req, res, next) => {
  try {
    // Получаем токен из заголовка или из query параметра (для preview)
    let token = req.headers.authorization?.replace('Bearer ', '');

    // Если токен не в заголовке, проверяем query (для iframe preview)
    if (!token && req.query.token) {
      token = req.query.token;
    }

    if (!token) {
      return res.status(401).json({ error: 'Требуется авторизация' });
    }

    const jwtSecret = process.env.JWT_SECRET || 'your-secret-key-change-
in-production';
    const decoded = jwt.verify(token, jwtSecret);

    // Получаем пользователя из БД
    const result = await pool.query(
      'SELECT id, email, full_name, role, position FROM users WHERE id =
$1',
      [decoded.userId]
    );

    if (result.rows.length === 0) {
      return res.status(401).json({ error: 'Пользователь не найден' });
    }

    req.user = result.rows[0];
    next();
  } catch (error) {
    console.error('Ошибка авторизации:', error.message);
    return res.status(401).json({ error: 'Недействительный токен' });
  }
};

module.exports = authMiddleware;
```

server/src/middleware/role.js

```
const roleMiddleware = (allowedRoles) => {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ error: 'Требуется авторизация' });
    }

    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Недостаточно прав для
выполнения этого действия' });
    }

    next();
  };
};

module.exports = roleMiddleware;
```

server/src/middleware/roleCheck.js

```
// Проверка роли пользователя
const roleCheck = (allowedRoles) => {
  return (req, res, next) => {
    // Проверяем, есть ли пользователь (должен быть установлен middleware
auth)
    if (!req.user) {
      return res.status(401).json({ error: 'Не авторизован' });
    }

    // Проверяем, входит ли роль пользователя в список разрешённых
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Недостаточно прав для
выполнения действия' });
    }

    next();
  };
};

module.exports = roleCheck;
```

server/src/routes/auth.js

```
const express = require('express');
const router = express.Router();
const multer = require('multer');
const path = require('path');
const fs = require('fs');
const authController = require('../controllers/authController');
const authMiddleware = require('../middleware/auth');

// Настройка multer для аватаров
```

```
const avatarStorage = multer.diskStorage({
  destination: (req, file, cb) => {
    const avatarDir = path.join(__dirname, '../../uploads/avatars');
    if (!fs.existsSync(avatarDir)) {
      fs.mkdirSync(avatarDir, { recursive: true });
    }
    cb(null, avatarDir);
  },
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    cb(null, `avatar_${req.user.id}_${Date.now()}${ext}`);
  },
});

const avatarUpload = multer({
  storage: avatarStorage,
  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB
  fileFilter: (req, file, cb) => {
    const allowedTypes = ['image/jpeg', 'image/png', 'image/gif',
'image/webp'];
    if (allowedTypes.includes(file.mimetype)) {
      cb(null, true);
    } else {
      cb(new Error('Разрешены только изображения (jpeg, png, gif,
webp)'));
    }
  },
});

// Публичные маршруты
router.post('/register', authController.register);
router.post('/login', authController.login);

// Защищенные маршруты
router.get('/profile', authMiddleware, authController.getProfile);
router.put('/profile', authMiddleware, authController.updateProfile);
router.post('/avatar', authMiddleware, avatarUpload.single('avatar'),
authController.uploadAvatar);
router.delete('/avatar', authMiddleware, authController.deleteAvatar);

module.exports = router;
```

server/src/routes/backup.js

```
const express = require('express');
const router = express.Router();
const backupController = require('../controllers/backupController');
const authMiddleware = require('../middleware/auth');
const roleMiddleware = require('../middleware/role');

// Все маршруты только для админа
router.use(authMiddleware);
router.use(roleMiddleware(['admin']));
```

```
router.get('/', backupController.getAll);
router.post('/', backupController.create);
router.post('/restore/:filename', backupController.restore);  // НОВЫЙ
маршрут
router.get('/download/:filename', backupController.download);
router.delete('/:id', backupController.delete);

module.exports = router;
```

server/src/routes/categories.js

```
const express = require('express');
const router = express.Router();
const categoryController = require('../controllers/categoryController');
const authMiddleware = require('../middleware/auth');
const roleCheck = require('../middleware/roleCheck');

// Все маршруты требуют авторизации
router.use(authMiddleware);

// GET /api/categories - Получение всех категорий (для всех)
router.get('/', categoryController.getAll);

// GET /api/categories/tags - Получение всех тегов (для всех)
router.get('/tags', categoryController.getAllTags);

// Остальные маршруты только для библиотекаря
// POST /api/categories - Создание категории
router.post('/',
  roleCheck(['librarian']),
  categoryController.create
);

// PUT /api/categories/:id - Обновление категории
router.put('/:id',
  roleCheck(['librarian']),
  categoryController.update
);

// DELETE /api/categories/:id - Удаление категории
router.delete('/:id',
  roleCheck(['librarian']),
  categoryController.delete
);

// POST /api/categories/tags - Создание тега
router.post('/tags',
  roleCheck(['librarian']),
  categoryController.createTag
);

// PUT /api/categories/tags/:id - Обновление тега
router.put('/tags/:id',
  roleCheck(['librarian']),
```

```
  categoryController.updateTag
);

// DELETE /api/categories/tags/:id - Удаление тега
router.delete('/tags/:id',
  roleCheck(['librarian']),
  categoryController.deleteTag
);

module.exports = router;
```

server/src/routes/documents.js

```
const express = require('express');
const router = express.Router();
const pool = require('../config/database');
const authMiddleware = require('../middleware/auth');
const multer = require('multer');
const path = require('path');
const fs = require('fs');

// Настройка загрузки файлов
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    const uploadDir = path.join(__dirname, '../../uploads');
    if (!fs.existsSync(uploadDir)) {
      fs.mkdirSync(uploadDir, { recursive: true });
    }
    cb(null, uploadDir);
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() *
1E9);
    cb(null, uniqueSuffix + path.extname(file.originalname));
  }
});

const upload = multer({
  storage,
  limits: { fileSize: 100 * 1024 * 1024 } // 100MB
});

// GET /api/documents - Получение списка документов
router.get('/', authMiddleware, async (req, res) => {
  try {
    const { search, type, category_id, status, page = 1, limit = 20 } =
req.query;
    const offset = (page - 1) * limit;

    let query = `
      SELECT d.*, c.name as category_name, u.full_name as created_by_name
      FROM documents d
      LEFT JOIN categories c ON d.category_id = c.id
      LEFT JOIN users u ON d.created_by = u.id
```

```
      WHERE 1=1
    `;
    const params = [];
    let paramIndex = 1;

    if (search) {
      query += ` AND (d.title ILIKE $${paramIndex} OR d.code ILIKE
$${paramIndex} OR d.description ILIKE $${paramIndex})`;
      params.push(`%${search}%`);
      paramIndex++;
    }

    if (type) {
      query += ` AND d.type = $${paramIndex}`;
      params.push(type);
      paramIndex++;
    }

    if (category_id) {
      query += ` AND d.category_id = $${paramIndex}`;
      params.push(category_id);
      paramIndex++;
    }

    if (status) {
      query += ` AND d.status = $${paramIndex}`;
      params.push(status);
      paramIndex++;
    }

    query += ` ORDER BY d.created_at DESC LIMIT $${paramIndex} OFFSET
$${paramIndex + 1}`;
    params.push(limit, offset);

    const result = await pool.query(query, params);

    // Получаем общее количество
    let countQuery = `SELECT COUNT(*) FROM documents d WHERE 1=1`;
    const countParams = [];
    let countParamIndex = 1;

    if (search) {
      countQuery += ` AND (d.title ILIKE $${countParamIndex} OR d.code
ILIKE $${countParamIndex})`;
      countParams.push(`%${search}%`);
      countParamIndex++;
    }
    if (type) {
      countQuery += ` AND d.type = $${countParamIndex}`;
      countParams.push(type);
      countParamIndex++;
    }
    if (category_id) {
      countQuery += ` AND d.category_id = $${countParamIndex}`;
```

```
      countParams.push(category_id);
      countParamIndex++;
    }
    if (status) {
      countQuery += ` AND d.status = $${countParamIndex}`;
      countParams.push(status);
    }

    const countResult = await pool.query(countQuery, countParams);

    res.json({
      documents: result.rows,
      total: parseInt(countResult.rows[0].count),
      page: parseInt(page),
      limit: parseInt(limit)
    });
  } catch (error) {
    console.error('Ошибка получения документов:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// GET /api/documents/favorites - Избранные документы
router.get('/favorites', authMiddleware, async (req, res) => {
  try {
    const userId = req.user.id;

    const result = await pool.query(`
      SELECT d.*, c.name as category_name
      FROM documents d
      JOIN favorites f ON d.id = f.document_id
      LEFT JOIN categories c ON d.category_id = c.id
      WHERE f.user_id = $1
      ORDER BY f.created_at DESC
    `, [userId]);

    res.json(result.rows);
  } catch (error) {
    console.error('Ошибка получения избранного:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// POST /api/documents/favorites/:id - Добавить в избранное
router.post('/favorites/:id', authMiddleware, async (req, res) => {
  try {
    const { id } = req.params;
    const userId = req.user.id;

    await pool.query(
      'INSERT INTO favorites (user_id, document_id) VALUES ($1, $2) ON
CONFLICT DO NOTHING',
      [userId, id]
    );
```

```
    res.json({ message: 'Добавлено в избранное' });
  } catch (error) {
    console.error('Ошибка добавления в избранное:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// DELETE /api/documents/favorites/:id - Удалить из избранного
router.delete('/favorites/:id', authMiddleware, async (req, res) => {
  try {
    const { id } = req.params;
    const userId = req.user.id;

    await pool.query(
      'DELETE FROM favorites WHERE user_id = $1 AND document_id = $2',
      [userId, id]
    );

    res.json({ message: 'Удалено из избранного' });
  } catch (error) {
    console.error('Ошибка удаления из избранного:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// GET /api/documents/:id - Получение документа по ID
router.get('/:id', authMiddleware, async (req, res) => {
  try {
    const { id } = req.params;

    const result = await pool.query(`
      SELECT d.*, c.name as category_name, u.full_name as created_by_name
      FROM documents d
      LEFT JOIN categories c ON d.category_id = c.id
      LEFT JOIN users u ON d.created_by = u.id
      WHERE d.id = $1
    `, [id]);

    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Документ не найден' });
    }

    // Получаем теги документа
    const tagsResult = await pool.query(`
      SELECT t.id, t.name
      FROM tags t
      JOIN document_tags dt ON t.id = dt.tag_id
      WHERE dt.document_id = $1
    `, [id]);

    const document = result.rows[0];
    document.tags = tagsResult.rows;
```

```
    res.json(document);
  } catch (error) {
    console.error('Ошибка получения документа:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// GET /api/documents/:id/preview - Предпросмотр документа
router.get('/:id/preview', async (req, res) => {
  try {
    // Проверяем токен из query параметра или заголовка
    const token = req.query.token ||
req.headers.authorization?.replace('Bearer ', '');

    if (!token) {
      return res.status(401).json({ error: 'Требуется авторизация' });
    }

    // Верифицируем токен
    const jwt = require('jsonwebtoken');
    try {
      jwt.verify(token, process.env.JWT_SECRET || 'your-secret-key');
    } catch (err) {
      return res.status(401).json({ error: 'Недействительный токен' });
    }

    const { id } = req.params;

    const result = await pool.query(
      'SELECT file_path, file_name FROM documents WHERE id = $1',
      [id]
    );

    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Документ не найден' });
    }

    const document = result.rows[0];
    const filePath = path.join(__dirname, '../../uploads',
document.file_path);

    if (!fs.existsSync(filePath)) {
      return res.status(404).json({ error: 'Файл не найден' });
    }

    // Расширенный список MIME типов
    const ext = path.extname(filePath).toLowerCase();
    const mimeTypes = {
      // Изображения
      '.jpg': 'image/jpeg',
      '.jpeg': 'image/jpeg',
      '.png': 'image/png',
      '.gif': 'image/gif',
      '.webp': 'image/webp',
```

```
      '.svg': 'image/svg+xml',
      '.bmp': 'image/bmp',
      '.ico': 'image/x-icon',
      '.tiff': 'image/tiff',
      '.tif': 'image/tiff',
      '.avif': 'image/avif',
      '.heic': 'image/heic',
      '.heif': 'image/heif',

      // Документы
      '.pdf': 'application/pdf',
      '.doc': 'application/msword',
      '.docx': 'application/vnd.openxmlformats-
officedocument.wordprocessingml.document',
      '.xls': 'application/vnd.ms-excel',
      '.xlsx': 'application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet',
      '.ppt': 'application/vnd.ms-powerpoint',
      '.pptx': 'application/vnd.openxmlformats-
officedocument.presentationml.presentation',
      '.odt': 'application/vnd.oasis.opendocument.text',
      '.ods': 'application/vnd.oasis.opendocument.spreadsheet',
      '.odp': 'application/vnd.oasis.opendocument.presentation',
      '.rtf': 'application/rtf',

      // Текстовые файлы
      '.txt': 'text/plain; charset=utf-8',
      '.md': 'text/plain; charset=utf-8',
      '.markdown': 'text/plain; charset=utf-8',
      '.log': 'text/plain; charset=utf-8',
      '.ini': 'text/plain; charset=utf-8',
      '.cfg': 'text/plain; charset=utf-8',
      '.conf': 'text/plain; charset=utf-8',
      '.yaml': 'text/plain; charset=utf-8',
      '.yml': 'text/plain; charset=utf-8',
      '.toml': 'text/plain; charset=utf-8',
      '.env': 'text/plain; charset=utf-8',

      // Данные
      '.json': 'application/json',
      '.xml': 'text/xml',
      '.csv': 'text/csv; charset=utf-8',
      '.tsv': 'text/tab-separated-values; charset=utf-8',

      // Код
      '.html': 'text/html; charset=utf-8',
      '.htm': 'text/html; charset=utf-8',
      '.css': 'text/css; charset=utf-8',
      '.js': 'text/javascript; charset=utf-8',
      '.jsx': 'text/javascript; charset=utf-8',
      '.ts': 'text/typescript; charset=utf-8',
      '.tsx': 'text/typescript; charset=utf-8',
      '.py': 'text/x-python; charset=utf-8',
      '.java': 'text/x-java; charset=utf-8',
```

```
'.c': 'text/x-c; charset=utf-8',
'.cpp': 'text/x-c++; charset=utf-8',
'.h': 'text/x-c; charset=utf-8',
'.hpp': 'text/x-c++; charset=utf-8',
'.cs': 'text/x-csharp; charset=utf-8',
'.go': 'text/x-go; charset=utf-8',
'.rs': 'text/x-rust; charset=utf-8',
'.rb': 'text/x-ruby; charset=utf-8',
'.php': 'text/x-php; charset=utf-8',
'.swift': 'text/x-swift; charset=utf-8',
'.kt': 'text/x-kotlin; charset=utf-8',
'.scala': 'text/x-scala; charset=utf-8',
'.sql': 'text/x-sql; charset=utf-8',
'.sh': 'text/x-shellscript; charset=utf-8',
'.bash': 'text/x-shellscript; charset=utf-8',
'.bat': 'text/x-batch; charset=utf-8',
'.ps1': 'text/x-powershell; charset=utf-8',
'.r': 'text/x-r; charset=utf-8',
'.lua': 'text/x-lua; charset=utf-8',
'.pl': 'text/x-perl; charset=utf-8',

// Видео
'.mp4': 'video/mp4',
'.webm': 'video/webm',
'.ogg': 'video/ogg',
'.ogv': 'video/ogg',
'.mov': 'video/quicktime',
'.avi': 'video/x-msvideo',
'.wmv': 'video/x-ms-wmv',
'.flv': 'video/x-flv',
'.mkv': 'video/x-matroska',
'.m4v': 'video/x-m4v',
'.3gp': 'video/3gpp',

// Аудио
'.mp3': 'audio/mpeg',
'.wav': 'audio/wav',
'.oga': 'audio/ogg',
'.m4a': 'audio/mp4',
'.aac': 'audio/aac',
'.flac': 'audio/flac',
'.wma': 'audio/x-ms-wma',
'.aiff': 'audio/aiff',
'.opus': 'audio/opus',

// CAD и чертежи
'.dwg': 'application/acad',
'.dxf': 'application/dxf',
'.dwf': 'application/x-dwf',
'.step': 'application/step',
'.stp': 'application/step',
'.iges': 'application/iges',
'.igs': 'application/iges',
'.stl': 'application/sla',
```

```
      // Архивы
      '.zip': 'application/zip',
      '.rar': 'application/x-rar-compressed',
      '.7z': 'application/x-7z-compressed',
      '.tar': 'application/x-tar',
      '.gz': 'application/gzip',
      '.bz2': 'application/x-bzip2',

      // Электронные книги
      '.epub': 'application/epub+zip',
      '.mobi': 'application/x-mobipocket-ebook',
      '.azw': 'application/vnd.amazon.ebook',
      '.fb2': 'application/x-fictionbook+xml',

      // Шрифты
      '.ttf': 'font/ttf',
      '.otf': 'font/otf',
      '.woff': 'font/woff',
      '.woff2': 'font/woff2',
      '.eot': 'application/vnd.ms-fontobject',
    };

    const contentType = mimeTypes[ext] || 'application/octet-stream';

    res.setHeader('Content-Type', contentType);
    res.setHeader('Content-Disposition', 'inline');
    res.sendFile(filePath);
  } catch (error) {
    console.error('Ошибка предпросмотра:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// GET /api/documents/:id/download - Скачивание документа
router.get('/:id/download', authMiddleware, async (req, res) => {
  try {
    const { id } = req.params;

    const result = await pool.query(
      'SELECT file_path, file_name, title, code FROM documents WHERE id =
$1',
      [id]
    );

    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Документ не найден' });
    }

    const document = result.rows[0];
    const filePath = path.join(__dirname, '../../uploads',
document.file_path);

    if (!fs.existsSync(filePath)) {
```

```
      return res.status(404).json({ error: 'Файл не найден' });
    }

    const fileName = document.file_name ||
`${document.code}${path.extname(document.file_path)}`;

    res.download(filePath, fileName);
  } catch (error) {
    console.error('Ошибка скачивания:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// GET /api/documents/:id/versions - История версий
router.get('/:id/versions', authMiddleware, async (req, res) => {
  try {
    const { id } = req.params;

    const result = await pool.query(`
      SELECT dv.id, dv.document_id, dv.version_no as version,
dv.file_path,
             dv.file_name, dv.file_size, dv.change_description,
             dv.created_at, u.full_name as created_by_name
      FROM document_versions dv
      LEFT JOIN users u ON dv.author_id = u.id
      WHERE dv.document_id = $1
      ORDER BY dv.version_no DESC
    `, [id]);

    res.json(result.rows);
  } catch (error) {
    console.error('Ошибка получения версий:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// GET /api/documents/:id/versions/:versionId/download - Скачивание
конкретной версии
router.get('/:id/versions/:versionId/download', authMiddleware, async
(req, res) => {
  try {
    const { id, versionId } = req.params;

    const result = await pool.query(
      `SELECT dv.file_path, dv.file_name, dv.version_no, d.code, d.title
       FROM document_versions dv
       JOIN documents d ON dv.document_id = d.id
       WHERE dv.id = $1 AND dv.document_id = $2`,
      [versionId, id]
    );

    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Версия не найдена' });
    }
```

```
    const version = result.rows[0];
    const filePath = path.join(__dirname, '../../uploads',
version.file_path);

    if (!fs.existsSync(filePath)) {
      return res.status(404).json({ error: 'Файл не найден' });
    }

    const fileName = version.file_name ||
`${version.code}_v${version.version_no}${path.extname(version.file_path)}
`;

    res.download(filePath, fileName);
  } catch (error) {
    console.error('Ошибка скачивания версии:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// GET /api/documents/:id/comments - Комментарии документа
router.get('/:id/comments', authMiddleware, async (req, res) => {
  try {
    const { id } = req.params;

    const result = await pool.query(`
      SELECT c.*, u.full_name as author_name
      FROM comments c
      LEFT JOIN users u ON c.user_id = u.id
      WHERE c.document_id = $1
      ORDER BY c.created_at DESC
    `, [id]);

    res.json(result.rows);
  } catch (error) {
    console.error('Ошибка получения комментариев:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// POST /api/documents/:id/comments - Добавить комментарий
router.post('/:id/comments', authMiddleware, async (req, res) => {
  try {
    const { id } = req.params;
    const { content } = req.body;
    const userId = req.user.id;

    if (!content || !content.trim()) {
      return res.status(400).json({ error: 'Комментарий не может быть
пустым' });
    }

    const result = await pool.query(
```

```
      'INSERT INTO comments (document_id, user_id, content) VALUES ($1,
$2, $3) RETURNING *',
      [id, userId, content.trim()]
    );

    res.status(201).json(result.rows[0]);
  } catch (error) {
    console.error('Ошибка добавления комментария:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// DELETE /api/documents/:id/comments/:commentId - Удалить комментарий
router.delete('/:id/comments/:commentId', authMiddleware, async (req,
res) => {
  try {
    const { commentId } = req.params;
    const userId = req.user.id;

    // Проверяем права (автор или админ)
    const comment = await pool.query('SELECT user_id FROM comments WHERE
id = $1', [commentId]);

    if (comment.rows.length === 0) {
      return res.status(404).json({ error: 'Комментарий не найден' });
    }

    if (comment.rows[0].user_id !== userId && req.user.role !== 'admin')
{
      return res.status(403).json({ error: 'Недостаточно прав' });
    }

    await pool.query('DELETE FROM comments WHERE id = $1', [commentId]);

    res.json({ message: 'Комментарий удален' });
  } catch (error) {
    console.error('Ошибка удаления комментария:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// POST /api/documents - Создание документа
router.post('/', authMiddleware, upload.single('file'), async (req, res)
=> {
  try {
    const { title, code, type, category_id, description, tags } =
req.body;
    const userId = req.user.id;

    // Проверяем права (только библиотекарь или админ)
    if (!['librarian', 'admin'].includes(req.user.role)) {
      return res.status(403).json({ error: 'Недостаточно прав' });
    }
```

```
    if (!req.file) {
      return res.status(400).json({ error: 'Файл не загружен' });
    }

    if (!title || !code || !type) {
      return res.status(400).json({ error: 'Заполните обязательные поля'
});
    }

    // Проверяем уникальность кода
    const existing = await pool.query('SELECT id FROM documents WHERE
code = $1', [code]);
    if (existing.rows.length > 0) {
      return res.status(400).json({ error: 'Документ с таким кодом уже
существует' });
    }

    const result = await pool.query(
      `INSERT INTO documents (title, code, type, category_id,
description, file_path, file_name, file_size, created_by, status,
version)
       VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, 'pending_approval', 1)
       RETURNING *`,
      [title, code, type, category_id || null, description,
req.file.filename, req.file.originalname, req.file.size, userId]
    );

    const documentId = result.rows[0].id;

    // Добавляем теги
    if (tags) {
      const tagList = JSON.parse(tags);
      for (const tagName of tagList) {
        // Создаем тег если не существует
        let tagResult = await pool.query('SELECT id FROM tags WHERE name
= $1', [tagName]);
        let tagId;

        if (tagResult.rows.length === 0) {
          const newTag = await pool.query('INSERT INTO tags (name) VALUES
($1) RETURNING id', [tagName]);
          tagId = newTag.rows[0].id;
        } else {
          tagId = tagResult.rows[0].id;
        }

        await pool.query('INSERT INTO document_tags (document_id, tag_id)
VALUES ($1, $2)', [documentId, tagId]);
      }
    }

    res.status(201).json(result.rows[0]);
  } catch (error) {
    console.error('Ошибка создания документа:', error);
```

```
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

// PUT /api/documents/:id - Обновление документа (новая версия)
router.put('/:id', authMiddleware, upload.single('file'), async (req,
res) => {
  const client = await pool.connect();

  try {
    const { id } = req.params;
    const { comment } = req.body;
    const userId = req.user.id;

    // Проверяем права (только библиотекарь или админ)
    if (!['librarian', 'admin'].includes(req.user.role)) {
      return res.status(403).json({ error: 'Недостаточно прав' });
    }

    // Получаем текущий документ
    const docResult = await client.query(
      'SELECT * FROM documents WHERE id = $1',
      [id]
    );

    if (docResult.rows.length === 0) {
      return res.status(404).json({ error: 'Документ не найден' });
    }

    const document = docResult.rows[0];

    // Проверяем, что документ в библиотеке
    if (document.status !== 'in_library') {
      return res.status(400).json({ error: 'Можно обновлять только
документы в библиотеке' });
    }

    if (!req.file) {
      return res.status(400).json({ error: 'Файл не загружен' });
    }

    await client.query('BEGIN');

    const currentVersion = parseInt(document.version) || 1;
    const newVersion = currentVersion + 1;

    // Используем имя файла из документа или генерируем из file_path
    const oldFileName = document.file_name || document.file_path ||
'unknown';
    const oldFileSize = document.file_size || 0;

    // Сохраняем текущую версию в историю
    await client.query(
```

```
      `INSERT INTO document_versions (document_id, version_no, file_path,
file_name, file_size, author_id, change_description)
       VALUES ($1, $2, $3, $4, $5, $6, $7)`,
      [id, currentVersion, document.file_path, oldFileName, oldFileSize,
userId, 'Предыдущая версия']
    );

    // Обновляем документ
    const newFilePath = req.file.filename;
    const newFileName = req.file.originalname;
    const newFileSize = req.file.size;

    await client.query(
      `UPDATE documents
       SET file_path = $1, file_name = $2, file_size = $3, version = $4,
updated_at = CURRENT_TIMESTAMP
       WHERE id = $5`,
      [newFilePath, newFileName, newFileSize, newVersion, id]
    );

    // Добавляем запись о новой версии
    await client.query(
      `INSERT INTO document_versions (document_id, version_no, file_path,
file_name, file_size, author_id, change_description)
       VALUES ($1, $2, $3, $4, $5, $6, $7)`,
      [id, newVersion, newFilePath, newFileName, newFileSize, userId,
comment || 'Новая версия']
    );

    await client.query('COMMIT');

    res.json({ message: 'Документ обновлен', version: newVersion });
  } catch (error) {
    await client.query('ROLLBACK');
    console.error('Ошибка обновления документа:', error);
    res.status(500).json({ error: 'Ошибка сервера: ' + error.message });
  } finally {
    client.release();
  }
});

// DELETE /api/documents/:id - Удаление документа
router.delete('/:id', authMiddleware, async (req, res) => {
  try {
    const { id } = req.params;

    // Проверяем права (только админ)
    if (req.user.role !== 'admin') {
      return res.status(403).json({ error: 'Недостаточно прав' });
    }

    await pool.query('DELETE FROM documents WHERE id = $1', [id]);

    res.json({ message: 'Документ удален' });
```

```
  } catch (error) {
    console.error('Ошибка удаления документа:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
});

module.exports = router;
```

server/src/routes/settings.js

```
const express = require('express');
const router = express.Router();
const settingsController = require('../controllers/settingsController');
const authMiddleware = require('../middleware/auth');
const roleMiddleware = require('../middleware/role');

router.use(authMiddleware);

// Публичные настройки - для всех авторизованных
router.get('/public', settingsController.getPublic);

// Все настройки - только для админа
router.get('/', roleMiddleware(['admin']), settingsController.getAll);
router.put('/', roleMiddleware(['admin']), settingsController.update);

module.exports = router;
```

server/src/routes/tickets.js

```
const express = require('express');
const router = express.Router();
const ticketController = require('../controllers/ticketController');
const authMiddleware = require('../middleware/auth');
const roleMiddleware = require('../middleware/role');

router.use(authMiddleware);
router.use(roleMiddleware(['librarian', 'department_head']));

router.get('/', ticketController.getAll);
router.get('/:id', ticketController.getById);
router.post('/:id/message', ticketController.addMessage);
router.post('/:id/approve', ticketController.approve);
router.post('/:id/reject', ticketController.reject);
router.post('/:id/request-changes', ticketController.requestChanges);

module.exports = router;
```

server/src/routes/users.js

```
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');
const authMiddleware = require('../middleware/auth');
const roleMiddleware = require('../middleware/role');
```

```
router.use(authMiddleware);

// Получить руководителей для выбора согласующих (для библиотекаря)
router.get('/approvers', async (req, res) => {
  try {
    const pool = require('../config/database');
    const result = await pool.query(`
      SELECT id, full_name, position, role
      FROM users
      WHERE role = 'department_head'
      ORDER BY full_name
    `);
    res.json(result.rows);
  } catch (error) {
    console.error('Ошибка получения руководителей:', error);
    res.status(500).json({ error: 'Ошибка получения списка' });
  }
});

// Аудит и аналитика
router.get('/audit', roleMiddleware(['admin']),
userController.getAuditLogs);
router.get('/analytics', roleMiddleware(['admin']),
userController.getAnalytics);

// CRUD - только для админа
router.get('/', roleMiddleware(['admin']), userController.getAll);
router.get('/:id', roleMiddleware(['admin']), userController.getById);
router.post('/', roleMiddleware(['admin']), userController.create);
router.put('/:id', roleMiddleware(['admin']), userController.update);
router.delete('/:id', roleMiddleware(['admin']), userController.delete);

module.exports = router;
```

server/src/services/auditService.js

```
import axios from 'axios';

const API_URL = import.meta.env.VITE_API_URL ||
'http://localhost:5000/api';

export const getApiUrl = () => API_URL.replace('/api', '');

const api = axios.create({
  baseURL: API_URL,
});

// Добавляем токен к каждому запросу
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
```

```
  return config;
});

// Обработка ошибок авторизации
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      localStorage.removeItem('user');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

// API для аутентификации
export const authApi = {
  login: (credentials) => api.post('/auth/login', credentials),
  register: (userData) => api.post('/auth/register', userData),
  me: () => api.get('/auth/me'),
};

// API для документов
export const documentsApi = {
  getAll: (params) => api.get('/documents', { params }),
  getById: (id) => api.get(`/documents/${id}`),
  create: (formData) => api.post('/documents', formData, {
    headers: { 'Content-Type': 'multipart/form-data' }
  }),
  update: (id, formData) => api.put(`/documents/${id}`, formData, {
    headers: { 'Content-Type': 'multipart/form-data' }
  }),
  delete: (id) => api.delete(`/documents/${id}`),
  download: (id) => api.get(`/documents/${id}/download`, { responseType:
'blob' }),

  // Версии
  getVersions: (id) => api.get(`/documents/${id}/versions`),
  downloadVersion: (documentId, versionId) =>
api.get(`/documents/${documentId}/versions/${versionId}/download`, {
    responseType: 'blob'
  }),

  // Комментарии
  getComments: (id) => api.get(`/documents/${id}/comments`),
  addComment: (id, content) => api.post(`/documents/${id}/comments`, {
content }),
  deleteComment: (documentId, commentId) =>
api.delete(`/documents/${documentId}/comments/${commentId}`),

  // Избранное
  getFavorites: () => api.get('/documents/favorites'),
  addToFavorites: (id) => api.post(`/documents/favorites/${id}`),
```

```
  removeFromFavorites: (id) => api.delete(`/documents/favorites/${id}`),

  // Согласование
  getPending: () => api.get('/documents', { params: { status:
'pending_approval' } }),
  approve: (id) => api.post(`/approval/${id}/approve`),
  reject: (id, reason) => api.post(`/approval/${id}/reject`, { reason }),
};

// API для категорий
export const categoriesApi = {
  getAll: () => api.get('/categories'),
  create: (data) => api.post('/categories', data),
  update: (id, data) => api.put(`/categories/${id}`, data),
  delete: (id) => api.delete(`/categories/${id}`),
};

// API для пользователей
export const usersApi = {
  getAll: () => api.get('/users'),
  getById: (id) => api.get(`/users/${id}`),
  create: (data) => api.post('/users', data),
  update: (id, data) => api.put(`/users/${id}`, data),
  delete: (id) => api.delete(`/users/${id}`),
};

export default api;
```

server/src/app.js

```
const express = require('express');
const cors = require('cors');
const path = require('path');
require('dotenv').config();

const app = express();

// ВАЖНО: Эти middleware должны быть ДО маршрутов!
app.use(cors());
app.use(express.json());  // <-- Это парсит JSON из тела запроса
app.use(express.urlencoded({ extended: true }));

// Статические файлы
app.use('/uploads', express.static(path.join(__dirname, '../uploads')));

// Маршруты (должны быть ПОСЛЕ middleware)
const authRoutes = require('./routes/auth');
const usersRoutes = require('./routes/users');
const documentsRoutes = require('./routes/documents');
const categoriesRoutes = require('./routes/categories');
const ticketsRoutes = require('./routes/tickets');
const settingsRoutes = require('./routes/settings');
const backupRoutes = require('./routes/backup');
```

```
app.use('/api/auth', authRoutes);
app.use('/api/users', usersRoutes);
app.use('/api/documents', documentsRoutes);
app.use('/api/categories', categoriesRoutes);
app.use('/api/tickets', ticketsRoutes);
app.use('/api/settings', settingsRoutes);
app.use('/api/backup', backupRoutes);

// Проверка здоровья сервера
app.get('/api/health', (req, res) => {
  res.json({ status: 'ok' });
});

// Запуск сервера
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Сервер запущен на порту ${PORT}`);
});
```